

VIETNAM NATIONAL UNIVERSITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



CAPSTONE PROJECT REPORT (CO4337)

INFORMATION RETRIEVAL
FOR LOW-RESOURCE LANGUAGES

Major: Computer Science

Committee:	Committee 2 Computer Science	
Instructors:	Nguyen An Khuong, PhD.	
	Pham Nhut Huy, Engineer	Zalo AI
Reviewer:	Le Hong Trang, Assoc. Prof.	
Students:	Trang Si Trong	2110621
	Do Tin Nghia	2111837
	Nguyen Thai Tan	2112256

Ho Chi Minh City, June, 2025

ĐẠI HỌC QUỐC GIA TP.HCM
TRƯỜNG ĐẠI HỌC BÁCH KHOA

HĐ 2 KHMT - 910
CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập - Tự do - Hạnh phúc

KHOA: KH & KT Máy tính
BỘ MÔN: KHMT

NHIỆM VỤ LUẬN VĂN/ ĐỒ ÁN TỐT NGHIỆP
Chú ý: Sinh viên phải dán tờ này vào trang nhất của bản thuyết trình

HỌ VÀ TÊN: Trang Sĩ Trọng
Đỗ Tín Nghĩa
Nguyễn Thái Tân
NGÀNH: Khoa học Máy Tính

MSSV: 2110621
MSSV: 2111837
MSSV: 2112256
LỚP: MT21KHTN

1. **Đầu đề luận văn/ đồ án tốt nghiệp:** "Truy vấn thông tin cho ngôn ngữ ít tài nguyên"

2. **Nhiệm vụ (yêu cầu về nội dung và số liệu ban đầu):**

- Thu thập và tiền xử lý dữ liệu.
- Hiện thực mô hình Truy vấn thông tin cho ngôn ngữ ít tài nguyên.
- Thiết kế các tiêu chuẩn đánh giá hiệu suất mô hình.
- Huấn luyện và kiểm thử mô hình.
- Viết báo cáo luận văn.

3. Ngày giao nhiệm vụ: 30/01/2023

4. Ngày hoàn thành nhiệm vụ: 09/06/2023

5. Họ tên giảng viên hướng dẫn:

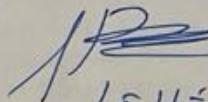
- 1) Nguyễn An Khương
- 2) Phạm Nhứt Huy

Phân hướng dẫn:

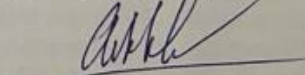
Hướng dẫn chung và giám sát thực hiện
Hướng dẫn kỹ thuật

Nội dung và yêu cầu LVTN/ ĐATN đã được thông qua Bộ môn.

Ngày tháng năm
CHỦ NHIỆM BỘ MÔN
(Ký và ghi rõ họ tên)


Lê Hồng Truy

GIẢNG VIÊN HƯỚNG DẪN CHÍNH
(Ký và ghi rõ họ tên)


Nguyễn An Khương

PHÂN DÀNH CHO KHOA, BỘ MÔN:

Người duyệt (chấm sơ bộ):

Đơn vị:

Ngày bảo vệ:

Điểm tổng kết:

Nơi lưu trữ LVTN/ĐATN:

Ngày 19 tháng 05 năm 2025

PHIẾU ĐÁNH GIÁ LUẬN VĂN/ ĐỒ ÁN TỐT NGHIỆP

(Dành cho người hướng dẫn/phản biện)

1. Họ và tên SV: Trang Sĩ Trọng MSSV: 2110621 Ngành (chuyên ngành): Khoa học Máy tính
Đỗ Tín Nghĩa MSSV: 2111837 Ngành (chuyên ngành): Khoa học Máy tính
Nguyễn Thái Tân MSSV: 2112256 Ngành (chuyên ngành): Khoa học Máy tính

2. Đề tài: "Truy vấn thông tin cho ngôn ngữ ít tài nguyên"

3. Họ tên người hướng dẫn/phản biện: Nguyễn An Khương và Phạm Nhựt Huy

4. Tổng quát về bản thuyết minh:

Số trang:

Số chương:

Số bảng số liệu

Số hình vẽ:

Số tài liệu tham khảo:

Phần mềm tính toán:

Hiện vật (sản phẩm)

5. Những ưu điểm chính của LV/ ĐATN:

- Đồ án thể hiện sự đầu tư nghiên cứu nghiêm túc qua việc học kỹ các kiến thức nền tảng đa dạng và chuyên sâu, từ lý thuyết Topology, Measure Theory, Optimal Transport cho đến các kiến trúc Transformer và thuật toán EM. Việc này tạo nền tảng vững chắc cho các phương pháp đề xuất. Nhóm sinh viên đã xây dựng được một quy trình huấn luyện (training pipeline) tương đối hoàn chỉnh, kết hợp nhiều kỹ thuật tiên tiến như Knowledge Distillation thông qua Optimal Transport, Query Expansion và kết hợp cả Lexical Matching lẫn Semantic Searching. Đặc biệt, việc áp dụng Optimal Transport để chưng cất kiến thức từ mô hình "teacher model" sang mô hình "student model" cho ngôn ngữ ít tài nguyên là một điểm sáng, cho thấy sự tìm tòi và cập nhật các kỹ thuật mới.
- Đồ án cũng cho thấy sự cẩn trọng trong việc xây dựng dữ liệu, bao gồm cả dữ liệu đơn ngữ và song ngữ cho tiếng Việt (lĩnh vực kinh tế) và tiếng Khmer (lĩnh vực văn hóa). Quy trình thực hiện đánh giá kết quả, dù còn hạn chế về tập dữ liệu (chỉ một tài liệu liên quan cho mỗi truy vấn), nhưng đã sử dụng các độ đo phù hợp như MRR, Accuracy@1 và Accuracy@5. Kết quả thực nghiệm bước đầu cho thấy mô hình hoạt động tốt, đặc biệt với truy vấn đã qua xử lý cho tiếng Việt. Báo cáo được trình bày có cấu trúc rõ ràng, các chương mục được phân chia hợp lý từ giới thiệu, kiến thức nền tảng, các công trình liên quan, phương pháp đề xuất, thực nghiệm và kết luận.

6. Những thiếu sót chính của LV/ĐATN:

- Một trong những hạn chế lớn nhất của đồ án là vấn đề về hiệu năng tính toán khi tính toán điểm tương đồng giữa một truy vấn và tất cả các đoạn (chunk) văn bản, điều này sẽ trở thành rào cản khi triển khai với tập dữ liệu lớn. Bên cạnh đó, việc đánh giá chỉ dựa trên một tài liệu liên quan duy nhất cho mỗi truy vấn khiến việc đo lường các chỉ số như Recall trở nên khó khăn và chưa phản ánh đầy đủ hiệu quả của hệ thống trong các tình huống thực tế cần nhiều kết quả. Mặc dù có đề cập đến việc mở rộng sang các ngôn ngữ ít tài nguyên khác, việc này trên thực tế đòi hỏi nhiều kiến thức ngôn ngữ học và tùy chỉnh tiền xử lý đáng kể.

- Phần mở rộng truy vấn dựa trên UnifiedStructPRF, dù có cơ sở lý thuyết, nhưng việc chỉ sử dụng BM25+ để chọn tài liệu liên quan ban đầu có thể chưa tối ưu hoàn toàn. Ngoài ra, việc kết hợp điểm từ truy vấn gốc và truy vấn mở rộng bằng cách giữ lại điểm cao hơn là một chiến lược còn đơn giản, có thể cần những phương pháp phức tạp hơn để tận dụng tốt hơn thông tin từ cả hai loại truy vấn. Mặc dù ĐA có đề cập đến việc fine-tuning mô hình ngôn ngữ, nhưng việc này được xem là tùy chọn và chưa được đào sâu phân tích hiệu quả cụ thể đối với các ngôn ngữ mục tiêu.

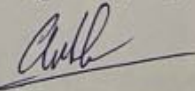
7. Đề nghị: Được bảo vệ ☒ Bổ sung thêm để bảo vệ ☐ Không được bảo vệ ☐

8. Các câu hỏi SV phải trả lời trước Hội đồng:

- Trong phần Semantic Searching, nhóm sử dụng công thức $P(rel|q,D) \approx 1 - \prod_{s \in D} (1 - P(rel|q,s))$ để tổng hợp độ tương đồng của các chunk. Công thức này có những ưu và nhược điểm gì so với các phương pháp tổng hợp khác như lấy trung bình (average) hoặc lấy giá trị lớn nhất (max)?
- Đồ án có đề cập đến việc fine-tuning mô hình ngôn ngữ (ví dụ mBERT, XLM-R) cho ngôn ngữ cụ thể hoặc chủ đề tài liệu cụ thể. Nhóm có thể chia sẻ kinh nghiệm về việc fine-tuning này (nếu có) và những khó khăn gặp phải, đặc biệt với ngôn ngữ Khmer?
- Kết quả thực nghiệm cho thấy hiệu năng giảm đáng kể khi bỏ câu đầu tiên của tài liệu, đặc biệt khi dùng truy vấn đã xử lý. Điều này có hàm ý gì về đặc điểm của tập dữ liệu hoặc khả năng khái quát hóa của mô hình? Nhóm có đề xuất giải pháp nào để cải thiện vấn đề này không?

9. Đánh giá chung (bằng chữ: Xuất sắc, Giỏi, Khá, TB): **Xuất sắc** Điểm: **10/10**

Ký tên (ghi rõ họ tên)



Nguyễn An Khương

Ngày 16 tháng 5 năm 2025

PHIẾU ĐÁNH GIÁ LUẬN VĂN/ ĐỒ ÁN TỐT NGHIỆP
(Dành cho người phân biện)

1. Họ và tên SV: *Đỗ Tín Nghĩa, Nguyễn Thái Tân, Trang Sĩ Trọng*
MSSV: 2111837, 2112256, 2110621 Ngành (chuyên ngành): Khoa học máy tính
2. Đề tài: **Truy vấn thông tin cho ngôn ngữ ít tài nguyên**
3. Họ tên người phân biện: *Lê Hồng Trang*
4. Tổng quát về bản thuyết minh:
Số trang: 89 Số chương: 7
Số bảng số liệu: 1 Số hình vẽ: 20
Số tài liệu tham khảo: 40 Phần mềm tính toán: 0
Hiện vật (sản phẩm): 0
5. Những ưu điểm chính của LV/ ĐATN:
 - Đồ án có bố cục và nội dung trình bày phù hợp, được viết bằng tiếng Anh.
 - Phương pháp đề xuất yêu cầu khá nhiều các kiến thức nền tảng liên quan, như Lý thuyết độ đo, Topology, Vận chuyển tối ưu, các Kiến trúc học sâu cho NLP và Phân cụm. Các nội dung trình bày cho thấy nhóm sinh viên đã tìm hiểu và nắm vững các kiến thức này.
 - Nhóm sinh viên đã nghiên cứu áp dụng kiến trúc chiết xuất (Distillation) với khối người dạy (teacher) là đại diện kiến thức cho tiếng Anh, và người học (student) đại diện cho các ngôn ngữ hạn chế về tài nguyên như tiếng Việt hay tiếng Khmer.
 - Phương pháp đề xuất được mô tả, hiện thực và thực nghiệm.
6. Những thiếu sót chính của LV/ĐATN:
 - Các kết quả thực nghiệm nên được phân tích và bình luận, cũng như so sánh với một số mô hình đã có để chỉ ra tính đúng đắn và mức độ hiệu quả của phương pháp đề xuất.
7. Đề nghị: Được bảo vệ ☒ Bổ sung thêm để bảo vệ ☐ Không được bảo vệ ☐
8. Các câu hỏi SV phải trả lời trước Hội đồng:
9. Đánh giá chung (bằng chữ: Xuất sắc, Giỏi, Khá, TB): *Giỏi* Điểm: 9.5/10

Ký tên (ghi rõ họ tên)



Lê Hồng Trang

Date: 10 June 2025

PhD. Nguyen An Khuong (Instructor)
Faculty of Computer Science and Engineering

Declaration

We hereby declare that this thesis was carried out by our team, under the supervision of Dr. Nguyen An Khuong and Mr. Pham Nhut Huy. The content presented in this thesis is original, with the exception of specific references in the text, and has not been previously submitted for any other academic organizations. All sources utilized in this thesis have been carefully selected and properly cited in the bibliography section. In the case of any instances of plagiarism, we shall take full responsibility before the Ho Chi Minh City University of Technology and the Faculty of Computer Science and Engineering.

Ho Chi Minh City, June 2025

Authors

Trang Si Trong

Do Tin Nghia

Nguyen Thai Tan

Acknowledgement

This specialized project is a major milestone for our graduation and would not have been possible without the support of many individuals. We would like to express our sincere gratitude to Dr. Nguyen An Khuong and Mr. Pham Nhut Huy, our dedicated supervisors, for their invaluable guidance, continuous support, and encouragement throughout this project. Their expertise have been instrumental in shaping the direction of our work.

We are also deeply thankful to the lecturers at Ho Chi Minh City University of Technology, especially those from the Faculty of Computer Science and Engineering, for imparting their knowledge and wisdom over the academic years.

Finally, we would like to extend our appreciation to our families and friends for their unwavering belief and for providing emotional support throughout this challenging journey at university.

Abstract

In our increasingly digital age, access to information has become a fundamental right. Unfortunately, language barriers often hinder this access, limiting individuals' ability to tap into the vast reservoir of knowledge and opportunities available online. This digital divide, particularly acute for those whose native languages are underrepresented, can lead to significant disadvantages in education, employment, and social participation. Models that enable querying information in these under-resourced languages offer a promising solution to bridge this linguistic gap. By enabling effective information retrieval in these underserved languages, individuals are able to access information in their preferred language, fostering inclusivity and equity. Furthermore, these models play a crucial role in preserving linguistic diversity, particularly for endangered languages. By facilitating the digitization and preservation of cultural heritage materials, these technologies help safeguard the unique cultural identities of minority groups.

Keywords: information retrieval, low-resource languages, knowledge distillation, optimal transport

Contents

Declaration	5
Acknowledgement	6
Abstract	7
1 Introduction	14
1.1 Motivation	14
1.2 Objectives	15
1.3 Scope	16
1.4 Challenges	16
1.5 Structure of the report	17
2 Preliminaries	18
2.1 Topology	18
2.1.1 Metric Space	18
2.1.2 Open Set	19
2.1.3 Topological Space	19
2.1.4 Closed Sets	20
2.2 Measure Theory	20
2.2.1 Algebra and σ -algebra	21
2.2.2 Measure and Measure Space	23
2.2.3 Null Set and Complete Measure	25
2.2.4 Lebesgue Measure	26
2.2.5 Simple Function	26
2.2.6 Measurable Function	27
2.2.7 Lebesgue Integral	29
2.3 Optimal Transport	30
2.3.1 The Monge Formulation	31
2.3.2 The Kantorovich Formulation	33
2.3.3 Optimal Transport on Discrete Distributions	35
2.4 Transformer Architecture	37

2.4.1	Positional Encoding	38
2.4.2	Attention Mechanisms	39
2.4.3	Transformer Architecture	41
2.5	Expectation-Maximization (EM) algorithm	42
2.5.1	Maximum Likelihood Estimation	42
2.5.2	EM Algorithm	43
3	Related Works	48
3.1	Term Weighting Approaches	48
3.2	Query Expansion	50
3.3	Neural Network Approaches	52
3.4	Knowledge Distillation	57
3.5	Optimal Transport	59
3.5.1	Entropic Regularization Approach	59
3.5.2	Inexact Proximal Point Method for Exact Optimal Transport Problem Approach	60
4	Proposed Approach	63
4.1	Problem Statement	63
4.2	Training Pipeline	64
4.2.1	Knowledge Distillation	66
4.2.2	Query Expansion	69
4.2.3	Lexical Matching	76
4.2.4	Semantic Searching	78
4.3	Proposed Model	83
5	Model Implementation	86
5.1	Data Construction	86
5.1.1	Language Corpus Datasets	86
5.1.2	Language Corpus Datasets Visualization	87
5.1.3	Cross-lingual Dataset	91
5.2	Training Pipeline Implementation	91
5.2.1	Optimal Transport Solver	91
5.2.2	Knowledge Distillation Class	91
5.2.3	Query Expansion	92
5.2.4	Lexical Matching	93
5.2.5	Chunk Separator	93
5.2.6	Custom Sentence Transformer	94
5.2.7	Monolingual Trainer Class	94

5.3 Retrieval Model Implementation	95
6 Experiment and results	97
6.1 Experiment Setup	97
6.1.1 Resource	97
6.1.2 Training and Testing Phase	97
6.2 Evaluation Metric	98
6.3 Experiment Results	99
6.3.1 Training Loss	99
6.3.2 Final Results	101
7 Conclusion	103
7.1 Achieved Results	103
7.2 Limitations	103
7.3 Future Research Directions	104

List of Figures

2.1	The Transformer model architecture [36]	41
3.1	Model structure of MatchPyramid [22]	53
3.2	Query relevance attentional neural network (QRANN) [39] model architecture. Each word in the query has an attention mechanism with the sentence to identify relevant spans, followed by a residual connection and layer normalization. After relevance matching, the outputs are fed to a feedforward layer to obtain relevance features of the entire query, which are used for final relevance estimation.	55
4.1	Pipeline for training process	64
4.2	Execution flow of the Knowledge Distillation component	67
4.3	Architecture and execution flow of Query Expansion component	69
4.4	Execution flow of Lexical Matching component	76
4.5	Execution flow of Semantic Searching component	79
4.6	SBERT architecture with classification objective function. The two BERT networks have tied weights (siamese network structure).	80
4.7	SBERT architecture at inference, for example, to compute similarity scores. This architecture is also used with the regression objective function.	80
4.8	The Triplet Loss minimizes the distance between an anchor and a positive, both of which have the same identity, and maximizes the distance between the anchor and a negative of a different identity.	80
4.9	Architecture and execution flow of our proposed model	84
5.1	Top 30 most frequent Vietnamese words without stopwords with translation	88
5.2	Top 30 most frequent Khmer words without stopwords with translation . . .	89
5.3	Top 30 most frequent Vietnamese words without stopwords with translation	90
5.4	Top 30 most frequent Khmer words without stopwords with translation . . .	90
6.1	Training loss for ‘Vietnamese with first sentence’ subsets	99
6.2	Training loss for ‘Khmer with first sentence’ subsets	100
6.3	Training loss for ‘Vietnamese without first sentence’ subsets	100

6.4	Training loss for ‘Khmer without first sentence’ subsets	101
-----	--	-----

List of Tables

6.1	Performance results	101
-----	-------------------------------	-----

Chapter 1

Introduction

1.1 Motivation

The field of Natural Language Processing (NLP) has made significant strides in recent years, revolutionizing various industries. However, these advancements have primarily benefited high-resource languages like English or Mandarin, leaving a vast majority of the world's 7,000 languages, often referred to as low-resource languages (LRLs), under-resourced.

Low-resource languages, characterized by limited linguistic resources, digital content, and dedicated research, face significant challenges in the digital age. These languages, often spoken by marginalized communities, risk digital marginalization and cultural erosion. The scarcity of training data, a critical component for building robust NLP models, poses a major hurdle. Without sufficient data, it becomes difficult to develop accurate and effective models, hindering the development of language technologies that could empower LRL communities.

Information Retrieval (IR) systems, which power search engines, chatbots, and question-answering systems, have become indispensable in our digital age. While these systems have achieved impressive performance on high-resource languages, their effectiveness diminishes significantly when applied to LRLs due to data scarcity and linguistic diversity. In particular, search engines often overlook or underperform in these languages due to their reliance on linguistics resources, large-scale data, and popularity, none of which are typically available for LRLs. As a result, important cultural-related content such as local knowledge, oral traditions, or specialized reports is often missing from or buried deep within these platforms.

Information Retrieval has evolved significantly over the years, transitioning from traditional term-based methods to sophisticated neural models. Early IR systems relied on keyword matching, which had limitations in capturing semantic and contextual nuances. Subsequent advancements, such as vector space models and statistical language models,

improved upon these limitations. However, the advent of deep learning has revolutionized the field, enabling the development of powerful models that can capture complex linguistic patterns and semantic relationships. Recent years have witnessed the application of various deep learning techniques to IR, including Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), and Transformer-based models like BERT, GPT-3, and GPT-4. These models have demonstrated impressive performance on a wide range of IR tasks, including information retrieval, question answering, and document summarization. However, their effectiveness is often contingent on the availability of large amounts of training data, which is a significant challenge for LRLs. A promising approach to mitigate the issue of data scarcity is Knowledge Distillation. Knowledge Distillation enables knowledge from a well-trained model in a high-resource language to be transferred to a model for a low-resource language. By doing so, we aim to improve the performance of the LRL model without requiring extensive training data. In addition, the motivation of this project expands further beyond improving technical performance, it positions IR as a tool for linguistic and cultural empowerment. By tailoring retrieval systems to culturally relevant domains, the system not only retrieves documents but also promotes visibility, relevance, and ongoing use of the language.

1.2 Objectives

This thesis aims to design and develop an information retrieval model that allows users to submit queries in a low-resource language, and retrieves relevant documents in that language. The specific objectives of this research are as follows

- Study state-of-the-art multilingual models, specifically query-document matching techniques, as well as knowledge distillation to enhance the retrieval result.
- Design and develop an Information Retrieval model that allows user to input a query in a low-resource language, and returns documents relevant to the query.
- Collect data from multiple languages, especially low-resource languages, and preprocess the collected data to match the input data structure required by the components in the architecture.
- Design a robust evaluation metric to assess the performance of the IR model in low-resource settings, focusing on relevance, and computational efficiency.
- Conduct experiment to evaluate the model’s performance and analyze its effectiveness based on the designed metric.

1.3 Scope

Given the vast number of low-resource languages and the wide range of potential query domains, this thesis narrows its focus to ensure feasibility and depth in the investigation. The scope of the thesis will be as described below

- Working with only text-based queries and documents, without extending to multi-modal retrieval (e.g., images or videos).
- Training and conducting experiment on two low-resource languages, Vietnamese and Khmer.
- Restricting the domain of queries in Vietnamese to the field of economy and queries in Khmer to the field of culture.

1.4 Challenges

Developing this model poses several challenges

- Data scarcity: Low-resource languages are characterized by the limited availability of digitized data and linguistic resources such as annotated corpora, dictionaries, or pre-trained embeddings. This scarcity makes acquiring a sufficient quantity and quality of data necessary for effective model development a challenging task. Data collection efforts often require manual curation, collaboration with native speakers, or the use of alternative techniques such as data augmentation or cross-lingual transfer, all of which can be resource-intensive and time-consuming.
- Strong computational infrastructure requirements: Training models for natural language processing tasks demands significant computational resources. This requirement is amplified when working with low-resource languages, as techniques such as pretraining on high-resource languages, transfer learning, or fine-tuning, are often employed, which involve processing large datasets and complex models. The lack of access to high-performance computing infrastructure can become a bottleneck in the development of such models.
- Knowledge requirements: Members of the research team must possess deep knowledge of linguistic principles to address language-specific challenges, a strong foundation in mathematics to understand and optimize complex algorithms, and familiarity with state-of-the-art models to leverage their strengths. Furthermore, it is necessary to conduct a review of existing approaches, analyzing their strengths and limitations.

1.5 Structure of the report

Based on the objectives and scope of the study, the structure of our thesis is as follow

Chapter 1: Introduction. This chapter covers the motivations that drive our research, scope of the study, objectives that we hope to achieve, and challenges faced when conducting the research.

Chapter 2: Preliminaries. In this chapter, we introduce the fundamental concepts and theoretical background necessary for understanding the subsequent chapters. Key definitions, mathematical formulations, and essential principles related to our research domain are presented. This chapter ensures that readers, regardless of their prior knowledge, can grasp the foundational elements required to comprehend the proposed methodology.

Chapter 3: Related works. This chapter provides a review of existing literature and previous research relevant to our study. We delve into algorithms designed to solve the Optimal Transport problem, and give the details of previous works on techniques used in information retrieval,

Chapter 4: Proposed approach. Here, we present our novel contributions, focusing on the proposed architecture for the training pipeline and retrieval model. Each component of the architecture is described in detail, including its design, execution flow, and purpose within the overall architecture.

Chapter 5: Implementation. This chapter describes the dataset used in our research and details the implementation of the main components in the training pipeline.

Chapter 6: Experiment and results. In this chapter, we evaluate the performance of our proposed method. Then, the results of our work is presented and analyzed.

Chapter 7: Conclusion. The conclusion summarizes the entire project, results we achieved, limitations and suggests potential directions for future researches.

Chapter 2

Preliminaries

This chapter is dedicated to the foundations that we found necessary to understand the proposed model. We introduce and give the properties or observations of these concepts.

2.1 Topology

Topology is a branch of mathematics that studies the properties of spaces that remain unchanged under continuous deformations such as stretching, twisting, or bending, but not tearing or gluing. It focuses on understanding the fundamental structure of geometric objects and their relationships, without concern for precise measurements like distance or angles. Key concepts in topology include open and closed sets, continuity, connectedness, and compactness. However, in this section, we only cover basic concepts necessary for measure theory and later, optimal transport. Unless otherwise stated, the definitions in this section are adapted from [1], while the accompanying examples were constructed by the student research team.

2.1.1 Metric Space

Definition 2.1.1. A *metric space* is a set equipped with a distance between every two points. Namely, a metric space is a set X with a map $d : X \times X \rightarrow \mathbb{R}$ such that for all $x, y, z \in X$:

- (a) $d(x, y) \geq 0$ (non-negativity)
- (b) $d(x, y) = 0$ if and only if $x = y$ (distance is zero iff the two points coincide)
- (c) $d(x, y) = d(y, x)$ (symmetry)
- (d) $d(x, z) \leq d(x, y) + d(y, z)$ (triangular inequality)

Example 2.1.2. One familiar metric is the distance of points in $\mathbb{R}^n$. In particular, given $x = (x_1, x_2, \dots, x_n)$ and $y = (y_1, y_2, \dots, y_n)$, the distance function d is defined as

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}.$$

2.1.2 Open Set

Definition 2.1.3. Given a metric space (X, d) . A *ball* B of radius r centered at a point $x \in X$ is a set of the form $B(x, r) = \{y \in X \mid d(x, y) < r\}$, where $r \in \mathbb{R}, r > 0$.

We observe that this ball encompasses all points whose distance from x is less than r .

Definition 2.1.4. A subset $\mathcal{U}$ of X is *open* if for every point $x \in \mathcal{U}$, there exists a positive real number ϵ such that $B(x, \epsilon)$ is contained in $\mathcal{U}$.

We see at once that a union of open sets is an open set. The intersection of two open sets is also an open set. Indeed, let A and B be open sets. Let $x \in A \cap B$. Since $x \in A$, there is a ball $B(x, r) \subset A$, and since $x \in B$, there is a ball $B(x, s) \subset B$. Let $t = \min\{r, s\}$, then $B(x, t) \subset A$ and $B(x, t) \subset B$, so $B(x, t) \subset A \cap B$. Thus, $A \cap B$ is open. By induction, we see that the intersection of any finite collection of open sets is an open set. However, an infinite intersection of open sets might not be an open sets, for example: $\bigcap_{n=1}^{\infty} B(x, \frac{1}{n}) = \{x\}$.

We also observe that the empty set is open since it doesn't have any point. The set X is also an open set since we can take any arbitrary positive real number r , by the definition of $B(x, r)$, all of its points are in X .

Example 2.1.5. Let $X = \mathbb{R}$. The interval $(0, 1)$ is open in X .

Proof. For every $x \in (0, 1)$, we define $r = \min\{x, 1 - x\}$. Consider the ball $B(x, r) = \{y \in \mathbb{R} \mid |x - y| < r\}$. The condition implies that $x - r < y < x + r$. By our choice of r , we have $x - r \geq 0$ and $x + r \leq 1$. Thus, $0 \leq x - r < y < x + r \leq 1$, which means that $y \in (0, 1)$. $\square$

2.1.3 Topological Space

Definition 2.1.6. A *topology* τ on a set X consists of subsets of X satisfying the following properties:

- i) The empty set $\emptyset$ and the set X both belongs to the topology.
- ii) The union of any collection of sets in τ is contained in τ .
- iii) The finite intersection of sets in τ is also contained in τ .

The elements of τ are called *open sets* of X in the topology τ

Definition 2.1.7. A set X together with a topology τ is called a *topological space*, denoted by (X, τ) , or just X if we do not need to specify the topology. An element of X is called a *point*.

Example 2.1.8. On any set X , one can always define the *trivial topology* $\{\emptyset, X\}$.

Example 2.1.9. Given the set $X = \{1, 2, 3\}$. The collection $\tau = \{\emptyset, \{1\}, \{2, 3\}, \{1, 2, 3\}\}$ is a topology on X . The open sets are $\emptyset, \{1\}, \{2, 3\}, \{1, 2, 3\}$.

2.1.4 Closed Sets

Definition 2.1.10. A set $\mathcal{U}$ is called a closed set if $\mathcal{U}^c$ is open.

Proposition 2.1.11. In a topological space X

- (a) $\emptyset$ and X are closed sets.
- (b) A finite union of closed sets is closed.
- (c) An intersection of closed sets is closed.

Proof.

Since $\emptyset = X^c$ and we have pointed out that $\emptyset$ and X are open sets, we have $\emptyset$ and X are closed sets.

For a collection of finite closed sets $(\mathcal{U}_i)_{i \in I}$, we have

$$\bigcup_{i \in I} \mathcal{U}_i = \left(\bigcap_{i \in I} \mathcal{U}_i^c \right)^c$$

As $\mathcal{U}_i^c$ are open sets, $\bigcap_{i \in I} \mathcal{U}_i^c$ is open and therefore, $\left(\bigcap_{i \in I} \mathcal{U}_i^c \right)^c$ is closed.

Similarly, we can prove that the intersection of closed sets is closed. □

Example 2.1.12. Given the set $X = \{1, 2, 3\}$ and a topology $\tau = \{\emptyset, \{1\}, \{2, 3\}, \{1, 2, 3\}\}$ on X . The collection of closed sets is $\{\emptyset^c, \{1\}^c, \{2, 3\}^c, \{1, 2, 3\}^c\}$ or equivalently $\{\{1, 2, 3\}, \{2, 3\}, \{1\}, \emptyset\}$.

2.2 Measure Theory

Measure theory is a branch of mathematics that provides a rigorous framework for understanding the concepts of size, length, area, and volume in both finite and infinite settings. It extends the idea of assigning “measures” to sets beyond simple geometric objects, allowing the systematic study of more complex or irregular sets. Central to measure theory

are the notions of σ -algebras, measurable functions, and integration, which generalize the Riemann integral to the more flexible Lebesgue integral. Unless otherwise stated, the definitions in this section are adapted from [20], while the accompanying examples were constructed by the student research team.

2.2.1 Algebra and σ -algebra

Definition 2.2.1. Let X be an arbitrary set. A collection $\mathcal{A}$ of subsets of X is an *algebra* on X if

- i) for each set S that belongs to $\mathcal{A}$, S^c belongs to $\mathcal{A}$,
- ii) for each finite sequence $S_1, \dots, S_n$ of sets that belong to $\mathcal{A}$, the set $\bigcup_{i=1}^n S_i$ belongs to $\mathcal{A}$.

Observe that $X = S \cup S^c$ for any $S \in \mathcal{A}$, so $X \in \mathcal{A}$. Since $\emptyset = X^c$, we also have $\emptyset \in \mathcal{A}$. From condition ii), we can infer that for each finite sequence $S_1, \dots, S_n$ of sets that belong to $\mathcal{A}$, the set $\bigcap_{i=1}^n S_i$ also belongs to $\mathcal{A}$ since

$$\bigcap_{i=1}^n S_i = \left(\bigcup_{i=1}^n S_i^c \right)^c.$$

Example 2.2.2. Given any non-empty set X . The collection $\mathcal{A} = \{\emptyset, X\}$ is a trivial algebra on X with the smallest cardinality among all algebras on X . The power set of X , denoted $\mathcal{P}(X)$ is also a trivial algebra on X with the largest cardinality among all algebras on X .

Example 2.2.3. Given the set $X = \{1, 2, 3\}$. The collection $\mathcal{A} = \{\emptyset, \{1\}, \{2, 3\}, \{1, 2, 3\}\}$ is an algebra on X .

Definition 2.2.4. Let X be an arbitrary set. A collection $\mathcal{A}$ of subsets of X is a σ -*algebra* on X if

- i) for each set S that belongs to $\mathcal{A}$, S^c belongs to $\mathcal{A}$ (closed under complement),
- ii) for $\{S_i\}_{i \in I}$ a countable collection of sets that belong to $\mathcal{A}$, the set $\bigcup_{i \in I} S_i$ belongs to $\mathcal{A}$ (closed under countable unions).

By reasoning similarly as we did in Definition 2.2.1, we can also conclude that $\bigcap_{i \in I} S_i \in \mathcal{A}$. Thus a σ -algebra on X is a family of subsets of X that contains X and is closed under complementation, under the formation of countable unions, and under the formation of countable intersections.

If $\mathcal{A}$ is a σ -algebra on the set X , it is sometimes convenient to call a subset of X *$\mathcal{A}$ -measurable* if it belongs to $\mathcal{A}$.

Example 2.2.5. For an arbitrary non-empty set X , one can always define the *trivial σ -algebra* $\mathcal{A} = \{\emptyset, X\}$.

Example 2.2.6. Let $\mathcal{A}$ be the collection of all finite disjoint unions of all intervals of the form $(-\infty, a], (a, b], (b, \infty), \emptyset, \mathbb{R}$. Then $\mathcal{A}$ is an algebra over $\mathbb{R}$, but not a σ -algebra because union of sets $\{(0, \frac{i-1}{i}]\}$ for all $i \geq 1$ is $(0, 1)$, which is not in $\mathcal{A}$.

Example 2.2.7. Given the set $X = \{1, 2, 3\}$. The collection $\mathcal{A} = \{\emptyset, \{1\}, \{2, 3\}, \{1, 2, 3\}\}$ is a σ -algebra on X .

Definition 2.2.8. For a non-empty set X and a collection $\mathcal{F}$ of subsets of X , the *σ -algebra generated by $\mathcal{F}$* is the smallest σ -algebra $\mathcal{A}$ such that $\mathcal{F} \subset \mathcal{A}$, and is denoted by $\mathcal{A}(\mathcal{F})$.

We can also define $\mathcal{A}(\mathcal{F})$ is the intersection of all σ -algebras containing $\mathcal{F}$. The following proposition will prove that these two definitions for $\mathcal{A}(\mathcal{F})$ is indeed equivalent.

Proposition 2.2.9. For a non-empty set X and a collection $\mathcal{F}$ of subsets of X , the smallest σ -algebra containing $\mathcal{F}$ is the intersection of all σ -algebra that contains $\mathcal{F}$.

Proof. To prove this proposition, we first need to prove that the intersection of two σ -algebras is also a σ -algebra. $\square$

Lemma 2.2.10. The intersection of two σ -algebras on X is also a σ -algebra on X .

Proof. Denote the two algebras as $\mathcal{M}$ and $\mathcal{N}$ and their intersection as $\mathcal{A}$.

For each set $S \in \mathcal{A}$, we have $S \in \mathcal{M}$ and $S \in \mathcal{N}$. Therefore, $S^c \in \mathcal{M}$ and $S^c \in \mathcal{N} \Rightarrow S^c \in \mathcal{M} \cap \mathcal{N} = \mathcal{A}$.

Consider an countable collection $\{S_i\}_{i \in I}$ of sets that belong to $\mathcal{A}$. As $\mathcal{A}$ is the intersection of $\mathcal{M}$ and $\mathcal{N}$, we also have $S_i \in \mathcal{M}$ and $S_i \in \mathcal{N}$, and thus $\bigcup_{i \in I} S_i$ also belongs to both $\mathcal{M}$ and $\mathcal{N}$ due to $\mathcal{M}$ and $\mathcal{N}$ being σ -algebras. Therefore $\bigcup_{i \in I} S_i$ belongs to $\mathcal{M} \cap \mathcal{N} = \mathcal{A}$.

We have proven that $\mathcal{A}$ satisfies the conditions in the definition. Therefore, we can conclude that $\mathcal{A}$ is a σ -algebra on X . $\square$

Back to the proof of the proposition, denote the smallest σ -algebra containing $\mathcal{F}$ as $\mathcal{M}$. Since there always exists at least one σ -algebra containing $\mathcal{F}$ (namely the power set of X : $\mathcal{F} \subset \mathcal{P}(X)$), $\mathcal{M}$ always exists.

Suppose that there exists another σ -algebra $\mathcal{N}$ that contains $\mathcal{F}$ but does not contain $\mathcal{M}$, we define $\mathcal{A}$ as $\mathcal{M} \cap \mathcal{N}$. As proven in the lemma, $\mathcal{A}$ is a σ -algebra. Furthermore, $\mathcal{A}$ also contains $\mathcal{F}$, since both $\mathcal{M}$ and $\mathcal{N}$ contain $\mathcal{F}$. Therefore $\mathcal{A}$ is a σ -algebra that contains $\mathcal{F}$. But since $\mathcal{A} \subset \mathcal{M}$, $\mathcal{M}$ can not be the smallest σ -algebra containing $\mathcal{F}$, which contradicts the definition of $\mathcal{M}$. Thus, all σ -algebras that contains $\mathcal{F}$ also contains $\mathcal{M}$. In other words, $\mathcal{M}$ is the intersection of all σ -algebras containing $\mathcal{F}$.

Example 2.2.11. Given the set $X = \{1, 2, 3\}$. The collection $\mathcal{C} = \{\{1\}\}$ generates the σ -algebra $\mathcal{F} = \{\emptyset, \{1\}, \{2, 3\}, \{1, 2, 3\}\}$.

Example 2.2.12. Suppose X is a metric space with metric d . Then we can consider the collection $\mathcal{F}$ of open subsets of X (if X is a general topological space we can consider the same collection). The σ -algebra $\mathcal{A}(\mathcal{F})$ generated by this collection is known as the *Borel σ -algebra* and is denoted $\mathcal{B}_X$.

2.2.2 Measure and Measure Space

Definition 2.2.13. Let X be a set, and let $\mathcal{A}$ be a σ -algebra on X . A function μ whose domain is the σ -algebra $\mathcal{A}$ and whose values belong to the extended half-line $[0, +\infty]$ is said to be *countably additive* if it satisfies

$$\mu\left(\bigcup_{i=1}^{\infty} A_i\right) = \sum_{i=1}^{\infty} \mu(A_i)$$

for a countable collection $\{A_i\}_{i \in I}$ of disjoint sets that belong to $\mathcal{A}$. A *measure* (or a *countably additive measure*) on $\mathcal{A}$ is a function $\mu : \mathcal{A} \rightarrow [0, +\infty]$ that satisfies $\mu(\emptyset) = 0$ and is countably additive.

If X is a set, $\mathcal{A}$ is a σ -algebra on X , and if μ is a measure on $\mathcal{A}$, then the triplet $(X, \mathcal{A}, \mu)$ is called a *measure space* and the elements of $\mathcal{A}$ are called *measurable sets*. Likewise, if X is a set and if $\mathcal{A}$ is a σ -algebra on X , then the pair $(X, \mathcal{A})$ is called a *measurable space*. If $(X, \mathcal{A}, \mu)$ is a measure space, then one can say that μ is a *measure on $(X, \mathcal{A})$* , or, if $\mathcal{A}$ is clear from context, a *measure on X* .

Example 2.2.14. Given a non-empty set X and a σ -algebra $\mathcal{A}$ on X , one can always define the zero measure $\mu(A) = 0$ for all $A \in \mathcal{A}$.

Example 2.2.15. Let $X = \mathbb{R}$, which we think of as a metric space equipped with the metric $d(x, y) = |x - y|$. Then let $\mathcal{B}_{\mathbb{R}}$ be the Borel σ -algebra in Example 2.2.12, and $F : \mathbb{R} \rightarrow \mathbb{R}$ be an increasing, continuous function. Then one can define a measure μ_F on $\mathcal{B}_{\mathbb{R}}$ by

$$\mu_F(S) := \inf \left\{ \sum_{j=1}^{\infty} [F(b_j) - F(a_j)] : \bigcup_{j=1}^{\infty} (a_j, b_j] \supseteq S \right\}.$$

That is, given a set $S \in \mathcal{B}_{\mathbb{R}}$, consider all the ways to cover S by countable unions of disjoint half-open intervals $\bigcup_{j=1}^{\infty} (a_j, b_j]$. This larger set is given the measure

$$\mu_F\left(\bigcup_{j=1}^{\infty} (a_j, b_j]\right) = \sum_{j=1}^{\infty} [F(b_j) - F(a_j)].$$

Taking the infimum of this quantity over all such coverings gives the measure for S . In particular, for any interval $(a, b]$, one has $\mu_F((a, b]) = F(b) - F(a)$. This equality also holds if $(a, b]$ is replaced with (a, b) , $[a, b]$, or $[a, b)$.

Example 2.2.16. Consider a measure that assigns the value of 1 to a set if a specific element x is in the set and the value of 0 if otherwise. This measure is also called the *Dirac measure*, denoted by δ_x . Mathematically, for a Borel measurable set S

$$\delta_x(S) = \begin{cases} 1, & \text{if } x \in S, \\ 0, & \text{otherwise.} \end{cases}$$

Theorem 2.2.17 (Properties of measures). *Let $(X, \mathcal{A}, \mu)$ be a measure space.*

(a) (**Monotonicity**) *If $S, T \in \mathcal{A}$ with $S \subseteq T$, then $\mu(S) \leq \mu(T)$.*

(b) (**Subadditivity**) *If $\{S_i\}_{i \in I} \subseteq \mathcal{A}$ is a countable collection of not necessarily disjoint subsets of X , then*

$$\mu \left(\bigcup_{i \in I} S_i \right) \leq \sum_{i \in I} \mu(S_i).$$

(c) (**Continuity from below**) *If $\{S_n\}_{n \in \mathbb{N}} \subseteq \mathcal{A}$ is an ascending chain $S_1 \subseteq S_2 \subseteq S_3 \subseteq \dots$, then*

$$\mu \left(\bigcup_{n=1}^{\infty} S_n \right) = \lim_{n \rightarrow \infty} \mu(S_n).$$

(d) (**Continuity from above**) *If $\{S_n\}_{n \in \mathbb{N}} \subseteq \mathcal{A}$ is a descending chain $S_1 \supseteq S_2 \supseteq S_3 \supseteq \dots$ and $\mu(S_1) < \infty$, then*

$$\mu \left(\bigcap_{n=1}^{\infty} S_n \right) = \lim_{n \rightarrow \infty} \mu(S_n).$$

Proposition 2.2.18 (Linear combination of measures). *Suppose $\mu_1, \mu_2, \dots, \mu_n$ are measures on a σ -algebra $\mathcal{A}$ of subsets of a set X , and $a_1, a_2, \dots, a_n$ is a non-negative sequence. The linear combination of these measures, denoted by*

$$\mu(A) = (a_1\mu_1 + a_2\mu_2 + \dots + a_n\mu_n)(A) = a_1\mu_1(A) + a_2\mu_2(A) + \dots + a_n\mu_n(A), \forall A \in \mathcal{A},$$

is a measure.

Proof.

First,

$$\mu(\emptyset) = a_1\mu_1(\emptyset) + a_2\mu_2(\emptyset) + \dots + a_n\mu_n(\emptyset) = 0 + 0 + \dots + 0 = 0.$$

For each infinite sequence $\{A_i\}$ of disjoint sets that belong to $\mathcal{A}$, we have

$$\begin{aligned}
\mu\left(\bigcup_{i=1}^{\infty} A_i\right) &= a_1\mu_1\left(\bigcup_{i=1}^{\infty} A_i\right) + a_2\mu_2\left(\bigcup_{i=1}^{\infty} A_i\right) + \dots + a_n\mu_n\left(\bigcup_{i=1}^{\infty} A_i\right) \\
&= a_1\sum_{i=1}^{\infty}\mu_1(A_i) + a_2\sum_{i=1}^{\infty}\mu_2(A_i) + \dots + a_n\sum_{i=1}^{\infty}\mu_n(A_i) \quad (\mu_1, \mu_2, \dots, \mu_n \text{ are countably additive}) \\
&= \sum_{i=1}^{\infty} a_1\mu_1(A_i) + \sum_{i=1}^{\infty} a_2\mu_2(A_i) + \dots + \sum_{i=1}^{\infty} a_n\mu_n(A_i) = \sum_{i=1}^{\infty} (a_1\mu_1(A_i) + a_2\mu_2(A_i) + \dots + a_n\mu_n(A_i)) \\
&= \sum_{i=1}^{\infty} (a_1\mu_1 + a_2\mu_2 + \dots + a_n\mu_n)(A_i) = \sum_{i=1}^{\infty} \mu(A_i).
\end{aligned}$$

. Therefore, the linear combination of measures is a measure. $\square$

2.2.3 Null Set and Complete Measure

Definition 2.2.19. Given a measure space $(X, \mathcal{A}, \mu)$, a subset $N \in \mathcal{A}$ such that $\mu(N) = 0$ is called a μ -null set (or just a *null set* when the measure is clear from the context). If some property is true for all points $x \in X$ except on a null set, we say the property holds μ -almost everywhere (or just *almost everywhere*).

Example 2.2.20. Let $X = \mathbb{R}$, and define the set $Y = \mathbb{Q} \cap [0, 1]$, where $\mathbb{Q}$ is the set of rational numbers. The rationals $\mathbb{Q}$ are countable. Hence Y , the subset of rationals in $[0, 1]$, is also countable. Any countable set, including Y , has a zero Lebesgue measure (a more detailed discussion over this concept will be given later in this section). Thus, Y is a null set.

Example 2.2.21. Let $f : [0, 1] \rightarrow \mathbb{R}$ be the function defined as

$$f(x) = \begin{cases} 1, & \text{if } x \in \mathbb{Q}, \\ 0, & \text{if } x \notin \mathbb{Q}. \end{cases}$$

Thus, $f(x) = 0$ almost everywhere on $[0, 1]$, because the set where $f(x) \neq 0$ is $\mathbb{Q} \cap [0, 1]$, which is a null set.

Because of the monotonicity condition, if $N \in \mathcal{A}$ is a null set and $S \subseteq N$ is also measurable ($S \in \mathcal{A}$), then $\mu(S) = 0$. However, it need not be the case that any subset of a null set is measurable. When this is the case, we have the following definition.

Definition 2.2.22. A measure space $(X, \mathcal{A}, \mu)$ is said to be *complete* if for any μ -null set N , all subsets $S \subseteq N$ are also measurable (and consequently also μ -null sets).

Just as with metric spaces, a measure space $(X, \mathcal{A}, \mu)$ that is not *a priori* complete has a *completion* $(X, \overline{\mathcal{A}}, \overline{\mu})$. That is, $(X, \overline{\mathcal{A}}, \overline{\mu})$ is a complete measure space such that $\mathcal{A} \subseteq \overline{\mathcal{A}}$

and $\bar{\mu}(S) = \mu(S)$ for all $S \in \mathcal{A}$. It is via this completion that we obtain the Lebesgue measure.

2.2.4 Lebesgue Measure

Definition 2.2.23. Define $F : \mathbb{R} \rightarrow \mathbb{R}$ by $F(x) = x$, and let μ_F be a measure as in Example 2.2.15. Let $(\mathbb{R}, \mathcal{L}, \lambda)$ be the completion of the measure space $(\mathbb{R}, \mathcal{B}_{\mathbb{R}}, \mu_F)$. Then λ is called the *Lebesgue measure*, and subsets $S \subseteq \mathbb{R}$ are said to be *Lebesgue measurable* if $S \in \mathcal{L}$.

Recall that for any half-open interval $(a, b]$, $\mu_F((a, b]) = F(b) - F(a) = b - a$ when $F(x) = x$. Since the Lebesgue measure extends this measure, we also have $\lambda((a, b]) = b - a$ (and similarly for $[a, b)$, $[a, b]$, and (a, b)). Thus, the Lebesgue measure is, for sufficiently nice subsets of $\mathbb{R}$ like the intervals, simply giving what we usually think of as the length. In particular, $m(\mathbb{R}) = \infty$ and $\lambda(\{x\}) = 0$ for any $x \in \mathbb{R}$.

Theorem 2.2.24. Any countable set has a Lebesgue measure of zero.

Proof. Let $\{x_i\}_{i=1}^{\infty}$ be an enumeration of the elements of S .

For any (strictly) positive real number ϵ , define A_i as an open cover for S ,

$$A_i = (x_i - 2^{-i}\epsilon, x_i + 2^{-i}\epsilon)$$

Then

$$S \subseteq \bigcup_{i=1}^{\infty} A_i,$$

and

$$\mu\left(\bigcup_{i=1}^{\infty} A_i\right) \leq \sum_{i=1}^{\infty} 2^{1-i}\epsilon = 2\epsilon.$$

Since our choice of ϵ was arbitrary, for any positive real z , we can construct a set T such that $S \subseteq T$ and $\mu(T) \leq z$.

Hence S has zero measure. □

2.2.5 Simple Function

Definition 2.2.25. For $S \subset \mathbb{R}$, the *characteristic (or indicator) function* of S is the function $\chi_S : \mathbb{R} \rightarrow \mathbb{R}$ defined by

$$\chi_S(x) = \begin{cases} 1, & \text{if } x \in S, \\ 0, & \text{otherwise.} \end{cases}$$

A function $\phi : \mathbb{R} \rightarrow \mathbb{R}$ is called a *simple function* if it is a finite linear combination of

characteristic functions; that is, there exists $n \in \mathbb{N}$, numbers $a_1, \dots, a_n \in \mathbb{R}$, and subsets $S_1, \dots, S_n \subset \mathbb{R}$ such that

$$\phi = \sum_{i=1}^n a_i \chi_{S_i}.$$

Observe that the numbers $a_1, \dots, a_n \in \mathbb{R}$ and subsets $S_1, \dots, S_n \subset \mathbb{R}$ need not be unique. For example,

$$1 \cdot \chi_{[0,1]} + 1 \cdot \chi_{[1/2,1]} = 1 \cdot \chi_{[0,1/2]} + 2 \cdot \chi_{[1/2,1]}.$$

But, if we insist that $a_1, \dots, a_n$ are distinct and $S_i = \phi^{-1}(a_i)$, then this decomposition is unique. Moreover, $S_1, \dots, S_n$ are *disjoint* subsets of $\mathbb{R}$. This is called the *standard representation* of ϕ , and in the above example is the expression on the right-hand side.

2.2.6 Measurable Function

Definition 2.2.26. Let $(X, \mathcal{A})$ be a measurable space. A function $f : X \rightarrow \mathbb{R}$ is said to be a *measurable function* if the set $f^{-1}((a, \infty)) = \{x \in X | f(x) > a\}$ (or the set $f^{-1}((-\infty, a)) = \{x \in X | f(x) < a\}$) is measurable for each $a \in \mathbb{R}$.

Equivalently, let $(X, \mathcal{A})$ be a measurable space. A function $f : X \rightarrow \mathbb{R}$ is said to be a *measurable function* if for every $E \in \mathcal{B}(\mathbb{R})$, $f^{-1}(E) \in \mathcal{A}$.

Example 2.2.27. A constant function $f(x) = c$ for some $c \in \mathbb{R}$ is a measurable function, since for any $E \in \mathcal{B}(\mathbb{R})$, $f^{-1}(E) = X$ if $c \in E$, $f^{-1}(E) = \emptyset$ if $c \notin E$, and both $X, \emptyset$ are measurable.

Definition 2.2.28. A function $f : \mathbb{R} \rightarrow \mathbb{R}$ is said to be *Lebesgue* (resp. *Borel*) *measurable* if for every $S \in \mathcal{B}_{\mathbb{R}}$, $f^{-1}(S) \in \mathcal{L}$ (resp. $f^{-1}(S) \in \mathcal{B}_{\mathbb{R}}$).

Since $\mathcal{B}_{\mathbb{R}} \subset \mathcal{L}$, all Borel measurable functions are Lebesgue measurable. For a characteristic function χ_S or a subset $S \subset \mathbb{R}$, note that for $T \subset \mathbb{R}$ we have

$$\chi_S^{-1}(T) = \begin{cases} \mathbb{R}, & \text{if } 0, 1 \in T, \\ S, & \text{if } 1 \in T, \text{ but } 0 \notin T, \\ S^c, & \text{if } 0 \in T, \text{ but } 1 \notin T, \\ \emptyset, & \text{otherwise.} \end{cases}$$

Consequently, χ_S is Lebesgue (resp. Borel) measurable if and only if $S \in \mathcal{L}$ (resp. $S \in \mathcal{B}_{\mathbb{R}}$). Similarly, a simple function $\phi = \sum_{i=1}^n a_i \chi_{S_i}$ is Lebesgue (resp. Borel) measurable if and only if $S_1, \dots, S_n \in \mathcal{L}$ (resp. $S_1, \dots, S_n \in \mathcal{B}_{\mathbb{R}}$).

The following proposition indicates that the set of measurable functions is closed under the usual arithmetic and limit operations.

Proposition 2.2.29. *Let $f, g : \mathbb{R} \rightarrow \mathbb{R}$ be measurable functions. Then $f + g$, $f - g$, $f \cdot g$, and f/g are measurable.*

Proof. We have

$$\{x \in X : f(x) + g(x) > a\} = \{x \in X : f(x) > a - g(x)\}$$

$$= \bigcup_{r \in \mathbb{Q}} (\{x \in X : a - g(x) < r\} \cap \{x \in X : f(x) > r\}) \quad (\text{there's always a } r \in \mathbb{Q} \text{ between two real numbers})$$

is a countable union of measurable sets, so it is also measurable. Therefore, $f + g$ is measurable.

If $c = 0$, then $cf(x) = 0$ is a measurable function.

If $c > 0$, then

$$\{x \in X : cf(x) > a\} = \{x \in X : f(x) > \frac{a}{c}\}$$

is measurable because f is measurable.

If $c < 0$, then

$$\{x \in X : cf(x) > a\} = \{x \in X : f(x) < \frac{a}{c}\}$$

is measurable because f is measurable.

Therefore, cf is measurable. Then $f - g = f + (-1)g$ is also measurable.

If $a < 0$, we have

$$\{x \in X : f^2(x) > a\} = X$$

is measurable.

If $a \geq 0$, we have

$$\{x \in X : f^2(x) > a\} = \{x \in X : f(x) > \sqrt{a}\} \cup \{x \in X : f(x) < -\sqrt{a}\}$$

is a countable union of measurable sets, so it is also measurable.

Therefore, f^2 is measurable.

We have

$$fg = \frac{1}{4} ((f + g)^2 - (f - g)^2)$$

Therefore, $f \cdot g$ is also measurable.

If $a = 0$, we have

$$\{x \in X : \frac{1}{f(x)} > a\} = \{x \in X : f(x) > 0\}$$

is measurable.

If $a > 0$, we have

$$\{x \in X : \frac{1}{f(x)} > a\} = \{x \in X : f(x) > 0\} \cap \{x \in X : f(x) < \frac{1}{a}\}$$

is a countable intersection of measurable sets, so it is also measurable.

If $a < 0$, we have

$$\{x \in X : \frac{1}{f(x)} > a\} = \{x \in X : f(x) > 0\} \cup \{x \in X : f(x) < \frac{1}{a}\}$$

is a countable union of measurable sets, so it is also measurable.

Then, $1/g$ is measurable. Therefore, $f/g = f \cdot (1/g)$ is measurable. $\square$

2.2.7 Lebesgue Integral

We first need to work with *positive* Lebesgue measurable functions before discussing Lebesgue integral. We denote L^+ to be the set of Lebesgue measurable functions $f : \mathbb{R} \rightarrow [0, \infty]$.

Definition 2.2.30. For a simple function $\phi \in L^+$ with standard representation $\sum_{i=1}^n a_i \chi_{S_i}$ (note that $a_1, \dots, a_n \in [0, \infty]$), its *Lebesgue integral* is defined as the (possibly infinite) quantity

$$\int_{\mathbb{R}} \phi d\lambda := \sum_{i=1}^n a_i \lambda(S_i).$$

Definition 2.2.31. For a function $f \in L^+$, its Lebesgue integral is defined as the (possibly infinite) quantity

$$\int_{\mathbb{R}} f d\lambda := \sup \left\{ \int_{\mathbb{R}} \phi d\lambda : \phi \leq f, \phi \in L^+ \text{ simple} \right\}.$$

Given a Lebesgue measurable function $f : \mathbb{R} \rightarrow \mathbb{R}$, consider the disjoint subsets

$$S_+ := f^{-1}([0, \infty)), \quad S_- := f^{-1}((-\infty, 0)).$$

Since the intervals $[0, \infty)$ and $(-\infty, 0)$ are Borel measurable, $S_+, S_- \in \mathcal{L}$ by virtue of f being Lebesgue measurable. Hence by Proposition 2.2.29, the functions $f_+ := f \cdot \chi_{S_+}$ and $f_- := -f \cdot \chi_{S_-}$ are Lebesgue measurable. These functions are called the *positive* and *negative* parts of f , respectively. By definition, we have

$$f = f_+ - f_- \quad \text{and} \quad f_+, f_- \in L^+.$$

Also note that $|f| = f_+ + f_-$.

Example 2.2.32. Consider the function defined on $[0, 2]$

$$f(x) = \begin{cases} 1, & \text{if } x \in [0, 1], \\ 2, & \text{if } x \in (1, 2]. \end{cases}$$

The Lebesgue integral of $f(x)$ over $[0, 2]$ is computed as

$$\int_{[0,2]} f(x)dx = 1 \cdot \mu([0, 1]) + 2 \cdot \mu((1, 2]) = 1 \cdot (1 - 0) + 2 \cdot (2 - 1) = 3.$$

Example 2.2.33. Let $f : \mathbb{R} \rightarrow \mathbb{R}$ (often called Dirichlet function) be the function defined as

$$f(x) = \begin{cases} 1, & \text{if } x \in \mathbb{Q}, \\ 0, & \text{if } x \notin \mathbb{Q}. \end{cases}$$

The Dirichlet function is discontinuous everywhere in $[0, 1]$, so it cannot be Riemann integrable. In contrast, the Lebesgue integral can handle this situation. The set where $f(x) = 1$ is the set of rational numbers in $[0, 1]$, which is countable. The Lebesgue measure of this countable set is zero. The set where $f(x) = 0$ is the irrationals, which is uncountable, but its measure is 1 (the length of the interval). Therefore,

$$\int_0^1 f(x)dx = 1 \cdot \text{measure of rationals} + 0 \cdot \text{measure of irrationals} = 1 \cdot 0 + 0 \cdot 1 = 0.$$

Proposition 2.2.34. Let $f : \mathbb{R} \rightarrow \mathbb{R}$ be a measurable function. If one of $\int_{\mathbb{R}} f_+ d\lambda$ or $\int_{\mathbb{R}} f_- d\lambda$ is finite, we define the Lebesgue integral of f to be the quantity

$$\int_{\mathbb{R}} f d\lambda = \int_{\mathbb{R}} f_+ d\lambda - \int_{\mathbb{R}} f_- d\lambda.$$

We say f is *Lebesgue integrable* if both $\int_{\mathbb{R}} f_+ d\lambda$ and $\int_{\mathbb{R}} f_- d\lambda$ are finite. Equivalently, since $|f| = f_+ + f_-$, f is Lebesgue integrable if $\int_{\mathbb{R}} |f| d\lambda$ is finite, in which case $|\int_{\mathbb{R}} f d\lambda| \leq \int_{\mathbb{R}} |f| d\lambda$ by the triangle inequality.

Theorem 2.2.35. For $a, b \in \mathbb{R}$ with $a < b$, let $f : [a, b] \rightarrow \mathbb{R}$ be a bounded function.

a. If f is Riemann integrable, then f is Lebesgue measurable and

$$\int_a^b f(x) dx = \int_{\mathbb{R}} f \cdot \chi_{[a,b]} d\lambda.$$

b. f is Riemann integrable if and only if $\{x \in [a, b] \mid f \text{ is discontinuous at } x\}$ is an m -null set (Lebesgue's integrability condition).

2.3 Optimal Transport

Optimal transport dates back to Gaspard Monge in 1781, with significant advancements by Leonid Kantorovich in 1942 and Yann Brenier in 1987. Currently optimal transport enjoys applications in image retrieval, signal and image representation, inverse problems,

cancer detection, texture and colour modelling, shape and image registration, and machine learning, to name a few.

In optimal transport, we assume that both measures are probability measures on spaces Ω_s and Ω_t respectively. Let $c : \Omega_s \times \Omega_t \rightarrow \mathbb{R}$ be a cost function where $c(x, y)$ measures the cost of transporting one unit of mass from $x \in \Omega_s$ to $y \in \Omega_t$. The optimal transport problem is how to transport μ_s to μ_t whilst minimizing the cost c . Unless otherwise stated, the definitions in this section are adapted from [32], while the accompanying examples were constructed by the student research team.

2.3.1 The Monge Formulation

Definition 2.3.1. Let $(X, \mathcal{A}_1, \alpha)$ be a measure space, $(Y, \mathcal{A}_2)$ a measurable space, and $T : X \rightarrow Y$ a measurable map. Then the function $\beta = T\#\alpha$ on $\mathcal{A}_2$ is called a *pushforward measure* if

$$\beta(B) = \alpha(T^{-1}(B)) \quad \forall B \in \mathcal{A}_2.$$

The $T\#$ notation is called *pushforward operator*.

Proposition 2.3.2.

1. The pushforward measure is a measure.
2. The pushforward operator reserves total mass: $T\#\alpha(Y) = \alpha(X)$.
3. Linearity: $T\#(\alpha_1 + \alpha_2)(B) = T\#\alpha_1(B) + T\#\alpha_2(B)$.

Proof.

1. We have

$$T\#\alpha(B) = \alpha(T^{-1}(B)) \geq 0 \quad (\text{because } \alpha \text{ is a measure}),$$

and

$$T\#\alpha(\emptyset) = \alpha(T^{-1}(\emptyset)) = \alpha(\emptyset) = 0 \quad (\text{preimage of empty set is empty set}).$$

For a countable collection $\{A_i\}_{i \in I}$ of disjoint sets that belong to $\mathcal{A}_1$,

$$T^{-1}(A_i) \cap T^{-1}(A_j) = T^{-1}(A_i \cap A_j) = T^{-1}(\emptyset) = \emptyset.$$

Therefore, $\{T^{-1}(A_i)\}_{i \in I}$ is pairwise disjoint.

Then we have,

$$\begin{aligned} T\#\alpha \left(\bigcup_{i=1}^{\infty} A_i \right) &= \alpha \left(T^{-1} \left(\bigcup_{i=1}^{\infty} A_i \right) \right) = \alpha \left(\bigcup_{i=1}^{\infty} T^{-1}(A_i) \right) \\ &= \sum_{i=1}^{\infty} \alpha(T^{-1}(A_i)) = \sum_{i=1}^{\infty} T\#\alpha(A_i). \end{aligned}$$

Then we can conclude that $T\#\alpha$ is a measure.

2. Because T maps X to Y , so that the preimage of the entire space Y is the entire space X , or $T^{-1}(Y) = X$. Therefore,

$$T\#\alpha(Y) = \alpha(T^{-1}(Y)) = \alpha(X).$$

3. By the definition of the pushforward measure,

$$T\#\alpha_1(B) = \alpha_1(T^{-1}(B)),$$

$$T\#\alpha_2(B) = \alpha_2(T^{-1}(B)),$$

$$T\#(\alpha_1 + \alpha_2)(B) = (\alpha_1 + \alpha_2)(T^{-1}(B)).$$

By the definition of linear combination of measures,

$$(\alpha_1 + \alpha_2)(T^{-1}(B)) = \alpha_1(T^{-1}(B)) + \alpha_2(T^{-1}(B)) \text{ or}$$

$$T\#(\alpha_1 + \alpha_2)(B) = T\#\alpha_1(B) + T\#\alpha_2(B).$$

□

Example 2.3.3. Suppose $X = [0, 1]$, $Y = \mathbb{R}$ and we are given the uniform measure α on X , which means that for any interval $A \subset [0, 1]$, the measure of A is simply the length of the interval,

$$\alpha(A) = \text{length}(A).$$

Define a mapping $T : X \rightarrow Y$, by $T(x) = 2x$. For any set $B \subset Y$, the preimage $T^{-1}(B)$ is the set of all $x \in [0, 1]$ such that

$$T^{-1}(B) = \{x \in [0, 1] : 2x \in B\}.$$

Therefore, the pushforward measure β on Y is given by

$$\beta(B) = \alpha(T^{-1}(B)) = \alpha(\{x \in [0, 1] : 2x \in B\}).$$

With the definition of α , this is simply the length of the set $[0, 1] \cap C$ where $C = \left\{\frac{y}{2} : y \in B\right\}$. Therefore, we have

$$\beta(B) = \text{length}([0, 1] \cap C).$$

For instance, suppose $B = [0, 10] \subset Y = \mathbb{R}$, we have $C = \left\{\frac{y}{2} : y \in [0, 10]\right\} = [0, 5]$, or

$$\beta(B) = \text{length}([0, 5] \cap [0, 1]) = \text{length}([0, 1]) = \alpha([0, 1]) = 1.$$

Definition 2.3.4. The Monge formulation aims at finding a mapping $T : \Omega_s \rightarrow \Omega_t$

$$\inf_{T \# \mu_s = \mu_t} \int_{\Omega_s} c(x, T(x)) \mu_s(x) dx.$$

For smooth measures, $\mu_s = \rho(x)dx$ and $\mu_t = \eta(x)dx$

$$T \# \mu_s = u_t \equiv \rho(T(x)) |\det(\partial T(X))| = \eta(x).$$

Example 2.3.5. Suppose we have

The source measure μ_s is a probability measure defined on the interval $[0, 1]$, where the density function is $d\mu_s(x) = 6x(1-x)dx$;

The target measure μ_t is a probability measure defined on the interval $[2, 3]$, where the density function is $d\mu_t(y) = 3(y-2)^2 dy$.

The transportation cost is given by the squared Euclidean distance $c(x, y) = (x - y)^2$.

The problem is to minimize

$$\min_{T: [0,1] \rightarrow [2,3]} \int_0^1 (x - T(x))^2 \mu_s(x) dx.$$

Proposition 2.3.6 (Properties of mapping T).

1. Existence of T is not guaranteed.
2. Unicity of T is not guaranteed.

Proof. We prove these properties by giving a counterexample

1. Consider $\mu_s = \delta_0$ (the Dirac measure at 0) and $\mu_t = \frac{1}{2}\delta_1 + \frac{1}{2}\delta_2$. Then it is obviously that there is no map can push the mass at a single point to two different points, or equivalently there is no transport map T that pushes μ_s to μ_t .

2. Consider $u_s = \frac{1}{2}\delta_0 + \frac{1}{2}\delta_1$ (the measure is a mixture of two Dirac deltas, at $x = 0$ and $x = 1$) and $u_t = \frac{1}{2}\delta_2 + \frac{1}{2}\delta_3$ (the measure is a mixture of two Dirac deltas, at $y = 2$ and $y = 3$).

There are several ways to find a map T that pushes the mass at $x = 0$ to $y = 2$ and the mass at $x = 1$ to $y = 3$. For example, both $T(x) = 2x + 2$ and $T(x) = 2x^2 + 2$ satisfy the constraint $T(0) = 2$ and $T(1) = 3$. $\square$

2.3.2 The Kantorovich Formulation

The key idea of Kantorovich is to relax the deterministic nature of transportation, namely the fact that a source point x_i can only be assigned to another point $T(x_i)$ only. Kantorovich proposes instead that the mass at any point x_i be potentially dispatched across several locations. Kantorovich moves away from the idea that mass transportation should

be *deterministic* to consider instead a *probabilistic* transport, which allows what is commonly known now as *mass splitting* from a source toward several targets.

Definition 2.3.7. The *Wasserstein distance*, often referred to as the Earth Mover's Distance (EMD), is a metric used to quantify the distance between two probability distributions over a given space. The *Generalized Wasserstein distance* between two probability measures μ_s and μ_t on metric spaces Ω_s and Ω_t is defined as

$$W_p(\mu_s, \mu_t) = \left(\inf_{\gamma \in P} \int_{\Omega_s \times \Omega_t} d(x, y)^p \gamma(x, y) dx dy \right)^{\frac{1}{p}},$$

where $d(x, y)$ is a distance metric, $\gamma(x, y)$ is the joint probability distribution (coupling) between μ_s and μ_t , the infimum is taken over all possible couplings γ between the distributions μ_s and μ_t .

Wasserstein Distance is a metric, which means it satisfies

1. Non-negativity: $W_p(\mu_s, \mu_t) \geq 0$,
2. Symmetry: $W_p(\mu_s, \mu_t) = W_p(\mu_t, \mu_s)$,
3. Triangle inequality: $W_p(\mu_s, \mu_t) \leq W_p(\mu_s, \mu_z) + W_p(\mu_z, \mu_t)$.

Proof.

1. Since $d(x, y)$ is a distance function, it is always non-negative, i.e. $d(x, y) \geq 0, \forall x, y$. Furthermore, $\gamma(x, y)$ is a probability measure, so it must satisfy $\gamma(x, y) \geq 0, \forall x, y$. Therefore, the integral $\int_{\Omega_s \times \Omega_t} d(x, y) \gamma(x, y) dx dy$ is always non-negative, and since we are taking the infimum over all possible couplings, we have

$$W_p(\mu_s, \mu_t) \geq 0.$$

$W_p(\mu_s, \mu_t) = 0$ if and only if $\mu_s = \mu_t$.

2. The distance function symmetry is trivial. Moreover, since the coupling $\gamma(x, y)$ is a joint distribution over $\Omega_s \times \Omega_t$, we can swap x and y ,

$$W_p(\mu_s, \mu_t) = \left(\inf_{\gamma \in P} \int_{\Omega_s \times \Omega_t} d(x, y)^p \gamma(x, y) dx dy \right)^{\frac{1}{p}} = \left(\inf_{\gamma \in P} \int_{\Omega_s \times \Omega_t} d(y, x)^p \gamma(y, x) dy dx \right)^{\frac{1}{p}} = W_p(\mu_t, \mu_s).$$

3. Consider any couplings γ_{sz} between μ_s and μ_z , γ_{zt} between μ_z and μ_t . We will construct a coupling γ_{st} between μ_s and μ_t by *composing* these couplings, i.e., we transport mass from μ_s to μ_z using γ_{sz} , and then from μ_z to μ_t using γ_{zt} .

Let γ_{st} be the *composition* of these couplings, defined on $\Omega_s \times \Omega_t$ as

$$\gamma_{st}(x, y) = \int_{\Omega_z} \gamma_{sz}(x, z) \gamma_{zt}(z, y) dz.$$

We have

$$\begin{aligned} W_p(\mu_s, \mu_t) &\leq \left(\int_{\Omega_s \times \Omega_t} d(x, y)^p \gamma_{st}(x, y) dx dy \right)^{\frac{1}{p}} = \left(\int_{\Omega_s \times \Omega_t} \int_{\Omega_z} d(x, y)^p \gamma_{sz}(x, z) \gamma_{zt}(z, y) dz dx dy \right)^{\frac{1}{p}} \\ &= \left(\int_{\Omega_s \times \Omega_t \times \Omega_z} d(x, y)^p \gamma_{sz}(x, z) \gamma_{zt}(z, y) dx dy dz \right)^{\frac{1}{p}}. \end{aligned}$$

Using the Minkowski Inequality, we have

$$\begin{aligned} W_p(\mu_s, \mu_t) &\leq \left(\int_{\Omega_s \times \Omega_t \times \Omega_z} d(x, z)^p \gamma_{sz}(x, z) \gamma_{zt}(z, y) dx dy dz \right)^{\frac{1}{p}} + \left(\int_{\Omega_s \times \Omega_t \times \Omega_z} d(z, y)^p \gamma_{sz}(x, z) \gamma_{zt}(z, y) dx dy dz \right)^{\frac{1}{p}} \\ &= \left(\int_{\Omega_s \times \Omega_z} d(x, z)^p \gamma_{sz}(x, z) dx dz \right)^{\frac{1}{p}} + \left(\int_{\Omega_t \times \Omega_z} d(z, y)^p \gamma_{zt}(z, y) dz dy \right)^{\frac{1}{p}} \\ &= W_p(\mu_s, \mu_z) + W_p(\mu_z, \mu_t). \end{aligned}$$

Therefore

$$W_p(\mu_s, \mu_t) \leq W_p(\mu_s, \mu_z) + W_p(\mu_z, \mu_t).$$

The Wasserstein distance is essentially a specific form of the Kantorovich optimal transport problem with the cost function being a distance metric. The Kantorovich formulation of optimal transport is more general, allowing arbitrary cost functions, but the Wasserstein distance is specifically about transporting mass with a cost proportional to the distance between points. $\square$

Definition 2.3.8. *The Kantorovich formulation seeks for a probabilistic coupling $\gamma \in \mathcal{P}(\Omega_s \times \Omega_t)$ between Ω_s and Ω_t such that*

$$\begin{aligned} \gamma_0 &= \operatorname{argmin}_{\gamma} \int_{\Omega_s \times \Omega_t} c(x, y) \gamma(x, y) dx dy, \\ \gamma &\in \mathcal{P} : \left\{ \gamma \geq 0, \int_{\Omega_t} \gamma(x, y) dy = \mu_s, \int_{\Omega_s} \gamma(x, y) dx = \mu_t \right\}. \end{aligned}$$

2.3.3 Optimal Transport on Discrete Distributions

Definition 2.3.9. Using the previous Kantorovich formulation, when $\mu_s = \sum_{i=1}^{n_s} a_i \delta_{x_i^s}$ and $\mu_t = \sum_{i=1}^{n_t} b_i \delta_{x_i^t}$, the *Optimal Transport on discrete distributions problem* is to find the optimal transport plan γ_0 such that

$$\gamma_0 = \operatorname{argmin}_{\gamma \in \mathcal{P}} \left\{ \langle \gamma, \mathbf{C} \rangle_F = \sum_{i,j} \gamma_{i,j} c_{i,j} \right\}.$$

where $\mathbf{C}$ is a cost matrix with $c_{i,j} = c(x_i^s, x_j^t)$ and the marginals constraints are

$$\mathcal{P} = \left\{ \gamma \in (\mathbb{R}^+)^{n_s \times n_t} \mid \gamma 1_{n_t} = a, \gamma^T 1_{n_s} = b \right\}.$$

The Kantorovich problem on discrete distributions mentioned is a constrained convex minimization problem, and as such, it can be naturally paired with a so-called dual problem, which is a constrained concave maximization problem. Next, we construct the dual problem from the primal problem.

To handle the constraints in the primal problem, we define the Lagrangian

$$\mathcal{L}(\gamma, \alpha, \beta) = \sum_{i,j} \gamma_{i,j} c_{i,j} + \sum_i \alpha_i \left(a_i - \sum_j \gamma_{i,j} \right) + \sum_j \beta_j \left(b_j - \sum_i \gamma_{i,j} \right).$$

Simplify the Lagrangian, we have

$$\begin{aligned} \mathcal{L}(\gamma, \alpha, \beta) &= \sum_{i,j} \gamma_{i,j} c_{i,j} - \sum_{i,j} \gamma_{i,j} \alpha_i - \sum_{i,j} \gamma_{i,j} \beta_j + \sum_i \alpha_i a_i + \sum_j \beta_j b_j \\ &= \sum_{i,j} \gamma_{i,j} (c_{i,j} - \alpha_i - \beta_j) + \sum_i \alpha_i a_i + \sum_j \beta_j b_j. \end{aligned}$$

The dual function is obtained by minimizing the Lagrangian $L(\gamma, \alpha, \beta)$ with respect to $\gamma_{i,j} \geq 0$. We also remark that

$$\min_{\gamma_{i,j} \geq 0} \gamma_{i,j} (c_{i,j} - \alpha_i - \beta_j) = \begin{cases} 0, & \text{if } c_{i,j} - \alpha_i - \beta_j \geq 0, \\ -\infty, & \text{otherwise.} \end{cases}$$

Therefore, the dual constraint becomes

$$c_{i,j} - \alpha_i - \beta_j \geq 0 \Leftrightarrow \alpha_i + \beta_j \leq c_{i,j} \quad \forall i, j.$$

Finally, we have the *Discrete OT dual formulation*

$$\begin{aligned} &\max_{\alpha \in \mathbb{R}^{n_s}, \beta \in \mathbb{R}^{n_t}} \alpha^T a + \beta^T b, \\ &s.t \quad \alpha_i + \beta_j \leq c_{i,j} \quad \forall i, j. \end{aligned}$$

This Discrete OT dual formulation can be solved as a Linear program with $n_s + n_t$ variables and $n_s n_t$ constraints.

Example 2.3.10. Consider we have two simple sets of word embeddings, each consisting of two words. These word embeddings are points in a 1-dimensional vector space. Let's consider the transportation cost is given by the squared Euclidean distance $c(x, y) = (x - y)^2$.

The source words *apple* and *banana* has word embeddings $u_1 = (1, 2)$ and $u_2 = (2, 1)$, respectively, the source measure μ_s is uniform, so $\mu_s(\{1, 2\}) = \frac{1}{2}$ for each.

The target words *fruit* and *vegetable* has word embeddings $v_1 = (3, 3)$ and $v_2 = (4, 4)$, respectively, the target measure μ_t is uniform, so $\mu_t(\{1, 2\}) = \frac{1}{2}$ for each.

The cost matrix C is given by

- from $u_1 = (1, 2)$ to $v_1 = (3, 3)$, $c(u_1, v_1) = (1 - 3)^2 + (2 - 3)^2 = 5$,
- from $u_2 = (2, 1)$ to $v_1 = (3, 3)$, $c(u_2, v_1) = (2 - 3)^2 + (1 - 3)^2 = 5$,
- from $u_1 = (1, 2)$ to $v_2 = (4, 4)$, $c(u_1, v_2) = (1 - 4)^2 + (2 - 4)^2 = 13$,
- from $u_2 = (2, 1)$ to $v_2 = (4, 4)$, $c(u_2, v_2) = (2 - 4)^2 + (1 - 4)^2 = 13$.

Thus, the cost matrix C is $\begin{pmatrix} 5 & 13 \\ 5 & 13 \end{pmatrix}$.

Let the transport plan be represented by the matrix P , where $P_{i,j}$ represents the amount of mass transported from source word i to target word j . The objective is to minimize the total transportation cost

$$\sum_i \sum_j P_{i,j} \cdot c(u_i, v_j).$$

with the constraints

$$\sum_j P_{1,j} = \frac{1}{2}, \sum_j P_{2,j} = \frac{1}{2}, \sum_i P_{i,1} = \frac{1}{2}, \sum_i P_{i,2} = \frac{1}{2}.$$

For simplicity, assume that the optimal transport plan is as follows (this is an intuitive guess for the case where the transport plan is symmetric and balanced), $P = \begin{pmatrix} \frac{1}{2} & 0 \\ 0 & \frac{1}{2} \end{pmatrix}$.

Therefore, the total cost is $\frac{1}{2} \cdot 5 + 0 \cdot 13 + 0 \cdot 5 + \frac{1}{2} \cdot 13 = 9$.

2.4 Transformer Architecture

The Transformer [36] is a revolutionary neural network architecture introduced by Vaswani et al. in 2017 to handle sequential data. Instead of processing input sequentially like traditional recurrent models like RNNs and LSTMs, the Transformer relies entirely on an Attention Mechanism, specifically the Self-Attention Mechanism, to capture dependencies between tokens regardless of their distance in the input sequence. The other key component of the Transformer is Positional Encoding, which retain the order of input sequences. These features make Transformers highly efficient, enabling them to power state-of-the-art models, especially in natural language processing area, such as BERT, GPT.

2.4.1 Positional Encoding

Positional Encoding is a significant component of Transformer models that allows them to capture the sequential order of input tokens. Recurrent Neural Networks (RNN) process input data sequentially, which means they lack inherent knowledge of token positions. Positional Encoding addresses this by adding positional information to the input token embeddings, enabling the model to differentiate between tokens based on their order. The method modifies each token $\mathbf{x}_n$, $n \in \{1, \dots, N\}$ element-wise to reflect positional information by

$$x_n^{(d)} \leftarrow x_n^{(d)} + \text{PE}(n, d),$$

where $\text{PE}(n, d)$ is a positional encoding function, for $n \in \{1, \dots, N\}$ and $d \in \{1, \dots, D\}$. In Transformer, the authors hypothesized that the effect of each pair of tokens to each other depends on the distance between them, i.e., for each $m \in \{1, \dots, N\}$, there exists a linear transformation $\Phi^{(m)}$ such that

$$\Phi^{(m)} \mathbf{x}_n = \mathbf{x}_{n+m},$$

where $n \in \mathbb{N}$ satisfying $0 \leq n + m \leq N$. Hence, given that D is even, they decided to use the sinusoid for positional encoding

$$\text{PE}(n, d) = \begin{cases} \sin(\omega_k \cdot n), & \text{if } d = 2k, \\ \cos(\omega_k \cdot n), & \text{if } d = 2k + 1, \end{cases}$$

where $\omega_k = \frac{1}{10000^{2k/D}}$.

Indeed, The function effectively satisfies their requirement. Consider two tokens $\mathbf{x}_n$ and $\mathbf{x}_{n+m}$, each at two dimensions $2k$ and $2k + 1$. We aim to find a matrix $M_k^{(m)} = \begin{pmatrix} u1 & u2 \\ v1 & v2 \end{pmatrix} \in \mathbb{R}^{2 \times 2}$ such that

$$M \begin{pmatrix} \sin(\omega_k \cdot n) \\ \cos(\omega_k \cdot n) \end{pmatrix} = \begin{pmatrix} \sin(\omega_k \cdot (n + m)) \\ \cos(\omega_k \cdot (n + m)) \end{pmatrix}.$$

Since

$$\sin(\omega_k \cdot (n + m)) = \sin(\omega_k \cdot n) \cos(\omega_k \cdot m) + \cos(\omega_k \cdot n) \sin(\omega_k \cdot m),$$

$$\cos(\omega_k \cdot (n + m)) = \cos(\omega_k \cdot n) \cos(\omega_k \cdot m) - \sin(\omega_k \cdot n) \sin(\omega_k \cdot m).$$

Plug the expansion into the previous equation, we get the solution

$$M_k^{(m)} = \begin{pmatrix} \cos(\omega_k \cdot m) & \sin(\omega_k \cdot m) \\ -\sin(\omega_k \cdot m) & \cos(\omega_k \cdot m) \end{pmatrix},$$

which only depends on m . Therefore, we have

$$\begin{pmatrix} M_1^{(m)} & 0 & \cdots & 0 \\ 0 & \ddots & \cdots & 0 \\ 0 & \cdots & 0 & M_{D/2}^{(m)} \end{pmatrix} \mathbf{x}_n = \mathbf{x}_{n+m}.$$

Example 2.4.1. Consider the sentence “I am happy”, we assume that each word is embedded into a four-dimensional vector space, i.e. a word of position k is represented by a vector $v_k = (v_{k0}, v_{k1}, v_{k2}, v_{k3})$. Consider the word at position 0, “i”. Then

$$PE(0, 0) = \sin 0 = 0, \quad PE(0, 1) = \cos 0 = 1,$$

$$PE(0, 2) = \sin 0 = 0, \quad PE(0, 3) = \cos 0 = 1.$$

And the positional encoding of the word “I” is

$$(v_{00}, v_{01}, v_{02}, v_{03}) + (0, 1, 0, 1).$$

Similarly,

$$PE(1, 0) = \sin \frac{1}{10000^{2 \cdot 0/4}} = \sin 1, \quad PE(1, 1) = \cos \frac{1}{10000^{2 \cdot 0/4}} = \cos 1,$$

$$PE(1, 2) = \sin \frac{1}{10000^{2 \cdot 1/4}} = \sin \frac{1}{100}, \quad PE(1, 3) = \cos \frac{1}{10000^{2 \cdot 1/4}} = \cos \frac{1}{100}.$$

The positional encoding of the word “am” is

$$(v_{10}, v_{11}, v_{12}, v_{13}) + \left(\sin 1, \cos 1, \sin \frac{1}{100}, \cos \frac{1}{100} \right).$$

And finally,

$$PE(2, 0) = \sin \left(\frac{1}{10000^{2 \cdot 0/4}} \cdot 2 \right) = \sin 2, \quad PE(2, 1) = \cos \left(\frac{1}{10000^{2 \cdot 0/4}} \cdot 2 \right) = \cos 2,$$

$$PE(2, 2) = \sin \left(\frac{1}{10000^{2 \cdot 1/4}} \cdot 2 \right) = \sin \frac{1}{50}, \quad PE(2, 3) = \cos \left(\frac{1}{10000^{2 \cdot 1/4}} \cdot 2 \right) = \cos \frac{1}{50}.$$

The positional encoding of the word “happy” is

$$(v_{20}, v_{21}, v_{22}, v_{23}) + \left(\sin 2, \cos 2, \sin \frac{1}{50}, \cos \frac{1}{50} \right).$$

2.4.2 Attention Mechanisms

Attention mechanisms are a powerful concept in neural networks that enable models to focus on the most relevant parts of the input when making predictions. Attention

dynamically assigns weights to different input elements based on their importance to the current context. This allows the model to capture long-range dependencies effectively.

Mathematically, a context includes a set of tokens after positional encoding, say $\mathbf{Y} = (y_1, \dots, y_N) \in \mathbb{R}^{D \times N}$. The tokens correspond to the values $\mathbf{V} = (v_1, \dots, v_N) \in \mathbb{R}^{D \times N}$, derived from keys $\mathbf{K} = (k_1, \dots, k_N) \in \mathbb{R}^{d_k \times N}$. Each token is also associated with a query $\mathbf{q}_n, n \in \{1, \dots, N\}$, collectively denoted as $\mathbf{Q} = (q_1, \dots, q_N)$. A query has a stronger influence on a token $\mathbf{y}_n, n \in \{1, \dots, N\}$, if it highly matches the corresponding key $\mathbf{k}_n$. The alignment is quantified using the dot product. Consequently, the dimensions of queries and keys are selected to be the same. For each query $\mathbf{q}_n, n \in \{1, \dots, N\}$, the authors computed the matching scores $\mathbf{q}_n \mathbf{K}^\top$, then normalized into softmax $\left(\frac{\mathbf{q}_n \mathbf{K}^\top}{\sqrt{d}} \right)$.

For simplicity, it can be written as

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{Q} \mathbf{K}^\top}{\sqrt{d}} \right) \mathbf{V}.$$

The authors used three learnable matrices $\mathbf{W}_v \in \mathbb{R}^{D \times D}$, $\mathbf{W}_k \in \mathbb{R}^{d_k \times D}$, and $\mathbf{W}_q \in \mathbb{R}^{d_k \times D}$ to extract the values, keys, and queries from the tokens, i.e.

$$\mathbf{V} = \mathbf{W}_v \mathbf{Y}, \quad \mathbf{K} = \mathbf{W}_k \mathbf{Y}, \quad \mathbf{Q} = \mathbf{W}_q \mathbf{Y}.$$

In practice, the authors utilized *Multi-head Attention*. Instead of a single value matrix, they employed $h := \frac{D}{d_v}$ value matrices $\mathbf{W}_v \in \mathbb{R}^{D \times d_v}$, where d_v is a divisor of D , yielding h d_v -dimensional output tokens in parallel. These outputs are then concatenated to form D -dimensional tokens. Additionally, keys and queries can be generated from two distinct sequences, which are called *Cross Attention*.

Example 2.4.2. Reconsider the sentence “I am happy” as in the Positional Encoding example. For simplicity, we assume after encoding, the word embeddings are respectively $(0.1, 0.2, 0.3, 0.4)$, $(0.5, 0.6, 0.7, 0.8)$, and $(0.9, 1.0, 1.1, 1.2)$.

We also use the same matrix for queries, keys and values.

The attention scores are computed as

$$\mathbf{s} = \left(\frac{\mathbf{Q} \mathbf{K}^\top}{\sqrt{d}} \right) = \frac{1}{\sqrt{4}} \begin{pmatrix} 0.1 & 0.2 & 0.3 & 0.4 \\ 0.5 & 0.6 & 0.7 & 0.8 \\ 0.9 & 1.0 & 1.1 & 1.2 \end{pmatrix} \begin{pmatrix} 0.1 & 0.5 & 0.9 \\ 0.2 & 0.6 & 1.0 \\ 0.3 & 0.7 & 1.1 \\ 0.4 & 0.8 & 1.2 \end{pmatrix} = \begin{pmatrix} 0.15 & 0.35 & 0.55 \\ 0.35 & 0.87 & 1.39 \\ 0.55 & 1.39 & 2.23 \end{pmatrix}.$$

Softmax is applied row-wise to normalize the scores

$$\text{softmax}(\mathbf{S}) = \begin{pmatrix} 0.2693 & 0.3289 & 0.4018 \\ 0.1814 & 0.3052 & 0.5134 \\ 0.1152 & 0.2668 & 0.6180 \end{pmatrix}.$$

The attention output is computed as

$$\begin{aligned} \text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) &= \text{softmax}(\mathbf{S}) \mathbf{V} = \begin{pmatrix} 0.2693 & 0.3289 & 0.4018 \\ 0.1814 & 0.3052 & 0.5134 \\ 0.1152 & 0.2668 & 0.6180 \end{pmatrix} \begin{pmatrix} 0.1 & 0.2 & 0.3 & 0.4 \\ 0.5 & 0.6 & 0.7 & 0.8 \\ 0.9 & 1.0 & 1.1 & 1.2 \end{pmatrix} \\ &= \begin{pmatrix} 0.5530 & 0.6530 & 0.7530 & 0.8530 \\ 0.6328 & 0.7328 & 0.8328 & 0.9328 \\ 0.7011 & 0.8011 & 0.9011 & 1.1011 \end{pmatrix}. \end{aligned}$$

2.4.3 Transformer Architecture

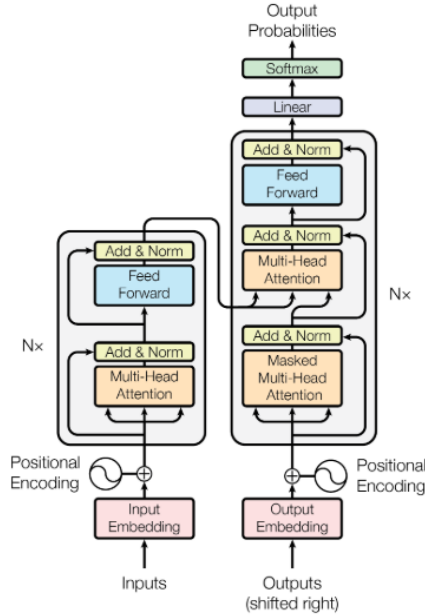


Figure 2.1: The Transformer model architecture [36]

As illustrated in Figure 2.1, the Transformer is based on an encoder-decoder structure.

- The encoder consists of N blocks, each built up by a multi-head attention layer followed by a position-wise feedforward layer, with residual connections and layer normalization enhancing stability and learning efficiency. The encoder processes the input sequence and generates contextual representations for each token.

- The decoder takes the sequence of right-shifted tokens $(\mathbf{x}_2, \dots, \mathbf{x}_{N+1})$, applying a multi-head attention with masking. These masked tokens are then passed through N identical blocks as the encoder, together with the output of the encoder. This interaction enables the decoder to incorporate both the context from the encoder and the generated sequence.
- The final output of the decoder is a probability distribution over the vocabulary for predicting the next token. A softmax layer is typically used.

The token $\mathbf{y}_n, n \in \{1, \dots, N\}$ is then modified by a linear combination of the values, weighted by the normalized matching scores

$$\mathbf{y}_n \leftarrow \mathbf{y}_n + \text{softmax} \left(\frac{\mathbf{q}_n \mathbf{K}^\top}{\sqrt{d}} \right) \mathbf{v}.$$

2.5 Expectation-Maximization (EM) algorithm

The Expectation-Maximization (EM) algorithm is a broadly applicable approach to the iterative computation of maximum likelihood (ML) estimates, useful in a variety of incomplete-data problems. The EM algorithm was proposed in 1977 by Dempster et al. [9]. Since then the EM algorithm has been applied in a staggering variety of general statistical problems such as resolution of mixtures, multiway contingency tables, variance components estimation, factor analysis, as well as in specialized applications in such areas as genetics, medical imaging, and neural networks.

2.5.1 Maximum Likelihood Estimation

Maximum likelihood estimation and likelihood-based inference are of central importance in statistical theory and data analysis. Maximum likelihood estimation is a general-purpose method with attractive properties. Maximum likelihood estimation is an ubiquitous technique and is used extensively in every area where statistical techniques are used.

We assume that the observed data y has probability density function (p.d.f.) $g(y; \psi)$, where ψ is the vector containing the unknown parameters in the postulated form for the p.d.f. of Y . Our objective is to maximize the likelihood $L(\psi) = g(y; \psi)$ as a function of ψ , over the parameter space Ω . That is,

$$\frac{\partial L(\psi)}{\partial \psi} = 0,$$

or equivalently, on the log likelihood,

$$\frac{\partial \log L(\psi)}{\partial \psi} = 0.$$

Example 2.5.1. Consider a scenario where the data $y_1, y_2, \dots, y_n$ comes from a normal distribution with unknown mean μ and known variance σ^2 . The p.d.f. of a single observation is

$$g(y_i; \mu) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - \mu)^2}{2\sigma^2}\right).$$

For n independent observations, the likelihood function is,

$$L(\mu) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - \mu)^2}{2\sigma^2}\right).$$

We have,

$$\log L(\mu) = -\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - \mu)^2.$$

To find the MLE, take the derivative of $\log L(\mu)$ with respect to μ and set it to zero:

$$\begin{aligned} \frac{\partial \log L(\mu)}{\partial \mu} &= 0 \\ \Leftrightarrow \frac{1}{\sigma^2} \left(\sum_{i=1}^n y_i - n\mu \right) &= 0 \\ \Leftrightarrow \sum_{i=1}^n y_i - n\mu = 0 &\Leftrightarrow \mu = \frac{\sum_{i=1}^n y_i}{n}. \end{aligned}$$

For example, consider we have $n = 5$ observations, $y_1 = 3, y_2 = 5, y_3 = 4, y_4 = 6, y_5 = 2$. The MLE for the mean is $\mu = \frac{1}{5}(3 + 5 + 4 + 6 + 2) = 4$, which represents the best estimate of the true mean based on the observed data.

2.5.2 EM Algorithm

In the application of statistical methods, one is often faced with the problem of estimation of parameters when the likelihood function is complicated in structure (such as grouped, censored or truncated data, multivariate data with some missing observations, multiway frequency data with a complex cell probability structure, and data from mixtures of distributions) resulting in difficult-to-compute (analytical or computational or both) maximization problems.

In many cases, solving a statistical problem becomes easier by introducing additional data, known as *augmented data*. This makes it possible to compute the Maximum Like-

likelihood Estimation (MLE) in a simpler and more efficient way. The augmented part of the data could also be called *missing data*. The augmented data is called *complete data*, while the original data is called *incomplete data*. The likelihood functions based on these datasets are termed *complete-data* likelihood and *incomplete-data* likelihood, respectively. Similarly, the process of finding the MLE using the complete data is the *complete-data problem*, while using the original data is the *incomplete-data problem*.

The EM algorithm is an iterative algorithm, in each iteration of which there are two steps, the Expectation Step (E-step) and the Maximization Step (M-step).

Within the incomplete-data framework of the EM algorithm, let x denote the vector containing the complete data and let y denote the vector containing the observed data. The complete-data probability density function (p.d.f.) is denoted by $g_c(x; \psi)$, where ψ represents the parameters of the model.

The complete-data log-likelihood function, which we could compute if the complete data x were fully observable, is given by

$$\log L_c(\psi) = \log g_c(x; \psi).$$

Since the complete data x is unobservable, the EM algorithm approaches the problem of maximizing the incomplete-data likelihood function indirectly. Instead of directly maximizing the incomplete-data log-likelihood $\log L(\psi) = \log g(y; \psi)$, the algorithm works iteratively with the conditional expectation of the complete-data log-likelihood $\log L_c(\psi)$, given the observed data y and the current parameter estimates $\psi^{(t)}$. On the $(k + 1)^{th}$ iteration of the EM algorithm,

E-step: Compute the conditional expectation, namely $Q(\psi; \psi^{(k)})$, expressed as

$$Q(\psi; \psi^{(k)}) = E_{\psi^{(k)}} \{\log L_c(\psi) | y\}.$$

M-step: Choose $\psi^{(k+1)}$ to be any value of $\psi \in \Omega$ that maximizes $Q(\psi; \psi^{(k)})$.

$$Q(\psi^{(k+1)}; \psi^{(k)}) \geq Q(\psi; \psi^{(k)}) \quad \forall \psi \in \Omega.$$

However, in practice, it may not be feasible to attempt to find the value of ψ that globally maximizes the function $Q(\psi; \psi^{(k)})$. For such situations, [9] proposed that $\psi^{(k+1)}$ can be chosen to increase the Q rather than to maximize it.

$$Q(\psi^{(k+1)}; \psi^{(k)}) \geq Q(\psi; \psi^{(k)}).$$

The E- and M-steps are alternated repeatedly until convergence, which may be determined, for instance, by using a suitable stopping rule like $\|\psi^{(k+1)} - \psi^{(k)}\| < \varepsilon$ for some

$\varepsilon > 0$ with some appropriate norm $\|\cdot\|$ or the difference $L(\psi^{(k+1)}) - L(\psi^{(k)})$ changes by an arbitrarily small amount in the case of convergence of the sequence of likelihood values $\{L(\psi^{(k+1)})\}$.

When $g_c(x; \psi)$ is from an exponential family,

$$g_c(x; \psi) = \frac{b(x) \exp(c^\top(\psi)t(x))}{a(\psi)},$$

where $t(x)$ is a $k \times 1$ ($k \geq d$) vector of complete-data sufficient statistics and $c(\psi)$ is a $k \times 1$ vector function of the $d \times 1$ parameter vector ψ , and $a(\psi)$ and $b(x)$ are scalar functions. Members of the exponential family include most common distributions, such as the multivariate normal, Poisson, multinomial and others. Then, the E- and M-steps can be written as

E-step: Compute

$$t^{(k)} = E_{\psi^{(k)}}\{t(X)|y\}.$$

M-step: Compute

$$\psi^{(k+1)} = \underset{\psi}{\operatorname{argmax}}\{c^\top(\psi)t^{(k)} - \log a(\psi)\}.$$

Example 2.5.2. We have a dataset $Y = \{y_1, y_2, \dots, y_N\} = \{1, 2, 3, 6, 7, 8\}$ that we suspect comes from a mixture of two Gaussian distributions. Each data point belongs to one of two clusters, but the cluster assignments are unknown (latent variables). Our goal is to estimate the means (μ_1, μ_2) , the variances (σ_1^2, σ_2^2) of the two clusters and the mixing coefficients π_1, π_2 (such that $\pi_1 + \pi_2 = 1$).

First, randomly initialize $\mu_1 = 2, \mu_2 = 7, \sigma_1^2 = 1, \sigma_2^2 = 1, \pi_1 = 0.5, \pi_2 = 0.5$.

E-step: Compute responsibilities $\gamma_{i,k}$

$$\gamma_{i,k} = \frac{\pi_k \cdot \mathcal{N}(y_i|\mu_k, \sigma_k^2)}{\sum_{j=1}^2 \pi_j \cdot \mathcal{N}(y_i|\mu_j, \sigma_j^2)},$$

where, $\mathcal{N}(y_i|\mu_k, \sigma_k^2)$ is the Gaussian density $\mathcal{N}(y_i|\mu_k, \sigma_k^2) = \frac{1}{\sqrt{2\pi\sigma_k^2}} \exp\left(-\frac{(y_i - \mu_k)^2}{2\sigma_k^2}\right)$.

The result is

$$\gamma = \begin{pmatrix} 0.999999975 & 0.000000025 \\ 0.999996273 & 0.000003727 \\ 0.999447221 & 0.000552779 \\ 0.000552779 & 0.999447221 \\ 0.000003727 & 0.999996273 \\ 0.000000025 & 0.999999975 \end{pmatrix}.$$

The expected complete-data log-likelihood is

$$Q(\psi) = \sum_{i=1}^N \sum_{k=1}^2 \gamma_{i,k} \left(\log \pi_k - \frac{1}{2} \log(2\pi\sigma_k^2) - \frac{(y_i - \mu_k)^2}{2\sigma_k^2} \right).$$

M-Step: We update the parameters

Update the μ_k

$$\begin{aligned} \frac{\partial Q}{\partial \mu_k} &= -\frac{1}{\sigma_k^2} \sum_{i=1}^N \gamma_{i,k} (y_i - \mu_k), \\ \frac{\partial Q}{\partial \mu_k} = 0 &\Leftrightarrow \gamma_{i,k} (y_i - \mu_k) = 0 \Leftrightarrow \mu_k = \frac{\sum_{i=1}^N \gamma_{i,k} y_i}{\sum_{i=1}^N \gamma_{i,k}} \\ &\Rightarrow \mu_1 = 2.0006, \quad \mu_2 = 6.9994. \end{aligned}$$

Update the σ_k^2

$$\begin{aligned} \frac{\partial Q}{\partial \sigma_k^2} &= -\frac{1}{2} \sum_{i=1}^N \left(\frac{1}{\sigma_k^2} - \frac{(y_i - \mu_k)^2}{(\sigma_k^2)^2} \right), \\ \frac{\partial Q}{\partial \sigma_k^2} = 0 &\Leftrightarrow \sum_{i=1}^N \left(\frac{1}{\sigma_k^2} - \frac{(y_i - \mu_k)^2}{(\sigma_k^2)^2} \right) = 0 \Leftrightarrow \sigma_k^2 = \frac{\sum_{i=1}^N \gamma_{i,k} (y_i - \mu_k)^2}{\sum_{i=1}^N \gamma_{i,k}} \\ &\Rightarrow \sigma_1^2 = 0.6695, \quad \sigma_2^2 = 0.6695. \end{aligned}$$

Update the π_k . The relevant part of $Q(\psi)$ for π_k is

$$\sum_{i=1}^N \sum_{k=1}^2 \gamma_{i,k} \log \pi_k,$$

and we have the constraint

$$\sum_{k=1}^2 \pi_k = 1.$$

Using a Lagrange multiplier λ to handle the constraint, define

$$\mathcal{L}(\pi, \lambda) = \sum_{i=1}^N \sum_{k=1}^2 \gamma_{i,k} \log \pi_k + \lambda \left(1 - \sum_{k=1}^2 \pi_k \right).$$

Then

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \pi_k} &= \frac{\sum_{i=1}^N \gamma_{i,k}}{\pi_k} - \lambda, \\ \frac{\partial \mathcal{L}}{\partial \pi_k} = 0 &\Leftrightarrow \pi_k = \frac{\sum_{i=1}^N \gamma_{i,k}}{\lambda}. \end{aligned}$$

However,

$$\sum_{k=1}^2 \pi_k = 1 \Rightarrow \sum_{k=1}^2 \frac{\sum_{i=1}^N \gamma_{i,k}}{\lambda} = 1 \Rightarrow \lambda = N.$$

Then

$$\begin{aligned} \pi_k &= \frac{\sum_{i=1}^N \gamma_{i,k}}{N} \\ \Rightarrow \pi_1 &= 0.5, \quad \pi_2 = 0.5. \end{aligned}$$

Chapter 3

Related Works

3.1 Term Weighting Approaches

This is an early approach to information retrieval, using the frequency and co-occurrence of words in the query and the document to determine relevance. Although this approach is not good at dealing with semantic similarity, it is still a fast and relatively effective approach, and is still used in many later models.

One of the most popular approaches is *tf-idf* [28]. In [28], the authors state that the recall function of retrieval appears to be best served by using broad, high-frequency terms that occur in many documents of the collection. Such terms may be expected to pull out many documents, including many of the relevant documents. The precision factor, however, may be best served by using narrow, highly specific terms that are capable of isolating the few relevant items from the mass of non-relevant ones. In practice, compromises are normally made by using terms that are broad enough to achieve a reasonable recall level without at the same time producing unreasonably low precision.

As mentioned in [28], the differing recall and precision requirements favor the use of composite term weighting factors that contain both recall- and precision-enhancing components. Three main considerations appear important in this connection:

- a) Terms that are frequently mentioned in individual documents, or document excerpts, appear to be useful as recall-enhancing devices. This suggests that a term frequency (*tf*) factor be used as part of the term weighting system measuring the frequency of occurrence of the terms in the document or query texts. Term frequency weights have been used for many years in automatic indexing environments.
- b) Term frequency factors alone cannot ensure acceptable retrieval performance. Specifically, when the high frequency terms are not concentrated in a few particular documents, but instead are prevalent in the whole collection, all documents tend to be retrieved, and this affects the search precision. Hence a new collection-dependent

factor must be introduced which favors terms concentrated in a few documents of a collection. The well-known inverse document frequency (*idf*) (or inverse collection frequency) factor performs this function. The *idf* factor varies inversely with the number of documents n to which a term is assigned in a collection of N documents. A typical *idf* factor may be computed as $\log(N/n)$.

Term discrimination considerations suggest that the best terms for document content identification are those able to distinguish certain individual documents from the remainder of the collection. This implies that the best terms should have high term frequencies but low overall collection frequencies. A reasonable measure of term importance may then be obtained by using the product of the term frequency and the inverse document frequency ($tf \times idf$).

Once we have the importance of the terms (we call this the weight of the terms), then as mentioned in [28], if w_{dk} (or w_{qk}) represents the weight of term t_k in document D (or query Q), and t terms in all are available for content representation, the term vectors for document D and query Q can be written as

$$\begin{aligned} D &= (t_0, w_{d0}; t_1, w_{d1}; \dots; t_t, w_{dt}), \\ Q &= (q_0, w_{q0}; q_1, w_{q1}; \dots; q_t, w_{qt}). \end{aligned} \quad (3.1)$$

A query-document similarity value in [28] may be obtained by comparing the corresponding vectors, using for example the conventional vector product formula

$$similarity(Q, D) = \sum_{k=1}^t w_{qk} \cdot w_{dk}. \quad (3.2)$$

In [28], they also consider when a length normalized term weighting system is used with the vector similarity function of expression (3.2), one obtains the well-known cosine vector similarity formula

$$similarity(Q, D) = \frac{\sum_{k=1}^t w_{qk} \cdot w_{dk}}{\sqrt{\sum_{k=1}^t (w_{qk})^2 \sum_{k=1}^t (w_{dk})^2}}. \quad (3.3)$$

From the $tf - idf$ approach with the concepts of tf , idf and weights for each term, another well-known approach that is still used today was built, which is the BM25 formula [26]. As state in [26], for ad-hoc retrieval, and ignoring any repetition of terms in the query, this formula can be simplified to

$$w_j(\bar{d}, C) := \frac{(k_1 + 1)d_j}{k_1((1 - b) + b\frac{dl}{avdl}) + d_j} \log \frac{N - df_j + 0.5}{df_j + 0.5}, \quad (3.4)$$

where $\bar{d}$ is an document belonging to a collection C , and $w_j(\bar{d}, C)$ is a term weighting function for the j term in $\bar{d}$. d_j denotes the term frequency of the j -th term, while df_j is the document frequency of term j . dl is the document length, $avdl$ is the average document length across the collection, and $k1$ and b are free parameters.

With these weights, in [26] the document score is then obtained by adding the document term weights of terms matching the query q

$$W(\bar{d}, q, C) = \sum_j w_j(\bar{d}, C) \cdot q_j. \quad (3.5)$$

3.2 Query Expansion

A query of user to a search engine is often unstructured and incomplete. These factors prevent a search engine from satisfying information need correctly and completely. Query expansion is proposed to solve this situation. Numerous query expansion techniques have been explored in Information Retrieval. Using an external resource to expand a query is one of the simplest and intuitive approaches. Various approaches to expand a query by adding similar terms using co-occurrence similarity, query logs, user feedback, pseudo relevance feedback,... have also been proposed.

Nevertheless, the use of query expansion to low-resource languages is still not widespread. In this section, we explore the *UnifiedStructPRF* and *MultiStructPRF* methods introduced by Arjun Atreya et al. [35], which will also be incorporated into our implementation phase.

Expansion model accomplishes the process of extracting the expansion terms from the set of relevant documents. Top k documents retrieved are assumed to be relevant. The authors extract the expansion terms in query language and assisting language separately using this model. This model utilizes the structure of the document for query expansion.

Terms from all the documents in corpus constitute a collection set. Relevant set consists of the terms from top k relevant documents retrieved for a query. Terms in relevant set act as potential expansion terms. The authors customize the EM algorithm for generating expansion terms.

They customize the EM algorithm to utilize the structure of Wikipedia documents to generate expansion terms. A Wikipedia document has four parts viz., title, body, categories and infobox. The terms in relevant set are divided into multiple sets based on their position of occurrence in the document. Each part of the document contributes to the expansion terms with different importance. For instance, a title precisely represents the document, whereas the body of the document is more generic in nature, elaborating

the title, as it were. Hence, expansion terms from the title are more important than the terms from the body of the document.

They propose a systematic approach called *UnifiedStructPRF*, which dynamically assigns the weights to each part of the document. For a document with four parts, there are four relevant sets and four collection sets corresponding to terms from four parts of the documents. Thus, the total number of sources in the model becomes

$$\text{no. of sources} = 2 \times \text{no. of parts}.$$

For ease of explanation, they consider the document with two parts (title and body). The proposed model is scalable to any number of parts.

The interpretation for the notations used in the formulas is as follows:

- N = observation sequence; all terms from relevant set.
- M = sources; relevant set (title) R_T , relevant set (body) R_B , collection set (title) C_T and collection set (body) C_B .
- L = outcome; all terms from collection.
- P_{jk} is the probability that the k^{th} expansion term uniquely represents j^{th} source.
- π_j is the probability of selecting j^{th} source.
- X_{ik} is the indicator variable representing whether the i^{th} term from the observation is same as the k^{th} expansion term or not.
- Z_{ij} is the hidden variable representing whether the source of i^{th} term is j or not.

They need to boost the probability of the expansion terms that uniquely represent the relevant set. EM algorithm iteratively performs this task and stores the probability of a term k , uniquely representing relevant set's title and body, in $P_{R_T k}$ and $P_{R_B k}$ respectively.

$L(\theta)$ is the likelihood of all the terms from relevant set being expansion terms.

$$L(\theta) = \prod_{i=1}^N \prod_{j=1}^M (\pi_j \cdot \prod_{k=1}^L P_{jk}^{X_{ik}})^{Z_{ij}}.$$

Taking the logarithm of the expression to get the log likelihood,

$$LL(\theta) = \sum_{i=1}^N \sum_{j=1}^M E(z_{ij}) \left(\log(\pi_j) + \sum_{k=1}^L X_{ik} \log(P_{jk}) \right).$$

They maximize the log likelihood to obtain the equations for the $P_{R_T k}$ and $P_{R_B k}$ subject to the constraints $\sum_{j=1}^M \pi_j = 1$ and $\sum_{k=1}^L P_{jk} = 1$ using the Lagrange multiplier α and β

$$MLE = LL(\theta) - \alpha \cdot \left(\sum_{j=1}^M \pi_j - 1 \right) - \beta \cdot \left(\sum_{k=1}^L P_{jk} - 1 \right).$$

Partially differentiating MLE with respect to π_j and P_{jk} respectively, the expressions for π_j and P_{jk} are

$$\pi_j = \frac{\sum_{i=1}^N E(Z_{ij})}{\sum_{i=1}^N \sum_{j=1}^M E(Z_{ij})},$$

$$P_{jk} = \frac{\sum_{i=1}^N x_{ik} \cdot E(z_{ij})}{\sum_{i=1}^N \sum_{k=1}^L x_{ik} \cdot E(z_{ij})}.$$

Finally, in the expectation step, they use the values of π_j and P_{jk} to compute a better estimate for the hidden variable Z_{ij} ,

$$E(z_{ij}) = \frac{\pi_j * \prod_{k=1}^L P_{jk}^{x_{ik}}}{\sum_{j=1}^M (\pi_j * \prod_{k=1}^L P_{jk}^{x_{ik}})}.$$

In order to find terms uniquely representing source R_T , they consider source R_T as the relevant set and remaining source R_B , C_T and C_B as the collection set. The important point to note is that R_B is considered as noise for R_T which ensures that they find the expansion terms which uniquely represent only the relevant set(title). Similarly, they consider R_T as noise for finding the expansion terms in the relevant set(body). The terms representing R_T and R_B individually constitute the final set of expansion terms.

In *MultiStructPRF*, they scale this mathematical model to accommodate four parts of the document viz., title, body, categories and infobox.

3.3 Neural Network Approaches

The approaches presented in the previous sections are essentially non-neural network approaches. They have the common advantage of short or even no training time, and provide fast results with low resource consumption. However, these approaches often have the disadvantage of not capturing semantics well, only considering lexical similarities, not to mention that some approaches depend on linguistic characteristics, so the results are often incomplete or have low relevance. Therefore, approaches using neural networks have been studied and applied.

One approach for this direction is to use CNNs, with a typical example model being the hierarchical CNN to model text matching as the problem of image recognition. This model, called *MatchPyramid*, was presented in paper [23] and specifically studied for information retrieval in paper [22]. As stated in [23], this model is inspired by the success

of convolutional neural network in image recognition, where neurons can capture many complicated patterns based on the extracted elementary visual patterns such as oriented edges and corners.

As can be seen in figure 3.1, the model consists of three parts: Matching Matrix, Hierarchical Convolution and Matching Score Aggregation.

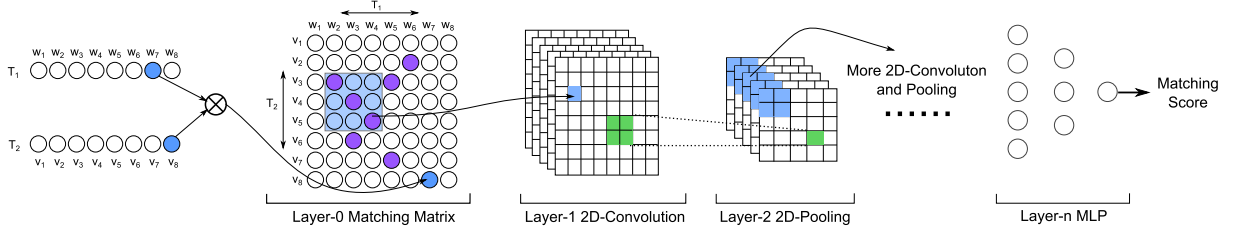


Figure 3.1: Model structure of MatchPyramid [22]

In the first part - Matching Matrix, as stated in [23], one challenging problem by modeling text matching as image recognition lies in the different representations of text and image: the former are two 1D (onedimensional) word sequences while the latter is typically a 2D pixel grid. Also, as pointed out in [22], another problem is to keep both the word-level similarity and the matching structures. So Matching Matrix was introduced to resolve these issues. According to the description in [22], Matching Matrix is a two-dimension structure where each element $\mathbf{M}_{ij}$ denotes the similarity between the i -th word w_i in the first piece of text and the j -th word v_j in the second piece of text:

$$\mathbf{M}_{ij} = w_i \otimes v_j \quad (3.6)$$

where $\otimes$ stands for a general operation to obtain the similarity. In [22], the authors proposed four kinds of similarity functions based on words w_i and v_j , or their embeddings $\vec{\alpha}_i$ and $\vec{\beta}_j$: Indicator Function, Cosine, Dot Product, and Gaussian Kernel.

In the next part - Hierarchical Convolution, the model conducts hierarchical convolution to extract matching patterns. The paper [23] has described this part in detail as follows:

For the first layer of CNN, the k -th kernel $\mathbf{w}^{(1,k)}$ scans over the whole matching matrix $\mathbf{z}^{(0)} = \mathbf{M}$ to generate a feature map $\mathbf{z}^{(1,k)}$:

$$\mathbf{z}_{i,j}^{(1,k)} = \sigma \left(\sum_{s=0}^{r_k-1} \sum_{t=0}^{r_k-1} \mathbf{w}_{s,t}^{(1,k)} \cdot \mathbf{z}_{i+s,j+t}^{(0)} + b^{(1,k)} \right) \quad (3.7)$$

where r_k denotes the size of the k -th kernel, and ReLU is adopted as the active function σ .

Dynamic pooling strategy is then used to deal with the text length variability. By applying dynamic pooling, we will get fixed-size feature maps:

$$\mathbf{z}_{i,j}^{(2,k)} = \max_{0 \leq s < d_k} \max_{0 \leq t < d'_k} \mathbf{z}_{i \cdot d_k + s, j \cdot d'_k + t}^{(1,k)} \quad (3.8)$$

where d_k and d'_k denote the width and length of the corresponding pooling kernel, which are determined by the text lengths n and m , and output feature map size $n' \times m'$.

After the first convolution and dynamic pooling, we continue to obtain higher level features $\mathbf{z}^{(l)}$, $l \geq 2$ by further convolution and max-pooling, with general formulations:

$$\mathbf{z}_{i,j}^{(l+1,k')} = \sigma \left(\sum_{k=0}^{c_l-1} \sum_{s=0}^{r_k-1} \sum_{t=0}^{r_k-1} \mathbf{w}_{s,t}^{(l+1,k')} \cdot \mathbf{z}_{i+s,j+t}^{(l,k)} + b^{(l+1,k)} \right), l = 2, 4, 6, \dots \quad (3.9)$$

$$\mathbf{z}_{i,j}^{(l+1,k)} = \max_{0 \leq s < d_k} \max_{0 \leq t < d'_k} \mathbf{z}_{i \cdot d_k + s, j \cdot d'_k + t}^{(l,k)}, l = 3, 5, 7, \dots \quad (3.10)$$

where c_l denote the number of feature maps in the l -th layer.

In the final part - Matching Score Aggregation, the paper [23] used a MLP (Multi-Layer Perception) to produce the final matching score. As an example mention by the authors, using binary classification and two-layer perceptron, we will obtain a 2-dimensional matching score vector:

$$(s_0, s_1)^\top = \mathbf{W}_2 \sigma(\mathbf{W}_1 \mathbf{z} + \mathbf{b}_1) + \mathbf{b}_2, \quad (3.11)$$

where s_0 and s_1 are the matching scores of the corresponding class, z is the output of the hierarchical convolution, $\mathbf{W}_i$ is the weight of the i -th MLP layer and σ denotes the activation function.

At the time the study about *MatchPyramid* model - the paper [22] was published, as it states, MatchPyramid has been shown state-of-the-art performances on several text matching tasks. But also in this study, the authors has shown that the *MatchPyramid* models can significantly outperform several recently introduced deep matching models on the retrieval task, but still cannot compete with the traditional retrieval models, such as BM25 and language models, and suggest more investigation need to be conducted in this direction. Because of this, we had to find another approach for this direction.

And our another potential approach is to use the attention model. In the paper [39], the authors propose a novel neural model for Ad-hoc CLIR (Cross-lingual Information Retrieval) against short queries using weak supervision instead of annotated CLIR corpora. In [39], they describe that their model computes the probability of relevance of each sentence in a foreign document to an input query. These probabilities are then combined

to compute a relevance score for a query-document pair.

As the authors of [39] themselves mentioned, low resource languages often lack relevance annotations for CLIR, and when available the training data usually has limited coverage for possible queries. At the same time, they also believe that compared to CLIR annotated data, sentence translations are often easier to obtain and have better coverage for short queries. So in [39], they did not rely on relevance-annotated data, but training their on samples extracted from parallel machine translation data as weak supervision.

To solve the problem of designing the model to effectively identify relevant spans in the possibly long foreign sentence, in [39] they use an attention mechanism to allow the model to learn what parts of the sentence to focus on without explicit supervision (e.g. word alignments). In [39] they also investigate element-wise interaction between the query and sentence representations to further improve relevance matching.

The model in [39], called Query Relevance Attentional Neural Network (QRANN), is designed to measures the relevance between an English query and a foreign sentence. Formally, given a query $q = [q_1, q_2, \dots, q_m]$ and a foreign sentence $s = [s_1, s_2, \dots, s_n]$, the model estimates relevance probability $P(\text{rel}|q, s) = [0, 1]$. The architecture and execution flow of QRANN are described in [39] and are presented in the figure 3.2 belows

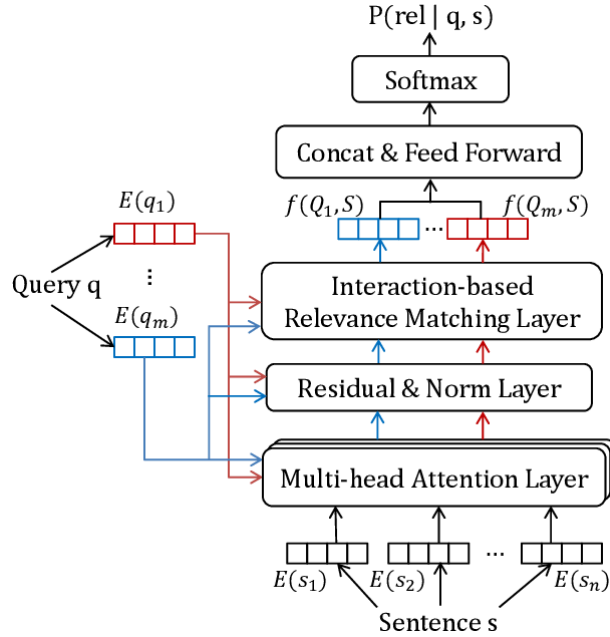


Figure 3.2: Query relevance attentional neural network (QRANN) [39] model architecture. Each word in the query has an attention mechanism with the sentence to identify relevant spans, followed by a residual connection and layer normalization. After relevance matching, the outputs are fed to a feedforward layer to obtain relevance features of the entire query, which are used for final relevance estimation.

The execution flow of QRANN is described in [39] as follows:

First, each word in q and s is represented as a d -dimensional embedding using lookup table $E(\cdot)$:

$$\begin{aligned} Q &= [Q_1, \dots, Q_m] = [E(q_1), \dots, E(q_m)], \\ S &= [S_1, \dots, S_n] = [E(s_1), \dots, E(s_n)]. \end{aligned} \quad (3.12)$$

After that, they use an attention mechanism over S , which allows the model to learn which spans contain relevance evidence, as well as how to weight that evidence. They compute a context vector C_j for word q_j as a weighted sum of S_j :

$$C_j = \sum_{i=1}^n \alpha_i^j S_j, \quad (3.13)$$

with attention weight α_i^j for each word s_i in sentence scores how well the sentence word s_i and q_j match: $\alpha_i^j = \text{softmax}(e_i^j)$, where $e_i^j = v_\alpha \tanh(U_\alpha Q_j + W_\alpha S_i)$. U_α , W_α and v_α are shared across each q_j in q for better generalization.

Next, in [39] they are inspired by recent work in machine translation [36], and found it beneficial to use a multi-head attention to allow the model to jointly attend to information at different positions. So they compute multi-head attention M_j by concatenating k different attention context vectors and applying a linear transformation:

$$M_j = [C_j^1, C_j^2, \dots, C_j^k] W_M. \quad (3.14)$$

In the next step, they use a residual connection to the attention outputs, followed by layer normalization: $N_j = \text{LayerNorm}(Q_j + M_j)$.

Following the normalization step, they propose an effective approach for relevance matching by computing both difference and relatedness between Q_j and N_j :

$$e_{N_j, Q_j}^{diff} = N_j - Q_j; e_{N_j, Q_j}^{prod} = N_j \odot Q_j. \quad (3.15)$$

The interaction-based relevance matching layer is a hidden layer on top of the concatenation of e_{N_j, Q_j}^{diff} and e_{N_j, Q_j}^{prod} :

$$f(Q_j, S) = \text{relu}(W_C [e_{N_j, Q_j}^{diff}; e_{N_j, Q_j}^{prod}] + b_C), \quad (3.16)$$

with W_C , b_C are shared across each q_j in q .

Then, they concatenate the relevance matching outputs and pass them through another hidden layer to concatenate query-word specific features. Each neuron has connections with all query-specific features, aiming to capture semantic relationships among query words:

$$g(Q_S) = \tanh(W_h[f(Q_1, S); \dots; f(Q_m, S) + b_h]). \quad (3.17)$$

Subsequently, the relevance probability between q and s is computed by:

$$P(rel|q, s) = \text{softmax}(W_o g(Q, S) + b_o), \quad (3.18)$$

where W_h , b_h , W_o and b_o are trainable parameters.

Finally, in [39] they experimented with different methods for combining sentence relevance scores, including average and maximum, and found the most effective method to be the probability of relevance to at least one sentence in the document:

$$P(rel|q, D) \approx 1 - \prod_{s \in D} (1 - P(rel|q, s)). \quad (3.19)$$

According to paper [39], the results from QRANN are better than some other approaches including CNN-based approaches. However, as paper [39] itself mentioned, this model is designed to focus on short queries. With the advent of Transformer [36] architecture and pre-trained BERT models that are more effective in language understanding, the pure attention-based models are gradually being studied less.

3.4 Knowledge Distillation

Proposed by [13], knowledge distillation is a method to train a model, called the student, using valuable information provided by the output of another model, called the teacher. This way, the teacher model’s knowledge can be transferred into the student model. The idea of knowledge distillation is widely used in the field of computer vision, natural language processing and information retrieval. There are several notable works in the context of information retrieval, such as EmbedDistill [16], TwinBERT [17], ReasoningRank [15].

However, the application of knowledge distillation to low-resource languages remains limited. In this section, we discuss OPTICAL: Optimal Transport distillation for low-resource Cross-lingual information retrieval [40], which will also be used in our implementation phase.

In this paper, they propose OPTICAL, an optimal transport knowledge distillation framework for Cross-lingual Information Retrieval (CLIR) task involving low-resource languages. OPTICAL builds CLIR models by separating the retrieval knowledge from the translation knowledge. First, the teacher model learns the retrieval knowledge in a monolingual setting. Then they design a knowledge distillation loss based on the optimal

transport costs to transfer the retrieval knowledge to the student model in a cross-lingual setting.

A typical framework for knowledge distillation relies on a teacher model to directly generate a target distribution. However, the authors’ method is different since the target distribution is determined by the optimal transport plan based on the cost matrix estimated by a teacher model.

The student model shares the same architecture as the teacher. If the document collection for the CLIR task remains in English, then the document encoder of the student model E_{S_d} can be a copy of the teacher’s document encoder, E_{M_d} . Here, they focus on the design of the query encoder of the student model, E_{S_q} , which handles non-English queries.

Assume that q is an English query and $\hat{q}$ is a non-English parallel query. The token representation of q generated by E_{M_q} contains rich knowledge for query document matching. If E_{S_q} behave like E_{M_q} , namely, if the output of E_{S_q} with $\hat{q}$ is close to the output of E_{M_q} with q , then the token representations generated by E_{S_q} can have a similar retrieval performance to the teacher model. Therefore, the training objective is to reduce the distance between the outputs of the teacher and student query encoders given parallel inputs.

Because words that are translations of each other from different languages tend to have a smaller distance in the hyper-space after the multilingual pre-training step, they initialize the student query encoder’s parameters weights using the same multilingual pre-trained language model as the teachers. Suppose the byte-pair encoding (BPE) tokenizer tokenizes q into L_q tokens and $\hat{q}$ into $L_{\hat{q}}$ tokens, they expand them to the same length L by appending [MASK] tokens. Then, they define the distance from $\hat{q}$ to q in the vector space of E_{S_q} and E_{M_q} as follows:

$$D(q, \hat{q}) = \underset{f: [1..L] \rightarrow [1..L]}{\operatorname{argmin}} \frac{1}{L} \sum_{i=1}^L 1 - E_{S_{\hat{q}}}(\hat{q}_i) \cdot E_{T_q}(q_{f(i)}).$$

where f is a bijective function which maps the token index from $\hat{q}$ to q . Although q and $\hat{q}$ have the same semantics, the arrangement of tokens varies between different languages. Therefore, the distance definition is equivalent to finding the mapping function that minimizes the average cosine distance between the token pairs. When L increases, using brute force to find the mapping f becomes computationally intractable.

Therefore, they approach this as an optimal transport (OT) problem. The authors consider the sentences q and $\hat{q}$ to be distributions of tokens, the mapping f to be the transportation plan, and the distance $D(q, \hat{q})$ to be the transportation cost. To elaborate further, they assign equal mass to the tokens in $\hat{q}$ and q by defining a uniform source

probability distribution, μ_s , on $\hat{q}$ and a uniform target probability distribution, μ_t , on q . That is: $\mu_s(i) = \frac{1}{L}$ and $\mu_t(j) = \frac{1}{L}$ where $1 \leq i, j \leq L$. In the context of their project, the chosen cost matrix C is a $L \times L$ matrix with C_{ij} being the cost of moving the mass from $\hat{q}_i$ to q_j , specifically the cosine distance between two tokens:

$$C_{ij} = 1 - E_{S_{\hat{q}}}(\hat{q}_i) \cdot E_{T_q}(q_{f(i)}).$$

3.5 Optimal Transport

Nowadays, in computer science, numerous methods have been proposed to solve the optimal transport (OT) problem. A popular method is to use a regularization term, in order to smooth the distance estimation and to construct a faster algorithm. Entropic regularization [8], Group Lasso [6], KL, Itakura Saito, β -divergences [10] are some popular work in this application. Additionally, there has been a rapid increase in the development of fast and approximate algorithms, such as Inexact Proximal point method [37], $\mathcal{R}_\rho$ metric space [3], min-Sliced Wasserstein Generalized Geodesics (min-SWGG) [18], discrete OT with group-sparse regularizers [14].

In this section, we discuss two approaches, Entropic regularization [8] and Inexact Proximal point method [37], which is also used in our implementation phase.

3.5.1 Entropic Regularization Approach

Using a regularization term in solving an OT problem is to introduce a regularization term in the objective, which means the Optimal Transport on discrete distributions problem is now to find the optimal transport plan γ_0^λ such that

$$\gamma_0^\lambda = \operatorname{argmin}_{\gamma \in \mathcal{P}} \{ \langle \gamma, \mathbf{C} \rangle_F + \lambda \Omega(\gamma) \}.$$

In this approach, the regularization term is specifically chosen as the entropic regularizer. Therefore, the problem is to find the optimal transport plan γ_0^λ such that

$$\gamma_0^\lambda = \operatorname{argmin}_{\gamma \in \mathcal{P}} \left\{ \langle \gamma, \mathbf{C} \rangle_F + \lambda \sum_{i,j} \gamma(i,j) (\log \gamma(i,j) - 1) \right\}.$$

With this entropic regularization term, the problem will

1. lose sparsity, gains stability,
2. become a strictly convex optimization problem,
3. ensure that the loss and OT matrix are differentiable.

To solve the entropy regularized problem, we define the Lagrangian

$$\mathcal{L}(\gamma, \alpha, \beta) = \sum_{i,j} \gamma_{i,j} c_{i,j} + \lambda \gamma_{i,j} (\log \gamma_{i,j} - 1) + \sum_i \alpha_i \left(a_i - \sum_j \gamma_{i,j} \right) + \sum_j \beta_j \left(b_j - \sum_i \gamma_{i,j} \right).$$

We have

$$\begin{aligned} \frac{\partial \mathcal{L}(\gamma, \alpha, \beta)}{\partial \gamma_{i,j}} = 0 &\Leftrightarrow c_{i,j} + \lambda \log \gamma_{i,j} + \alpha_i + \beta_j = 0 \\ &\Leftrightarrow \gamma_{i,j} = \exp \left(\frac{\alpha_i}{\lambda} \right) \exp \left(-\frac{c_{i,j}}{\lambda} \right) \exp \left(\frac{\beta_j}{\lambda} \right). \end{aligned}$$

Thus, by Sinkhorn and Knopp's theorem (1967),

$$\exists! u, v > 0 : \gamma_0^\lambda = \text{diag}(u) \exp \left(-\frac{\mathbf{C}}{\lambda} \right) \text{diag}(v),$$

where $u_i = \exp \left(\frac{\alpha_i}{\lambda} \right)$ and $v_j = \exp \left(\frac{\beta_j}{\lambda} \right)$.

Therefore, this problem can be solved with **Sinkhorn-Knopp algorithm**.

Input: Probabilities $\{\mu_s, \mu_t\}$ on support points $\{x_i^s\}_{i=1}^m, \{x_j^t\}_{j=1}^n$, cost matrix $\mathbf{C}$, parameter λ

Output: Transport plan γ

$u \leftarrow \mathbf{1}_m$

$G_{ij} \leftarrow \exp \left(-\frac{C_{ij}}{\lambda} \right)$

for $i = 1, 2, 3, \dots, n$ **do**

$v \leftarrow \mu_t \oslash G^\top u$

$u \leftarrow \mu_s \oslash Gv$

end

return $\gamma = \text{diag}(u) \cdot G \cdot \text{diag}(v)$

The algorithm performs alternatively a scaling along the rows and columns of $G = \exp \left(-\frac{\mathbf{C}}{\lambda} \right)$ to match the desired marginals. The time complexity of his algorithm is $O(kn^2)$ where k iterations are required to reach convergence.

3.5.2 Inexact Proximal Point Method for Exact Optimal Transport Problem Approach

In this paper, the authors demonstrate that regularized variations with large regularization parameter will degradate the performance in several important machine learning applications, and small regularization parameter will fail due to numerical stability issues with existing algorithms. Therefore, they address this challenge by developing an Inexact Proximal point method for exact Optimal Transport problem (IPOT).

To propose this new method, the authors present the background of two key concepts, the Sinkhorn-Knopp algorithm and the Generalized Proximal Point Method. While the background on entropic regularization and the Sinkhorn-Knopp algorithm was discussed in the previous section, we now proceed with an overview of the Generalized Proximal Point Method.

Generalized proximal point method. Given a convex objective function f defined on a convex set $\mathcal{X}$ with optimal solution set $\mathcal{X}^* \subset \mathcal{X}$, the generalized proximal point algorithm aims to solve the following problem

$$\operatorname{argmin}_{x \in \mathcal{X}} f(x).$$

The algorithm generates a sequence $\{x^{(t)}\}_{t=1,2,\dots}$ by performing the following generalized proximal point iteration:

$$x^{(t+1)} = \operatorname{argmin}_{x \in \mathcal{X}} f(x) + \beta^{(t)} d(x, x^{(t)}),$$

where d is a regularization term used to define the proximal operator, usually defined to be a closed proper convex function.

In cases where the proximal operator is approximated or not computed exactly in each iteration, the method is referred to as the inexact proximal point method.

Back to the problem, the authors use the generalized proximal point method to solve the optimal transport problem. They take $f(\gamma) = \langle \gamma, \mathbf{C} \rangle_F$, $\mathcal{X} = \mathcal{P}$ and d to be Bregman divergence D_h based on entropy function $h(x) = \sum_i x_i \log x_i$ or

$$D_h(x, y) = \sum_i x_i \log \frac{x_i}{y_i} - \sum_i x_i + \sum_i y_i,$$

where $\mathbf{C}$ is a cost matrix with $c_{i,j} = c(x_i^s, x_j^t)$ and $\mathcal{P} = \{\gamma \in (\mathbb{R}^+)^{n_s \times n_t} \mid \gamma \mathbf{1}_{n_t} = \mathbf{a}, \gamma^T \mathbf{1}_{n_s} = \mathbf{b}\}$.

As a result, the proximal point iteration for problem can be written as

$$\gamma^{(t+1)} = \operatorname{argmin}_{\gamma \in \mathcal{P}} \{\langle \gamma, \mathbf{C} \rangle_F + \beta^{(t)} D_h(\gamma, \gamma^{(t)})\}.$$

Substituting Bregman divergence into proximal point iteration,

$$\gamma^{(t+1)} = \operatorname{argmin}_{\gamma \in \mathcal{P}} \{\langle \gamma, \mathbf{C} - \beta^{(t)} \log \gamma^{(t)} \rangle_F + \beta^{(t)} h(\gamma)\} . (*)$$

Denote $\mathbf{C}' = \mathbf{C} - \beta^{(t)} \log \gamma^{(t)}$. It is already known that $\gamma^{(t)}$ is a fixed value that is not relevant to optimization variable γ . Therefore, the previous problem can be considered as an entropy regularized optimal transport problem, which can be solved based on Sinkhorn

algorithm. This new algorithm is called IPOT algorithm.

Input: Probabilities $\{\mu_s, \mu_t\}$ on support points $\{x_i^s\}_{i=1}^m, \{x_j^t\}_{j=1}^n$, cost matrix $\mathbf{C}$

Output: Transport plan γ

$v \leftarrow \frac{1}{m} \mathbf{1}_m$

$G_{ij} \leftarrow \exp\left(-\frac{C_{ij}}{\lambda}\right)$

$\gamma^{(1)} \leftarrow \mathbf{1} \mathbf{1}^\top$

for $t = 1, 2, 3, \dots$ **do**

$Q \leftarrow G \circ \gamma^{(t)}$

for $l = 1, 2, 3, \dots, L$ **do**

$u \leftarrow \frac{\mu_s}{Qv}$

$v \leftarrow \frac{\mu_t}{Q^\top u}$

end

$\gamma^{(t+1)} \leftarrow \text{diag}(u) \cdot Q \cdot \text{diag}(v)$

end

Different from Sinkhorn algorithm, proximal point algorithms converge to the optimal solution with nested iterative loops. Exact proximal point method, i.e., solving (*), provides a feasible solution that is closest to the optimal solution set in each proximal iteration until the optimal solution reached. However, the disadvantage for exact proximal point method is that it's not efficient.

Therefore, the proposed inexact proximal point method (IPOT) does not solve (*) exactly. Instead, a very small amount of Sinkhorn iteration, e.g., only one iteration ($L = 1$), is suggested. There are some reasons for this

- The convergence of Sinkhorn algorithm in each proximal iteration is not required, since it is just intermediate step.
- Usually in numerical optimization, the first a few iterations achieve the most decreasing in the objective function.
- It is observed after experiments that IPOT can still converge to an exact solution with small amount of inner iterations.

This nice property results in similar convergence and computation time comparing to Sinkhorn method. However, IPOT provides a robust and accurate computation of Wasserstein distance and associated transport plan, which leads to a better performance.

Chapter 4

Proposed Approach

In this chapter, we will present our approach to the problem of information retrieval in low-resource languages. We will first present the pipeline for training and describe the components in the pipeline to provide a general view of how the components work and interact with each other. From there, we will present the architecture of the model we propose.

4.1 Problem Statement

Developing an information retrieval system for low-resource languages presents several technical hurdles, particularly when constructing a semantically accurate model with limited training data. While one might consider translation-based approaches where queries or documents are translated into a high-resource language such as English, this strategy introduces several issues: poor translation quality for LRLs, semantic drift when translating from culturally rich contexts, and importantly, translating documents to high-resource languages creates a tendency to marginalize the original language, discouraging documents from continue to be written in that language. This undermines long-term language and culture preservation aim of the project.

To address these issues, this project approaches in an alternative direction: designing an IR model that operates directly within the low-resource language, without relying on translation. By employing techniques such as knowledge distillation and query expansion, the system transfers understanding from a high-resource model and learns to perform retrieval in low-resource settings without relying on large manually labeled corpora. These design choices allow the model to better capture the linguistic and cultural nuances of the target languages, making retrieval more relevant and empowering continued digital engagement in underrepresented language communities.

4.2 Training Pipeline

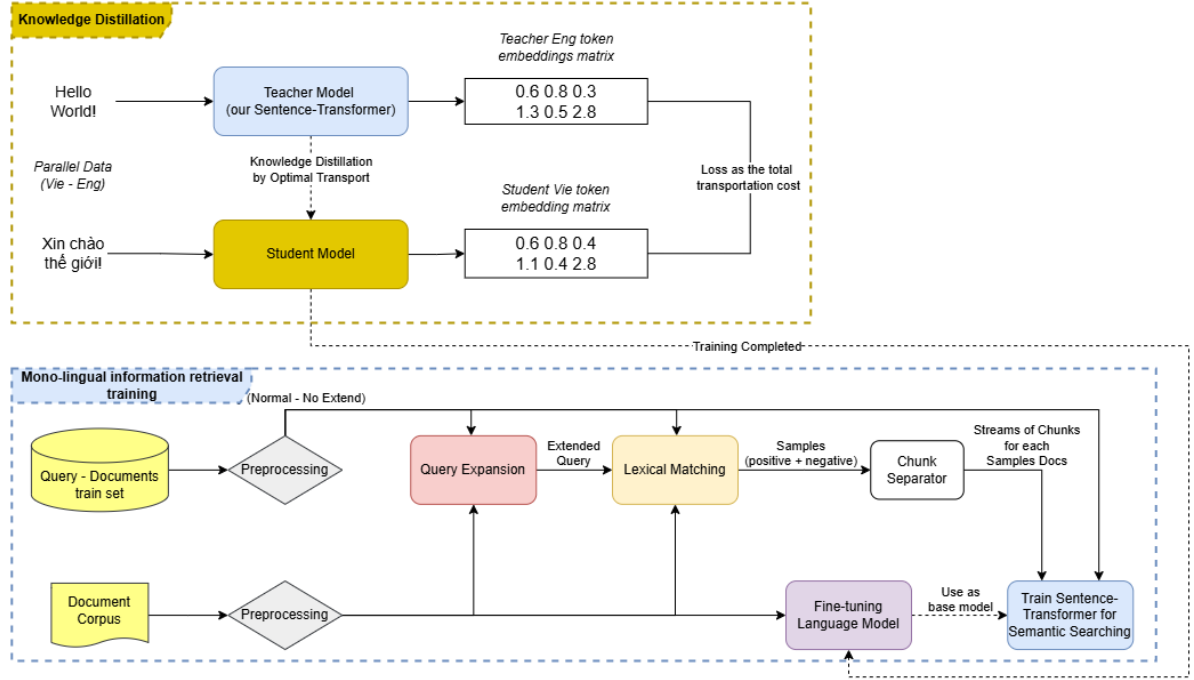


Figure 4.1: Pipeline for training process

Figure 4.1 provides an overview of the components in the pipeline and their execution flow. As can be seen in the figure, the pipeline consists of two major parts: the knowledge distillation part, and the training part for mono-lingual information retrieval.

As mentioned in [25], the aim of knowledge distillation is to extend existing sentence embedding models to new languages, which help us to create multilingual versions from previously monolingual models. In [25], they use mean-squared loss function during training. Here, we decided to experiment with a different approach for training, which is to use the optimal transport distillation. This is the approach presented in [40]. We will describe this approach in detail in the Subsection 4.2.1.

After the knowledge distillation part, we should obtain a language model that is expected to be good enough to encode the information of a sentence in the language of the student model. We will then use this trained model for the mono-lingual information retrieval training process. In the mono-lingual information retrieval training part, we build our pipeline based on the idea of using both lexical matching and semantic searching which was mention in [24], combined with the use of query expansion component and some adjustments related to the similarity scoring process between query and documents. The execution flow goes through the following steps:

1. First, the queries in the query-document dataset along with the documents in the

document corpus, which are usually stored in raw text, go through preprocessing. This process includes steps such as stripping extra spaces, stemming, stop-word removing, words and sentences segmentating, and tokenizing. The result of this step will be documents and queries that have been processed and saved in a form more suitable for training. For documents that are marked as relevant documents for each query in the query-document dataset, they will be used to determine positive samples for training.

2. After preprocessing, for each sample in the query-document set, the query is passed to the query expansion component. The goal of this component is to create an extended query with more terms, thereby increasing the ability to match more documents when performing lexical matching. A detailed description for this query expansion component will be presented later in the Subsection 4.2.2.
3. With the newly obtained expanded query, both the original query and the expanded query are then passed independently into the lexical matching component. This component will calculate the lexical similarity between the query and each document in the document corpus. The result of this step will be a list with each element being the lexical similarity score between the query and each document in the document corpus, sorted in descending order for convenience of subsequent components. We will talk more about this component in the Subsection 4.2.3.
4. The following step after lexical matching is the document chunking step. In the approach mentioned in [24], they just concatenate the title and content of each article in the legal corpus into one "very long" sentence. While they do mention the percentage of articles based on length, they do not mention how to handle articles with word counts exceeding the model's limit, nor do they mention splitting the articles. Meanwhile, we argue that for documents of different lengths, longer lengths may either provide more details relevant to the query, or may contain more irrelevant content and reduce the relevance and usefulness of the document. Within the same document, some parts may be more relevant to the query than others, and the parts that are truly relevant to the query may be distributed in later positions of the document. Therefore, we believe that separating the document into small chunks, calculating the query relevance to each chunk, and then combining them may help solve the above problems.
5. Along with the sample preparation process, similar to [24], we can perform the fine-tuning process of the language model before using this model for the semantic similarity process. Normally, with languages with few resources such as Khmer, we will rarely have a language model optimized specifically for that language but often

have to use multilingual models (such as mBERT, XLM-R, ...). Therefore, fine-tuning these multilingual models to make them more compatible with the language we want to use is necessary. In addition, similar to [24], if we target the model to documents belonging to a few specific topics (like in [24] is legal documents), we can also fine-tuning to make the model more compatible. However, in our current approach this step is optional, since we have already used a student model that has been distilled from a teacher model which is already effective in representing semantics, and also sometimes the language used already has a good enough native language model, or the document topic is so broad that fine-tuning may lead to overfitting. In [24], they use the legal corpus to fine-tune masked language modeling (MLM) the language model. Referring to our case, it is similar that we will use the document corpus to MLM fine-tuning the language model. Then we use the fine-tuned model as the base model for the semantic searching step. If we don't do fine-tuning, we can directly use the language model as the base model.

6. Finally, for each query and list of documents selected as samples (each document has been divided into chunks), they will be passed into the semantic searching training loop. The main component of this step is the Sentence Transformer model [21], which will help us calculate the relevance between the query and each chunk in each document. Description of how this and other components work will be presented in the Subsection 4.2.4.

4.2.1 Knowledge Distillation

In this first component, we aim to transfer that rich knowledge of a teacher Sentence Transformer model to the student model so that we can obtain a student Sentence Transformer that can encode semantic information of a sentence better for the other languages. By utilizing this component, we enhance the student model by helping it learning the way semantic meaning is represented in the teacher model, which is usually more efficient due to being trained on a larger dataset. As a result, it help improve the performance of the student model which often performs poorly due to lack of training resources. Also, as state in [40], the distillation training only needs to focus on the translation knowledge as a general textual encoder, so we can use bitext data to train the student query encoder, which is more feasible for low resource languages. The execution flow of this component is as follows:

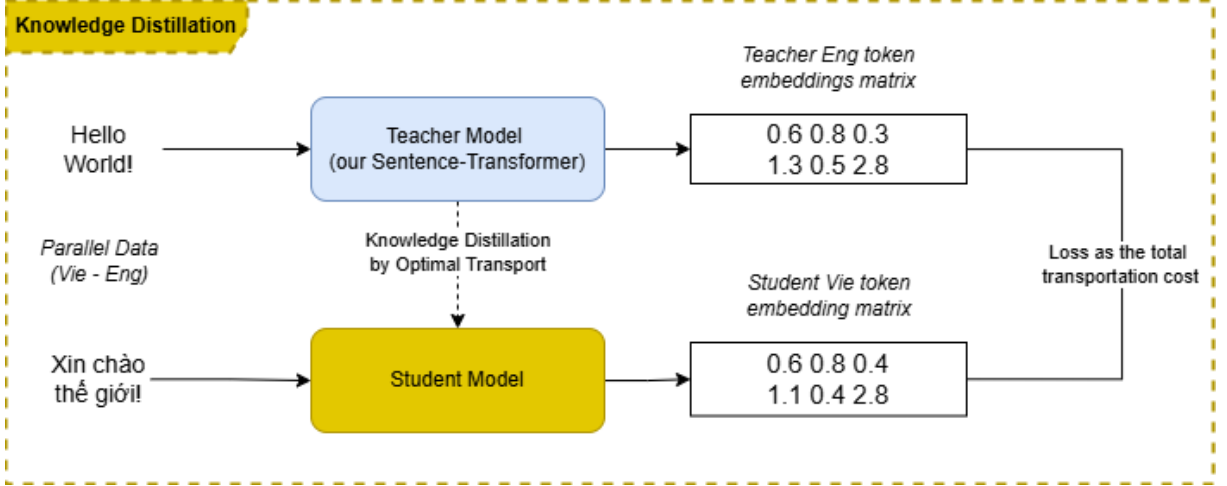


Figure 4.2: Execution flow of the Knowledge Distillation component

1. First, we need parallel data of sentences in the source language, the language that was used for training in the monolingual retrieval phase, and in the target language in which we need to distill knowledge. For each pair of parallel sentences (s, t) , s will be fed into the teacher model, while t will be fed into the student model. The teacher and student Sentence Transformers will then calculate token embeddings of the tokens of the sentences passed into them. We denote $E(\cdot)$ and $\hat{E}(\cdot)$ the token embeddings of the teacher and student models, respectively.
2. As stated in the purpose of this component, we need to ensure that tokens that are translation of each other should have similar embeddings, i.e, for a token s_i and its translation token t_j , we should have $E(s_i) \approx \hat{E}(t_j)$. In other words, we first find a mapping f that maps a token in s with its corresponding token that has the same meaning in t , then we minimize the average cosine distance between each mapped pair. In order to achieve this, we will use the same approach as in proposed [40]. We view this problem as an Optimal Transport problem. We consider each sentence to be a distribution of tokens, the mapping f to be the transportation plan, and the distance that needs minimizing to be the transportation cost. To elaborate further, we assign mass to the tokens by considering each token to contain a proportion of the whole sentence's meaning. This part is also where our approach differs from that mentioned in [40], which consider the token distribution to be uniform. As sentences are tokenized differently in different languages, one token in a language may correspond to more than one tokens in another language. Therefore, if we consider tokens to have the same mass, we cannot find a transportation plan that maps the tokens correctly. Thus, to capture the semantic importance of each token in s and t , we define a source distribution, μ_s , on s and a uniform target distribution,

μ_t , on t , both of which is based on the $tf - idf$ scores of the tokens. That is: $\mu_s(i) = tf - idf(s_i, s, S)$ and $\mu_t(j) = tf - idf(t_j, t, T)$ where S and T are the set of source sentences and target sentences respectively. By using $tf - idf$ score, we can assign tokens that are semantically important with higher mass, allowing them to be mapped to many tokens with lower mass.

The set of transportation plans between these two distributions is then the set of doubly stochastic matrices $\mathcal{P}$ defined as

$$\mathcal{P} = \{\gamma \in (\mathbb{R}^+)^{m \times n} | \gamma \mathbf{1}_n = \mu_s, \gamma^T \mathbf{1}_m = \mu_t\}.$$

where m and n respectively are the number of tokens in the source sentence and target sentence, $\mathbf{1}_m$ is a m -dimensional vector of ones, and γ is a transportation plan. Then the computation of the initial distance become the optimal transport problem:

$$\gamma_0 = \underset{\gamma \in \mathcal{P}}{\operatorname{argmin}} \langle \gamma, C \rangle_F$$

where $\langle \cdot, \cdot \rangle_F$ is the Frobenius inner product, i.e. the sum of the matrices' elementwise products, γ_0 is the optimal transportation plan and C is a $m \times n$ cost matrix with C_{ij} being the cost of moving the mass from s_i to t_j . In the context of this project, the chosen cost is the cosine distance between two tokens, or in other words, the difference in meanings of two tokens:

$$C_{ij} = 1 - E(s_i) \cdot \hat{E}(t_j)$$

3. With the problem redefined, our goal now is to minimize the total transportation cost, which is also chosen as the loss function for training the student model. In general, since this is a linear programming problem, we can solve it with $O((m+n)^3)$ complexity. When m and n increases, the algorithm becomes intractable. Thus, we use the Inexact Proximal point method for Optimal Transport (IPOT) [37], of which the solution has been discussed in Subsection 3.5.2. As has been stated in Subsection 3.5.2, the complexity of IPOT is similar to that of Sinkhorn, which is $O(k(m+n)^2)$, thus reducing the time needed to find the solution, and further, the training time. Minimizing the loss for each pair of parallel sentence will prompts the student model to learn to reduce the distance between its and the teacher's embeddings.

4.2.2 Query Expansion

As mentioned in the Related Works chapter, PRF [2] [19] is an efficient query expansion method that relies only on the query itself and the document corpus, does not require user feedback, and can be applied to all languages regardless of language characteristics. Several variations of PRF have been created to improve this technique, including using expectation maximization technique to extract expansion terms from the initially retrieved top k documents [35] [38], recognizing and assigning different weights to parts of the document [34] [35], or using the help of a resource-rich language [4] [5] [35].

In our current proposed model and pipeline, we will not use the approach of using the help of a resource-rich language. There are two main reasons for this decision:

The first reason is the difference in the importance of this component. The approaches in the cited papers use PRF and query expansion as the main steps in the information retrieval process, so they need to focus on this step. Meanwhile, our approach considers the query expansion component as a supporting step, which helps to match more documents in the lexical similarity step and avoids the possibility of missing relevant documents. Therefore, we want to design this component to work more simply to focus on the next steps.

The second reason is about the performance of the model and pipeline. As mentioned in the first reason, we aim for this component to be a support component, so we want it to run faster so that we can focus on the following steps. Not using additional support from another language will help avoid the additional time needed to translate queries from the original language to the resource-rich language, perform resource-rich query expansion, and combine the results of both languages. We will also avoid being dependent on the translation model, and also avoid the need for additional datasets in the resource-rich language.

For the two reasons mentioned above, we decided to choose an approach that focused on the first two variants. We finally design our query expansion component as a variation of a technique mentioned in [35], which is *UnifiedStructPRF*. The image below shows the architecture and execution flow of this component:

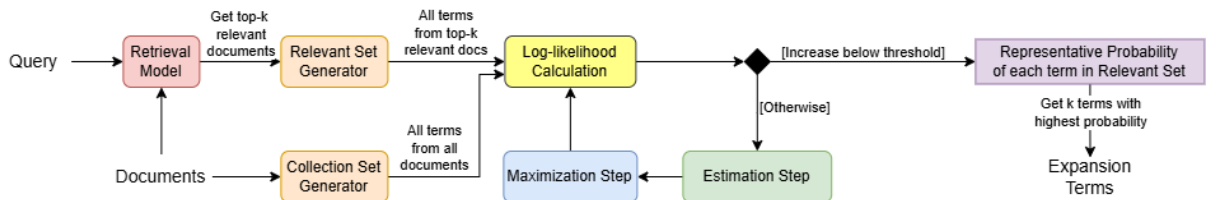


Figure 4.3: Architecture and execution flow of Query Expansion component

The flow of this component is as follows:

At the beginning, the original query and the documents in the document set are fed into the retrieval model. This component performs a relevance ranking between the query and the documents, in order to select potential documents to use in the next step. In the approach presented in [35], they generated a language model θ_D for each document D in the collection and language model θ_Q for each query Q , then use KL-divergence (D_{KL}) metric to measure the similarity between θ_Q and θ_D . The more relevance of D , the less is D_{KL} .

However, in our approach we do not use this method. There are two reasons for our decision. First, as defined in [27], a statistical language model is a probability distribution over all possible sentences or spoken utterances, documents, or any other linguistic unit. This means that creating a language model for each query and document is the process of finding the right parameters for the probability distribution of each query and document. This will increase the time required for the computation, which is something we want to avoid. Second, in the lexical similarity step that follows this query expansion step, we used BM25+ ranking function to calculate the similarity score between the query and the documents to rank them. Therefore, to ensure consistency, we decided to use BM25+ for this step. In the implementation, we use BM25+ provided by the `rank_bm25` library, which implements the formula presented in [33] as follows:

$$rsv_q = \sum_{t \in q} \log\left(\frac{N+1}{df_t}\right) \cdot \left(\frac{(k_1+1) \cdot tf_{td}}{k_1 \cdot ((1-b) + b \cdot (\frac{L_d}{L_{avg}}))} + \delta\right),$$

with q is the given query and rsv_q , which is calculated by the sum of individual term t , is the retrieval status value for q . N is the number of documents, df_t is the number of documents containing the term (the document frequency), tf_{td} is the number of times term t occurs in document d . L_d is the length of the document (in terms) and L_{avg} is the mean of the document lengths. b and k_1 are two tuning parameters, while δ is a positive constant added to the tf_{td} component before multiplying.

After going through the retrieval model, we will get a list of ranked relevance of each document to the query. At this point, we will select the k most relevant documents, extract all the terms in these k documents. In [35], they call this the *relevant set*. Terms in relevant set will be our potential expansion terms, and in the next step, we will calculate to choose the words to use to expand the query. Along with that, all the words in all the documents are also extracted, and is called the *collection set*. Both of these sets will be used in the next step.

In this step, we will follow the approach mentioned in [35], which is to use a customize version of the EM algorithm proposed in [38]. As presented in [35], EM algorithm is a process of iteratively assigning a probability to expansion terms. In each iteration, the probability of the terms which uniquely represent the relevant set is increased while prob-

ability of other terms are decreased. Thus, yielding the representative terms of relevant set. In the basic version proposed in [38], all part of the document have the same contribution to the expansion terms, while in [35], they argue that each part of the document contributes to the expansion terms with different importance. In [35], they state that a title precisely represents the document, whereas the body of the document is more generic in nature, elaborating the title, as it were. Hence, expansion terms from the title are more important than the terms from the body of the document. We agreed with their point of view, and therefore we decided to use the approach they proposed.

The interpretation for the notations used in the formulas of [35] is as follows:

- N = observation sequence; all terms from relevant set.
- M = sources; relevant set (title) R_T , relevant set (body) R_B , collection set (title) C_T and collection set (body) C_B .
- L = outcome; all terms from collection.
- P_{jk} is the probability that the k^{th} expansion term uniquely represents j^{th} source.
- π_j is the probability of selecting j^{th} source.
- X_{ik} is the indicator variable representing whether the i^{th} term from the observation is same as the k^{th} expansion term or not.
- Z_{ij} is the hidden variable representing whether the source of i^{th} term is j or not.

With this notation, what we need the EM algorithm to boost are $P_{R_T k}$ and $P_{R_B k}$, which are the probability of a term k uniquely representing relevant set's title and body. Paper [35] estimates $L(\theta)$, the likelihood of all the terms from relevant set being expansion terms in terms of marginal likelihood of the observed data (X_{ik} and Z_{ij}), with the equation as follows:

$$L(\theta) = \prod_{i=1}^N \prod_{j=1}^M (\pi_j \cdot \prod_{k=1}^L P_{jk}^{X_{ik}})^{Z_{ij}}. \quad (4.1)$$

We can deduce (4.1) as follows:

1. From the interpretation of the equation's notation, X_{ik} is the indicator variable representing whether the i^{th} term from the observation is same as the k^{th} expansion term or not. This is something we can check and get a definite result by simply comparing whether term i is similar to term k , so we can conclude that X_{ik} is an observable variable. Meanwhile, for any expansion term i in the relevant set, whether it originates from source j or not is something we cannot affirm with certainty, so we consider this as a latent variable.

2. Now, we call f is the probability density function of a term i uniquely representing a source j . To ensure the generality of the function for all available observation sequences N and sources M , we will consider all terms k in the collection L , and call P_{jk} is the probability that word k represents source j .

Note that, if term k in the collection is different from term i , that is, term k is not in our relevance set, then we are not interested in maximizing the probability that term k represents source j . We are only interested in the probability when k equals i , which means the probability that term k represents source j is also the probability that word i represents source j . The function f will now have the form:

$$f(i, j, k, P_{jk}) = \begin{cases} P_{jk}, i = k, \forall k = 1, 2, \dots, L. \\ 0, i \neq k, \end{cases}$$

3. With X_{ik} is the indicator variable, we can construct the function f simply as follows:

$$f(i, j, X_{ik}, P_{jk}) = \prod_{k=1}^L P_{jk}^{X_{ik}}$$

It can be seen that when i is different from k , X_{ik} is 0, meaning the power in the expression of the product series will be to the power 0, the result is 1 and does not contribute anything to the product series. Only when i is k , X_{ik} is 1, then the power in the expression of the product series will be to the power 1 and equal to the probability itself, at this time it will contribute to the final result.

4. With f is the probability density function representing that a term i represents a single source j , we will now compute it for all the sources for term i , which is the entire relevant set. Let L_i be this probability. Once again, to ensure the generality of the function for all available observation sequences N , we will consider all sources j in M .

Same as above, if source j in M is not the origin of term i , then we are not interested in estimating the probability that term i belongs to source j . We are only interested in the probability when source j in M is not the origin of term i . For each source j , the probability of choosing source j , which is also the importance of source j in contributing to the relevant set, is also different, and the final total probability will therefore be an expected value according to this probability. So, the function L_i will now have the form:

$$L_i(i, j, \pi_j, X_{ik}, P_{jk}) = \begin{cases} \pi_j \cdot f(i, j, X_{ik}), source(i) = j, \forall j = 1, 2, \dots, M. \\ 0, source(i) \neq j, \end{cases}$$

5. With Z_{ij} is the latent variable representing whether the source of i^{th} term is j or not, we can construct the function L_i simply as follows:

$$f(i, \pi_j, Z_{ij}, X_{ik}, P_{jk}) = \prod_{j=1}^M (\pi_j \cdot \prod_{k=1}^L P_{jk}^{X_{ik}})^{Z_{ij}}.$$

Similar to step 3, it can be seen that when $source(i)$ is not from j , Z_{ij} is 0, meaning the power in the expression of the product series will be to the power 0, the result is 1 and does not contribute anything to the product series. Only when $source(i)$ is j , Z_{ij} is 1, then the power in the expression of the product series will be to the power 1 and equal to the expression itself, at this time it will contribute to the final result.

6. Finally, with the probability for each term i in N , to calculate the likelihood of all the terms from relevant set being expansion terms, we just need to multiply all the probabilities of each term together:

$$L(\pi_j, Z_{ij}, X_{ik}, P_{jk}) = \prod_{i=1}^N \prod_{j=1}^M (\pi_j \cdot \prod_{k=1}^L P_{jk}^{X_{ik}})^{Z_{ij}}.$$

Set $\theta = (\pi_j, Z_{ij}, X_{ik}, P_{jk})$, we receive the equation (4.1) above.

Now, for ease of calculation, we take the log of the expression to get the log likelihood:

$$LL(\theta) = \sum_{i=1}^N \sum_{j=1}^M E(z_{ij}) \left(\log(\pi_j) + \sum_{k=1}^L X_{ik} \log(P_{jk}) \right). \quad (4.2)$$

From the notations defined above, we can derive two constraints: $\sum_{j=1}^M \pi_j = 1$ (the sum of the probabilities of choosing a source j in all sources is equal to 1) and $\sum_{k=1}^L P_{jk} = 1$ (the sum of the probabilities that a term k in the collection represents source j is equal to 1). To solve the optimization problem of maximizing (4.2) to obtain the equations for $P_{R_T k}$ and $P_{R_B k}$, we will use the Lagrange multiplier α and β with the two constraints just mentioned. We will obtain the maximum log likelihood expression as follows:

$$MLE = LL(\theta) - \alpha \cdot \left(\sum_{j=1}^M \pi_j - 1 \right) - \beta \cdot \left(\sum_{k=1}^L P_{jk} - 1 \right). \quad (4.3)$$

In the maximization step (M step), from (4.3) we can obtain the expressions for π_j và P_{jk} by partially differentiating MLE with respect to π_j và P_{jk} respectively. In [35], the two equations are defined as follows:

$$\pi_j = \frac{\sum_{i=1}^N E(Z_{ij})}{\sum_{i=1}^N \sum_{j=1}^M E(Z_{ij})}, \quad (4.4)$$

$$P_{jk} = \frac{\sum_{i=1}^N x_{ik} \cdot E(z_{ij})}{\sum_{i=1}^N \sum_{k=1}^L x_{ik} \cdot E(z_{ij})}. \quad (4.5)$$

The proof process for equations (4.4) and (4.5) is as follows:

Proof for (4.4):

1. For each value j , rewrite MLE and $LL(\theta)$ in terms of π_j :

$$LL(\theta) = \sum_{i=1}^N E(z_{ij}) \cdot \log(\pi_j) + \underbrace{\sum_{i=1}^N \sum_{k=1, k \neq j}^M E(z_{ik}) \left(\log(\pi_k) + \sum_{\ell=1}^L X_{i\ell} \log(P_{k\ell}) \right)}_{\text{Not depend on } \pi_j},$$

$$MLE = LL(\theta) - \alpha \pi_j + \alpha - \underbrace{\alpha \left(\sum_{i=1, i \neq j}^M \pi_i - 1 \right) - \beta \left(\sum_{k=1}^L P_{jk} - 1 \right)}_{\text{Not depend on } \pi_j}.$$

2. Partially differentiating $LL(\theta)$ and MLE with respect to π_j :

$$\frac{\partial LL(\theta)}{\partial \pi_j} = \frac{\sum_{i=1}^N E(z_{ij})}{\pi_j},$$

$$\frac{\partial MLE}{\partial \pi_j} = \frac{\partial LL(\theta)}{\partial \pi_j} - \alpha = \frac{\sum_{i=1}^N E(z_{ij})}{\pi_j} - \alpha.$$

3. With $\frac{\partial MLE}{\partial \pi_j} = 0$, we have

$$\pi_j = \frac{\sum_{i=1}^N E(z_{ij})}{\alpha} \forall j = 1, 2, \dots, M.$$

Simultaneously, we have the constraint presented above: $\sum_{j=1}^M \pi_j = 1$. From these, we deduce that

$$\sum_{j=1}^M \frac{\sum_{i=1}^N E(z_{ij})}{\alpha} = 1 \Leftrightarrow \alpha = \sum_{j=1}^M \sum_{i=1}^N E(z_{ij}) = \sum_{i=1}^N \sum_{j=1}^M E(z_{ij}).$$

4. From α , we can calculate π_j by substituting α into π_j equation in step 3. We get the same result as (4.4) which we need to prove:

$$\pi_j = \frac{\sum_{i=1}^N E(z_{ij})}{\alpha} = \frac{\sum_{i=1}^N E(Z_{ij})}{\sum_{i=1}^N \sum_{j=1}^M E(Z_{ij})}.$$

Proof of (4.5):

1. Similar to the proof for (4.4), for each value j , k we rewrite MLE and $LL(\theta)$ in terms of P_{jk} , then, by partially differentiating $LL(\theta)$ and MLE with respect to P_{jk} , we got:

$$\begin{aligned} \frac{\partial LL(\theta)}{\partial P_{jk}} &= \frac{\sum_{i=1}^N E(z_{ij}) X_{ik}}{P_{jk}}, \\ \frac{\partial MLE}{\partial P_{jk}} &= \frac{\partial LL(\theta)}{\partial P_{jk}} - \beta = \frac{\sum_{i=1}^N E(z_{ij}) X_{ik}}{P_{jk}} - \beta. \end{aligned}$$

2. With $\frac{\partial LL(\theta)}{\partial P_{jk}} = 0$, we have

$$P_{jk} = \frac{\sum_{i=1}^N E(z_{ij}) X_{ik}}{\beta} \forall j = 1, 2, \dots, M.$$

Simultaneously, we have the constraint presented above: $\sum_{k=1}^L P_{jk} = 1$. From these, we deduce that

$$\sum_{k=1}^L \frac{\sum_{i=1}^N E(z_{ij}) X_{ik}}{\beta} = 1 \Leftrightarrow \beta = \sum_{k=1}^L \sum_{i=1}^N E(z_{ij}) X_{ik} = \sum_{i=1}^N \sum_{k=1}^L E(z_{ij}) X_{ik}.$$

3. From β , we can calculate P_{jk} by substituting β into P_{jk} equation in Step 2. We get the same result as (4.5) which we need to prove:

$$P_{jk} = \frac{\sum_{i=1}^N E(z_{ij}) X_{ik}}{\beta} = \frac{\sum_{i=1}^N E(z_{ij}) X_{ik}}{\sum_{i=1}^N \sum_{k=1}^L E(z_{ij}) X_{ik}}.$$

In the expectation step (E step), we will use the just-computed values of π_j and P_{jk} to compute a better estimate for the hidden variable Z_{ij} (X_{ik} is an indicator variable, and as we mentioned in the proof of (4.1), we can obtain exact value and no need to estimate). The paper [35] estimates this value as follows:

$$E(z_{ij}) = \frac{\pi_j \cdot \prod_{k=1}^L P_{jk}^{x_{ik}}}{\sum_{j=1}^M (\pi_j \cdot \prod_{k=1}^L P_{jk}^{x_{ik}})}. \quad (4.6)$$

Following the EM algorithm, after each log-likelihood calculation, this value will be compared with the log-likelihood value of the previous iteration. The algorithm will stop when the current log-likelihood value increases less than a preset threshold value.

Or to define it mathematically, call θ_t is the parameters of the log-likelihood function $LL(\theta_t)$ at the t^{th} iteration, and ϵ is the preset threshold, the algorithm will stop when $LL(\theta_t) \leq LL(\theta_{t-1}) + \epsilon$. If this condition is not satisfied, a new iteration will start with the expectation and maximization step performed, to yield a new log-likelihood value for comparison next time.

When the algorithm stops, we will have two probability lists where each term represents each source R_T and R_B . From here, we select a number of terms with the highest probability from each list, combine them, and remove duplicate terms and terms that are already in the original query, to obtain the final expanded terms list. We simply add these terms to the original query to get the expanded query.

4.2.3 Lexical Matching

As mentioned in Section 4.2, the goal of this step is to calculate the lexical similarity between the query and each document in the document corpus, in order to identify documents with high similarity to be used as samples for the following steps. The execution flow in this step is as follows:

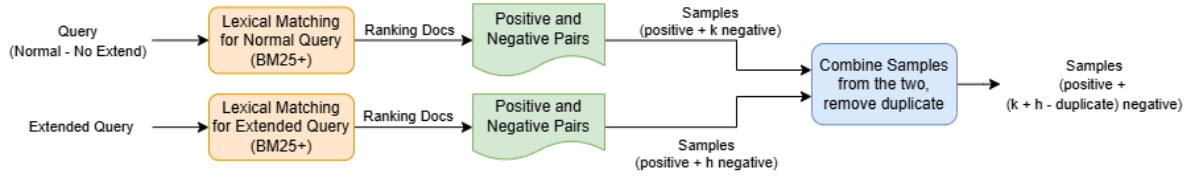


Figure 4.4: Execution flow of Lexical Matching component

1. Firstly, after the query expansion step mentioned in Subsection 4.2.2, we get an extended query. Together with the normal (unextended) query, both queries are passed independently to the lexical matching component. This component will calculate the lexical similarity score between the query and each document in the corpus, using the BM25+ formula mentioned in Subsection 4.2.2. Since we passed the two queries independently, we will get two separate lists for each type of query, with each element in the list being the lexical similarity between the query and the corresponding document, sorted in descending order.
2. In the next step, similar to the approach in [24], positive and negative samples will be generated to train the semantic similarity search model through contrastive learning. There will be a positive and negative sample generator component that will take care of this, which will sit right after the lexical matching component, and take in the lexical similarity score list for each query independently. As mentioned in Section 4.2, the documents marked as relevant for each query are used to identify positive samples.

For negative samples, similar to [24], we will take k documents that are not in the positive sample set and have the highest similarity score in the previous training round result (in this case it will be the top- k highest lexical similarity scores). Because we pass both the original query and the extended query independently into this component, we will get two separate sample lists with potentially different negative samples and different similarity score values for each sample.

3. The two sample lists are then combined. Since we pass two queries independently, and the expanded query has some additional words added to the query, there are two problems that need to be solved during the merging process:
 - First, the same document but the similarity score in the two lists of the two queries will be different, due to the increased frequency of some terms in the expanded query, leading to different scores when applying BM25+.
 - Second, the ranking of the same document may also be different in the two lists, due to the appearance of new documents when using the extended query.

To solve the first problem, we scale the similarity score in each list to the range $(0,1]$. We then apply Min-Max Scaling with Positive Range formula

$$x'_i = \frac{x_i - \min(x)}{\max(x) - \min(x)} \cdot (b - a) + a, a > 0, b > a.$$

We chose this formula for several reasons: first, the similarity scores between a document and a query are independent of other documents, and these scores do not follow any probability distribution. Secondly, it is easy to implement and ensures all values fall within the same positive range, no element is zero after scaling. And thirdly, it can be applied independently to each list, the scaling is based only on the characteristics of each list, which is consistent with the fact that we are passing two queries independently.

For the second problem, we just simply included all documents appear in both list. If the same document appears as a sample in both lists, our current strategy is to keep the higher score and discard the lower one. We can represent this strategy mathematically as follows:

Let the two lists of similarity score for normal query (N) and extended query (E) be:

$$N = \{(d_1, s_1), (d_2, s_2), \dots, (d_n, s_n)\},$$

$$E = \{(d'_1, s'_1), (d'_2, s'_2), \dots, (d'_m, s'_m)\}.$$

Where d_i and d'_i represent the documents, and s_i and s'_i represent the corresponding similarity scores.

Define the combined list C as:

$$C = \{(d, s) : d \in D_N \cup D_E, s = \max(s_N, s_E)\},$$

where

$$D_N = \{d_i : (d_i, s_i) \in N\}, \quad D_E = \{d'_i : (d'_i, s'_i) \in E\}.$$

The values s_N and s_E are defined as

$$s_N = \begin{cases} s_i, & \text{if } (d, s_i) \in N, \\ -\infty, & \text{if } d \notin N, \end{cases}$$

$$s_E = \begin{cases} s'_i, & \text{if } (d, s'_i) \in B, \\ -\infty, & \text{if } d \notin B. \end{cases}$$

Thus, for each $d \in D_N \cup D_E$, the value s is determined as:

$$s = \begin{cases} s_A, & \text{if } d \in D_N \text{ and } d \notin D_E, \\ s_B, & \text{if } d \in D_E \text{ and } d \notin D_N, \\ \max(s_A, s_B), & \text{if } d \in D_N \cap D_E. \end{cases}$$

The main reason for this strategy is that the similarity between the original query and the documents should always be given the highest priority, since the original query most accurately reflects what the user input. If strategies such as adding scores between the two lists or passing these values through a hidden layer are applied, the similarity of the expanded query will likely influence the final score of the sample. Another reason is that this approach avoids the need to train additional parameters that would be required if using a hidden layer, so the training time of the pipeline may be reduced.

4.2.4 Semantic Searching

In this step, we will train a model that determines the semantic similarity between the query and the documents. The input to the training process will come from the previous steps, including: a normal query (no extend), a language model (with or without fine-tuning) as a base model, and a list of documents that have been selected as test samples after the lexical matching step, each document has been chunked. Based on the approach

mentioned in [24], we also train the model in three rounds. The training process is performed as follows:

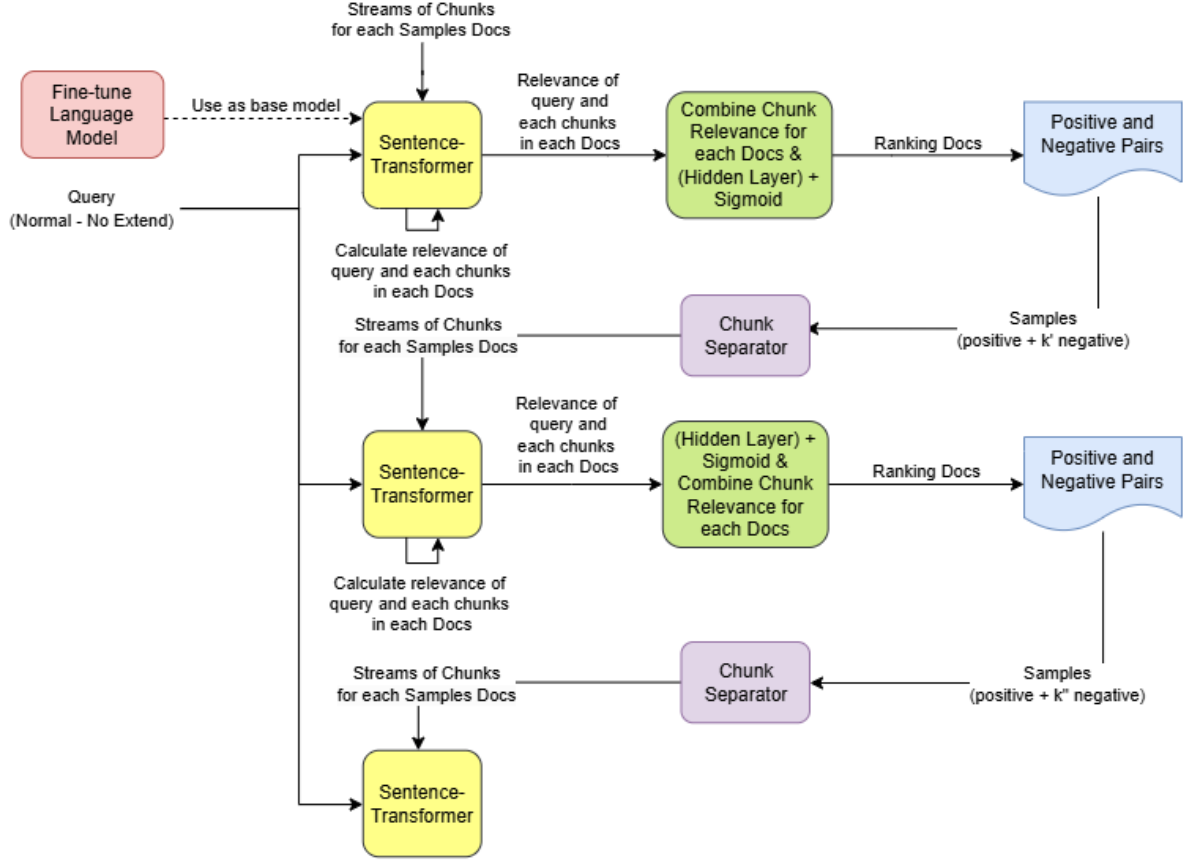


Figure 4.5: Execution flow of Semantic Searching component

The central component of the semantic similarity determination process, which is also the main component trained in this pipeline, is the Sentence Transformer model. Similar to the approach in [24], we build this Sentence Transformer model based on Sentence-BERT (SBERT) [21]. As mentioned in [21], SBERT is a modification of the pretrained BERT network that use siamese and triplet network structures [30] to derive semantically meaningful sentence embeddings that can be compared using cosine-similarity.

For a more detailed description of SBERT, SBERT adds a pooling operation to the output of BERT/RoBERTa to derive a fixed sized sentence embedding. There are three pooling strategies have been experimented listed in [21]: Using the output of the CLS-token, computing the mean of all output vectors (MEAN-strategy, this is the default configuration), and computing a max-over-time of the output vectors (MAX-strategy).

In order to fine-tune BERT/RoBERTa, in [21] they create siamese and triplet networks [30] to update the weights such that the produced sentence embeddings are semantically meaningful and can be compared with cosine-similarity. Depending on the available train-

ing dataset, the structure of the network and the objective function will be different.

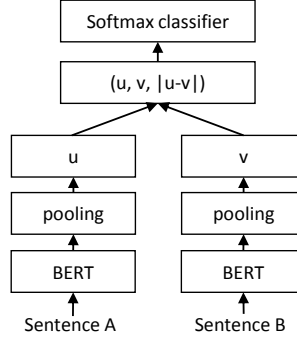


Figure 4.6: SBERT [21] architecture with classification objective function. The two BERT networks have tied weights (siamese network structure).

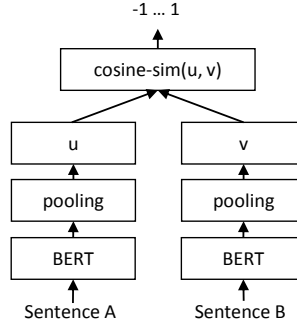


Figure 4.7: SBERT [21] architecture at inference, for example, to compute similarity scores. This architecture is also used with the regression objective function.



Figure 4.8: The Triplet Loss [30] minimizes the distance between an anchor and a positive, both of which have the same identity, and maximizes the distance between the anchor and a negative of a different identity.

Coming back to our approach, we are not only interested in the semantic similarity between two individual sentences, our goal is to determine the semantic similarity between the query and the document. Therefore, for each chunk in each document, we will feed these chunks into the sentence transformer to get the corresponding embedding for the whole chunk, then perform a comparison using cosine similarity with the query embedding. Doing this for all the chunks in the document, we will get the semantic similarity score between the query and each chunk in the document.

With the similarities of the individual chunks, we will combine them to get the semantic similarity between the query and the document. Usually, when combining, there will be some popular methods such as taking the average, taking the maximum, or multiplying all the similarities... Here, we decided to experiment the formula that was presented and confirmed to be the most effective in [39] as follows:

$$P(rel|q, D) \approx 1 - \prod_{s \in D} (1 - P(rel|q, s)). \quad (4.7)$$

with q is the query, D is the current document and s is the sentence in that document (in our case, s refers to a chunk of several sentences). $P(rel|q, D)$ is the semantic similarity point of the query q and the document D , while $P(rel|q, s)$ is of the query q and chunk s in D .

Formula (4.7) can be understood as the semantic similarity between the query and the document is considered as a probability value, and in [39], they found that the most efficient method for estimating this value is the probability of relevance to at least one sentence in the document. As can be seen from the formula, $P(rel|q, s)$ can be viewed as the probability that a sentence s is considered relevant to the query. So $1 - P(rel|q, s)$ is also equivalent to $P(\neg rel|q, s)$, that is, the probability that a sentence s is considered irrelevant to the query. Multiplying the irrelevance probabilities of all sentences together ($\prod_{s \in D} P(\neg rel|q, s) = \prod_{s \in D} (1 - P(rel|q, s))$) gives us $P(\neg rel|q, D)$, the probability that the entire document is irrelevant to the query. Now, subtracting this probability from 1 gives us the probability that at least one sentence in the document is relevant to the query.

After obtaining the semantic similarity score, similar to [24], we will combine the lexical similarity score with the semantic similarity score to get the final value. However, unlike [24], our approach will use a hidden layer with these two similarity score values as input, and the output will be a single value and will be passed through the sigmoid activation function to ensure that the final value is scaled to the range [0,1].

Let $\mathbf{x}_{q,D} = \begin{pmatrix} lexical_score(q, D) \\ semantic_score(q, D) \end{pmatrix} \in \mathbf{R}^2$ be a vector containing two values of lexical similarity and semantic similarity score between query q and document D , $\mathbf{W} \in \mathbf{R}^{1 \times 2}$ be the weight matrix and $\mathbf{b} \in \mathbf{R}$ be the bias ($\mathbf{W}$ and $\mathbf{b}$ are trainable parameters), the final similarity score for a query with a document can be computed as follows:

$$relevant_score(q, D) = \text{sigmoid}(\mathbf{W}\mathbf{x}_{q,D} + \mathbf{b}). \quad (4.8)$$

The reason for the difference between our approach and [24] is that in [24], they use

the score value directly from BM25+ for the lexical similarity score, without scaling and also without going through the step of combining two lists of two query types like us, so the BM25+ value will be much larger than the cosine similarity, which has a value in the range $[-1,1]$. As mentioned in [24] itself, combining score for ranking in previous work like $(1 - \alpha) * \text{lexical_score} + \alpha * \text{semantic_score}$ will not work or take much time to fine the α , so they proposed two approaches: $\text{sqrt}(\text{BM25_score}) * \text{cos_sim}$ and $\text{BM25_score} * \text{cos_sim}$. While in our approach, these values are all scaled to the range $[0,1]$, so the hidden layer approach is more suitable as we can learn the importance of each similarity score value.

Once we have the list of final similarity score between the query and each document selected as samples, this also marks the end of 1 round of training. The score list will be passed to the positive and negative sample generator (mentioned in Subsection 4.2.3), to select the positive and negative samples for the next round of training.

The important point to note here is that the number of negative samples after each training round will decrease. Let k , k' and k'' be the number of negative samples of the first, second and third training rounds, then this triplet of values needs to satisfy the constraint $k'' < k' < k$. With the first training round when k is still large, there will still be many negative samples are high lexical similarity to queries but low semantic similarity. When moving to the second training round with smaller k' , the selected negative samples are also simultaneously have better semantic similarity to queries. In the third round with the smallest k'' , negative samples are really close semantic to queries. This process of training is expected to help the model to be able to recognize more detailed semantic differences and better distinguish documents that are unrelated but semantically very close to the query.

For the loss function, similar to [24], we used contrastive loss [12]. Formula for this loss function is as follows:

$$L = \frac{1}{2}y \cdot d^2 + \frac{1}{2}(1 - y) \cdot \max(0, \text{margin} - d)^2, \quad (4.9)$$

where y is the label (1 for relevant, 0 for non-relevant), d is the distance (or dissimilarity score) between the predicted output and a reference value, and margin is a hyperparameter defining the minimum distance for dissimilar pairs.

As state in [12], unlike conventional learning systems where the loss function is a sum over samples, the loss function here runs over pairs of samples (or in our case is pairs of a query and a document). As can be seen from the formula, there are two main parts contributing to the loss function: loss for positive pairs and loss for negative pairs.

- With the loss for positive pairs part: $L_{pos} = \frac{1}{2}d^2$, the goal is to make relevant pairs have distances close to 0, which corresponds to high similarity.

- With the loss for negative pairs part: $L_{neg} = \frac{1}{2} \max(0, margin - d)^2$, the goal is depends on the *margin*:
 - If $d > margin$: The term $\max(0, margin - d)^2$ becomes 0, and the loss is 0, This means that when a non-relevant pair’s distance d exceeds the margin, it is considered ”distant enough,” and the loss stops penalizing it further.
 - If $d \leq margin$: The loss is actively penalized, increasing quadratically as the distance d approaches 0. This forces the model to push non-relevant streams further away from the relevant ones until their distances d exceeds the margin.

As can be seen above, the *margin* has a significant impact in negative loss. Increasing the *margin* will

- allow more flexibility for non-relevant pairs to be considered “distant enough”,
- reduce penalties for dissimilar pairs unless their distances d is very close to the query (relevant),
- make the model more tolerant of borderline dissimilar pairs.

Decreasing the *margin* will

- tighten the boundary for what is considered a dissimilar pair,
- increase penalties for non-relevant pairs, even if they are moderately dissimilar (closer to the margin),
- force the model to push non-relevant pairs further away, potentially at the cost of overfitting or instability.

In conclusion, using this contrastive loss function with parameters like this will encourage the model to learn meaningful distinctions between relevant and non-relevant documents for a query.

4.3 Proposed Model

After presenting the execution flow of the training pipeline as well as analyzing some important components in detail, in this section we would like to present the model we propose. This model includes the components described above, and has an execution flow similar to the training pipeline. The general architecture and execution flow of the model are as follows:

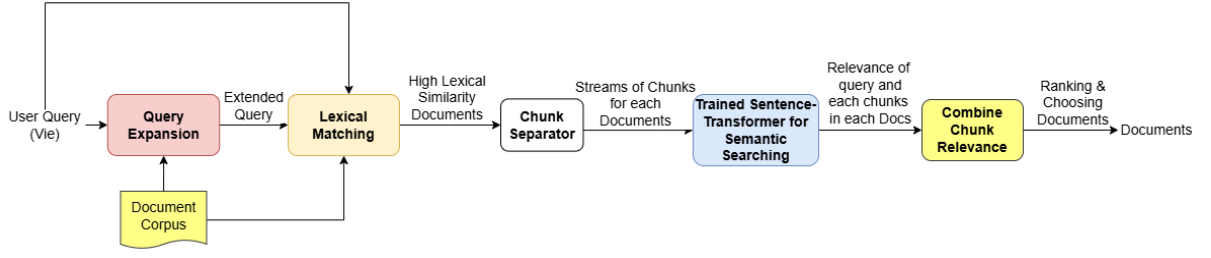


Figure 4.9: *Architecture and execution flow of our proposed model*

The detail execution flow of all the steps is as follows:

1. First, the raw query received from the user will be preprocessed to obtain a query in a form more suitable for the following steps. The work in this preprocessing step is similar to the query preprocessing step in the pipeline mentioned in Section 4.2.
2. Next, the preprocessed query is passed through the query expansion component presented in Subsection 4.2.2 to obtain an extended query.
3. Once we have the extended query, both the regular query and the extended query will be fed into the BM25+ formula independently, to determine the lexical similarity between each query and each document in the corpus. In the pipeline for the lexical matching presented in Subsection 4.2.3, we have a positive-negative sample generator component to limit the number of samples, so after the BM25+ step we only need to pass the entire list of documents with scores into this generator component. However, in the current model, since we are running the actual query with the user and not on the training set, we will not have this component. Therefore, to limit the number of documents before passing them to the Sentence Transformer, after the BM25+ step and sorting the scores in descending order, we need to proactively pre-select the number of documents needed. We will choose k documents with the highest lexical similarity score with the normal query, and h documents with the highest score for the extended query.
4. We then combine these two document lists, remove duplicate documents, and scale the lexical similarity score using the same method as in the pipeline, as presented in Subsection 4.2.3.
5. Then, the selected documents are fed into the chunk separator component. For each document in the list of documents selected in the previous step, they will be divided into small chunks with a maximum length of each chunk (or more specifically, the number of tokens in each chunk) smaller than the processing limit of the trained Sentence Transformer model. This component is similar to the chunk separator in

the pipeline, and the reason for this component is also the same as described in Section 4.2.

6. The query and chunks for each document are then fed into the semantic searching part to perform semantic similarity computation. The execution flow and computation process is quite similar to what was described for the pipeline’s semantic searching component in Subsection 4.2.4. However, since we are running the actual model with user queries instead of labeled datasets, there is no positive-negative samples generator component and no need to run the model for 3 rounds. We only run it once to get the final similarity scores for all documents.
7. Finally, once we have the similarity score for each document, we will now rank the documents in decreasing order of similarity. We will define a `threshold` value, and one document would be chosen if their similarity scores in range `[max_relevant_score - relevant_threshold, max_relevant_score]`. We will also need to define a `relevant_default_lowerbound` value, the minimum similarity score required for a document to be chosen. If this value is higher than `max_relevant_score - relevant_threshold`, this value will be chosen as the lower bound instead of `max_relevant_score - relevant_threshold`, which helps us prevent documents with too low similarity scores from being selected.

As a result, from a user query, we expect to get a list of documents that the model judges to be relevant.

Chapter 5

Model Implementation

5.1 Data Construction

To implement this project, for each language, we need to construct a corpus dataset, as well as cross-lingual data that pairs the low-resource language with a rich-resource language.

5.1.1 Language Corpus Datasets

To obtain the necessary data described, we used the Wikipedia dataset available on Hugging Face¹, a widely-used tool for managing large datasets in machine learning projects, to extract the required language data.

Wikipedia dataset containing cleaned articles of all languages, which means the mark-down and unwanted sections (such as references, etc.) has been stripped. The dataset is built from the Wikipedia dumps² with one subset per language, each containing a single train split. The data fields are the same among all configurations, specifically,

- id (str): ID of the article,
- url (str): URL of the article,
- title (str): Title of the article,
- text (str): Text content of the article.

The Vietnamese subset and Khmer subset contains about 12,000 rows and 1.29M rows respectively, out of 61,614,907 total rows of the whole dataset.

To load the dataset, we utilized the *datasets* library, an open-source Python library developed by Hugging Face, designed to simplify the process of accessing, sharing, and

¹<https://huggingface.co/datasets/wikimedia/wikipedia>

²<https://dumps.wikimedia.org/>

working with large datasets in machine learning projects. This approach is simple because the dataset is also accessible on Hugging Face.

After loading the dataset, we use the `gpt-4o` model to initially filter articles related to topics as discussed (economy documents for Vietnamese and cultural documents for Khmer). Then, we manually filter them once again to ensure the quality of the datasets. Finally, we successfully construct the Vietnamese Economic Document dataset with 2002 entries and the Khmer Cultural Document dataset with 1983 entries.

In the next step, we follow a similar approach as seen in [29] but with little adjustments. We determined that each first sentence in a document is the query, after which the title words are removed from it. For instance, the first sentence of the article **Inflation** is **Inflation affects purchasing power**, so the query becomes **affects purchasing power**. The intuition to choose the query is that the first sentence is usually a well-defined summary of its corresponding article, and the deletion prevents the task from becoming an easy keyword matching task [29]. However, we also aim to observe the impact of a strange sentence on the model. As a result, we remove the first sentence from each document to create another dataset.

All of the information is then combined into a single JSON file, configured with the fields

- `id (int)`: ID of the article,
- `url (str)`: URL of the article,
- `title (str)`: Title of the article,
- `text (str)`: Text content of the article,
- `query (str)`: Text of the query

In conclusion, we have established 4 datasets for the further training and testing phases.

5.1.2 Language Corpus Datasets Visualization

Except for the stopwords, we want to know which words appear most frequently in the texts. Therefore, we plot bar chart for these vocabulary words. VNCORENLP library has been used to identify Vietnamese vocabulary, because Vietnamese does not simply use spaces to separate words (for example, *tài chính* is a word). Similarly, due to the characteristic of Khmer, the Khmer-NLTK library has been used to recognize Khmer vocabulary.

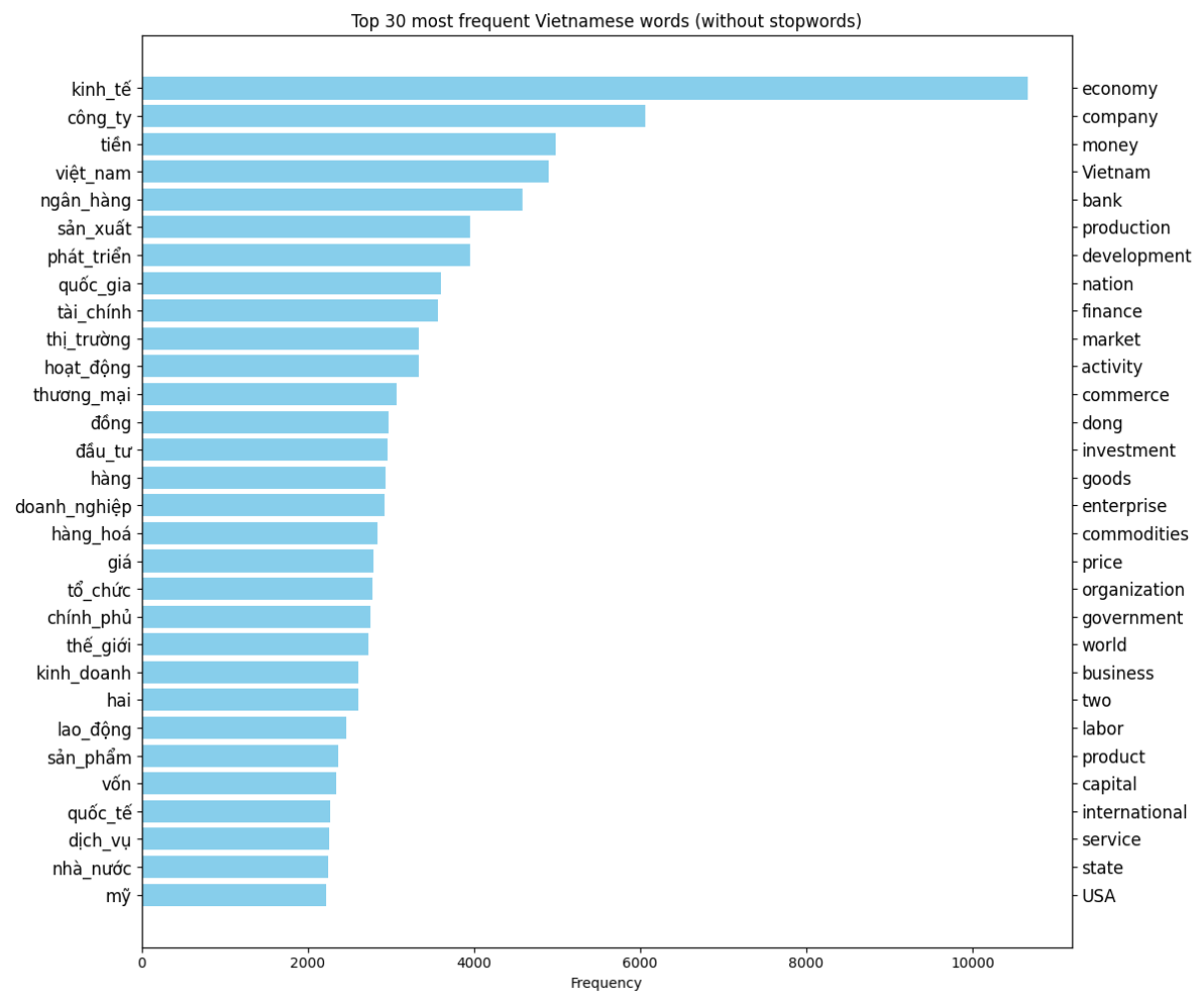


Figure 5.1: Top 30 most frequent Vietnamese words without stopwords with translation

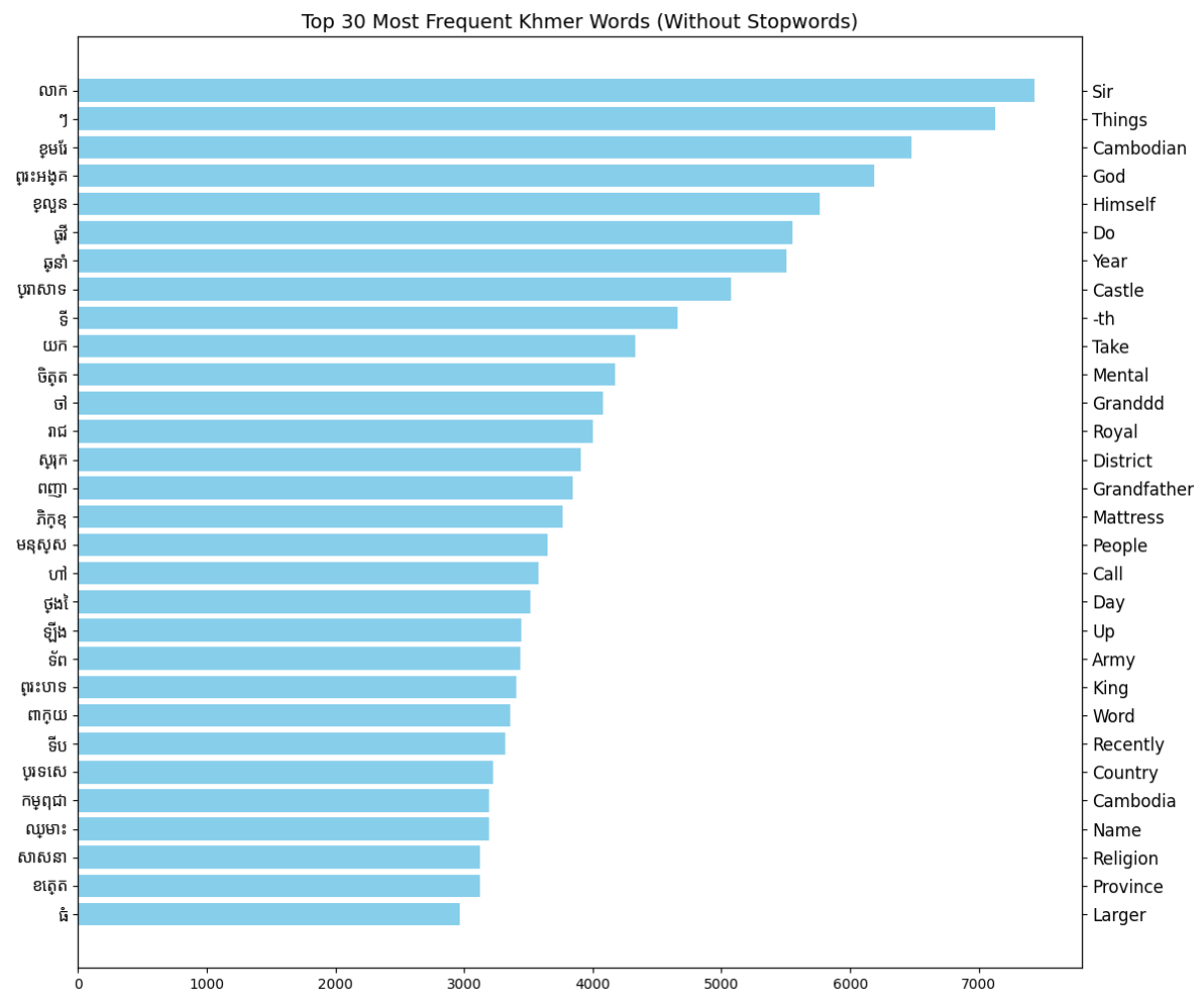


Figure 5.2: *Top 30 most frequent Khmer words without stopwords with translation*

From the chart, we can observe that the vocabularies align well with the topics we have identified earlier, Economics for Vietnamese documents and Culture for Khmer documents. It demonstrates that the content is consistent with our expectations.

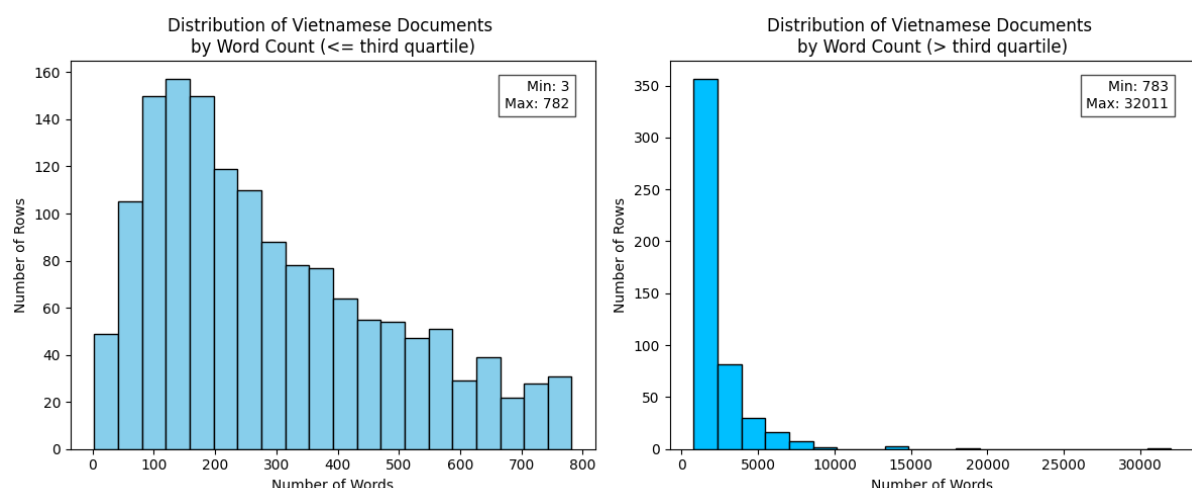


Figure 5.3: *Top 30 most frequent Vietnamese words without stopwords with translation*

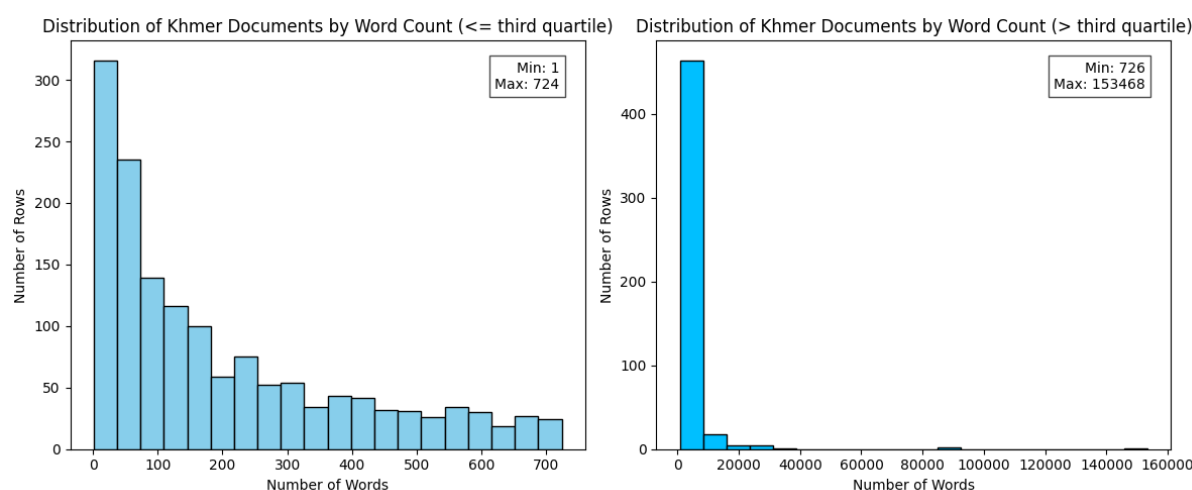


Figure 5.4: *Top 30 most frequent Khmer words without stopwords with translation*

The distribution of word counts in Vietnamese and Khmer documents reveals notable differences. For Khmer, most of the documents are very short, with a strong right-skew in the distribution and many entries containing fewer than 200 words. Vietnamese documents do not show a strong right-skewed distribution as Khmer documents. They instead appear more balanced, with a smoother decline. On the other hand, these distributions still share a common feature - a long tail following. Nevertheless, we do not consider them as outliers, because in practice, many articles or documents contain a large number of words. This contributes to the diversity of the dataset.

5.1.3 Cross-lingual Dataset

For acquiring the cross-lingual dataset, we relied on OPUS³, a growing collection of translated texts from the web.

OPUS offers 13 resources for Khmer-English translation. Among these, the NLLB corpus [11] [31] stands out as the largest, containing 5,863,140 sentences, 60,515,848 Khmer tokens, and 66,724,579 English tokens. As a result, we chose this corpus over the other 12 resources.

5.2 Training Pipeline Implementation

5.2.1 Optimal Transport Solver

This implementation provides a method to solve the optimal transport problem using the IPOT approach. As described in the Related Works chapter, this algorithm only performs simple matrix operations, so we used the NumPy library. Additionally, to compute the cost matrix, we utilized the OT library, an open source Python library provides several solvers for Optimal Transport problems.

Apart from the steps mentioned in the algorithm, this implementation also supports warm-start initialization for efficient convergence. Moreover, it offers an option (enabled by a boolean parameter) to track the loss progression, which allows us to detect abnormalities more easily.

5.2.2 Knowledge Distillation Class

This class is used to distill knowledge, specifically token embeddings, from the teacher sentence transformer to the student sentence transformer, which will later be used in the monolingual retrieval.

The class first take in a parallel corpus of sentence pairs, one in the teacher model’s language and one in the student model’s language. Each training loop reads in one sentence pair from the corpus. The optimal transport problem is then formulated as described in Subsection 4.2.1. The class also provides two options for token distribution, the uniform distribution as in [40] and the $tf-idf$ based distribution. After the optimal transport problem is defined, the Optimal Transport Solver class is used to solve for the transportation plan and loss. The loss is then used to update the student model’s parameters through backpropagation, while the transportation plan can be printed to track mapping of tokens between the two sentences.

³<https://opus.nlpl.eu/>

5.2.3 Query Expansion

The `QueryExpansion` class is implemented using the Expectation-Maximization (EM) algorithm to operate on document collections to extract relevant terms for query expansion.

The implementation defines several important numerical constants:

- `LIMIT_K_DOCS_FOR_RELEVANT_SET = 10`: Number of top-ranked documents used for expansion. If this value is too high, it will affect the speed when constructing the relevant set, especially if the document is long. The paper uses this number in their experiment, so we decide to keep it.
- `NUMBER_OF_EXPANSION_TERM = 30`: Maximum number of terms to add to the original query. If too much terms are added, it is possible that some terms with negligible relevance will also be added unintentionally. The paper also uses this number in experiment, so we also not change it.
- `THRESHOLD_FOR_EM_ALGO = 0.000001`: Convergence threshold for the EM algorithm.

The constructor takes the input text dataset, separating the title and content into separate sets to be used as two source sets (collection set - title and collection set - content mentioned in Subsection 4.2.2) for query expansion. The class also initializes the collection set from all the terms that appear in the entire document set. A `BM25Plus` ranker object from the `rank_bm25`⁴ library is then initialized using the entire document set to be used for lexical relevance-based retrieval. Three probability dictionaries (P_{jk} , π_j and Z_{ij} mention in Subsection 4.2.2) are also initialized with uniform initial distributions, creating the necessary probability framework for the EM algorithm.

When a new query requires expansion, the system executes a two-phase process:

First, in the retrieval phase, any data from previous queries is cleared, and the `BM25Plus` ranker computes relevance scores between the query and all documents in the collection. The top `LIMIT_K_DOCS_FOR_RELEVANT_SET` highest-scoring documents are selected, extracts the titles and contents separately to form the remaining two source sets (relevant set - title and relevant set - content mention in Subsection 4.2.2).

Next, in the expansion phase, the system applies the EM algorithm (mention in Subsection 4.2.2) iteratively to both the title and content relevant terms set. The algorithm converges when the log likelihood change falls below the `THRESHOLD_FOR_EM_ALGO`, at which point expansion term probabilities are finalized.

Finally, terms from the original query are filtered out, and the remaining candidates from both title and content sources are combined, ranked by probability, and the top

⁴<https://pypi.org/project/rank-bm25/>

`NUMBER_OF_EXPANSION_TERM` highest-scoring terms are returned as the final expansion set.

5.2.4 Lexical Matching

The `LexicalMatching` class implements lexical-relevant based retrieval method using the BM25Plus algorithm. This component provides the list of top lexical-relevant document for our query.

The initialization step begins by processing the provided document dataset to extract document components (titles and contents) and their corresponding identifiers from the dataset. For each document, the system combines its tokenized title and content to form a unified document representation, which then be used to initialize the BM25Plus ranker. The class also keeps a parallel list of document IDs that preserves the mapping between the internal BM25Plus document indices and their original identifiers.

When a query requires document retrieval, we provide the tokenized query to the `get_documents_ranking` method. Then we use the BM25Plus object we initialized to perform BM25Plus algorithm to compute similarity scores between the query and every document in the collection we provided. These raw numerical scores represent the lexical similarity between each document and the query. We then sorts these scores in descending order to identify the most relevant documents. For each document in this ranked order, we retrieves the corresponding document identifier from the parallel ID list and pairs it with the document's relevance score. The final output is a sorted list of (`document_id`, `score`) pairs, representing the most relevant documents for the query according to the lexical similarity.

5.2.5 Chunk Separator

This class is designed to divide long documents into smaller, manageable chunks while preserving the coherence of the content.

The main functionality of the class lies in its method to separate chunks from documents. This method processes a single document, defined by its ID, and splits it into chunks based on the token length constraint. First, the document is retrieved from the dataset by ID, and we take the title and content in a preprocessed form (split into sentences). The sentences are processed iteratively, with a cumulative count of tokens maintained to determine when a chunk exceeds the specified limit. When the limit is reached, the accumulated sentences are combined into a chunk and a new chunk is started. This ensures that each chunk remains coherent, as it consists of full sentences rather than arbitrary slices of text.

5.2.6 Custom Sentence Transformer

Our `CustomSentenceTransformer` class is implemented using PyTorch to facilitate tensor operations and model construction, alongside the Sentence Transformer framework, which specializes in generating embeddings for sentences, provided by Hugging Face⁵.

The Custom Sentence Transformer has a method to calculate the similarity score between the query and a document. The method takes in the chunks of the document, which has been separated by the Chunk Separator component previously. Document chunks are processed in batches, where each batch is converted into embeddings using the document-specific model. The embeddings are compared to the query vector using cosine similarity. A ReLU activation ensures non-negative similarity values, which are further transformed into probabilities. The similarity scores of all chunks in a document are aggregated into a single score, representing the document’s overall semantic relevance to the query.

For the forward pass, the class incorporates the “forward” method to combine semantic similarity with pre-computed lexical or topic similarity scores. These scores are passed through a linear layer followed by a sigmoid activation function to compute the final similarity score for each document.

5.2.7 Monolingual Trainer Class

The `MonolingualRetrievalTrainer` class implements our proposed information retrieval training pipeline that combines query expansion, lexical matching, and semantic searching with a multi-stage training strategy to optimize document ranking performance.

The implementation using these training parameters:

- `margin = 1.0`: Controls separation between positive and negative examples in contrastive loss. With this high value, dissimilar pairs must be pushed further apart to avoid loss, which makes the model becomes stricter about separating dissimilar pairs.
- `chunk_length_limit = 128`: Maximum token count per document chunk, balancing context and efficiency.
- `learning_rate = 1e-5`
- `batch_size = 32`
- `epochs = 4`

For the multi-stage training step, we keep the parameters that has been used in [24]:

⁵<https://huggingface.co/sentence-transformers>

- `FIRST_ROUND_NEGATIVE_SAMPLE_COUNT = 35`: In this stage, the selected negative samples are high lexical similarity to queries but tend to be low in semantic similarity.
- `SECOND_ROUND_NEGATIVE_SAMPLE_COUNT = 20`: These negative samples picked in this stage usually have better semantic similarity to queries.
- `THIRD_ROUND_NEGATIVE_SAMPLE_COUNT = 15`: In this stage, negative samples chosen are really close semantic to queries.

The `train` method processes batches of query-document pairs through epoch-based iterations, with each batch will be chosen randomly and be shuffled automatically with the help of `DataLoader` class in PyTorch to avoid overfitting. For each query-document pair in each batch, the method executes a training workflow:

- **Enhancement**: This is the query expansion process described in Subsection 4.2.2 and implemented in the class `QueryExpansion` class.
- **Lexical Matching**: Described in Subsection 4.2.3 and implemented in `LexicalMatching` class. Both version of the query (original and extended) are used to retrieve independently.
- **Positive/Negative samples generation and combining**: From independent result of original and expanded queries, we get the relevant document and label it 1, `FIRST_ROUND_NEGATIVE_SAMPLE_COUNT` number of non-relevant documents and label them 0, to form the sample list for each type of query. From these two separate list, we combine to form the final sample list.
- **Semantic Searching**: We trained 3 rounds for sentence transformer describe in Subsection 4.2.4 and implement in `CustomSentenceTransformer` class.

After training completion, the system saves:

- The complete `CustomSentenceTransformer` model including the Sentence Transformer model and the linear layer.
- The `CustomSentenceTransformer` model part for future distillation if needed.

5.3 Retrieval Model Implementation

The `MonoLingualRetrival` class implements a production-ready hybrid retrieval system that combines lexical and semantic approaches for our proposed information retrieval model.

We have thoroughly described the model in Section 4.3, so in here we only mention about configuration parameters we have used:

- `original_query_doc_count = 30`: Define how many documents should be retrieved from the lexical-retrieval phase, using the original query. The higher this number is, the more documents will be passed to the semantic-retrieval phase, which used Sentence Transformer model that can be very resource-consuming. But low value means there is a risk of omitting the true relevant document. So we choose this number as this is the number used in Query Expansion technique, which we considered appropriate for production-model too.
- `extended_query_doc_count = 30`: Define how many documents should be retrieved from the lexical-retrieval phase, using the extended query. The reason we choose this number is similar to above.
- `relevant_threshold = 0.1`: Maximum score difference from top document for inclusion. The documents would be chosen if their similarity scores in range `[max_relevant_score - relevant_threshold, max_relevant_score]`.
- `relevant_default_lowerbound = 0.25`: The minimum similarity score required for a document to be chosen. If this value is higher than `(max_relevant_score - relevant_threshold)`, this value will be chosen as the lower bound instead of `max_relevant_score - relevant_threshold`.
- `chunk_length_limit = 128`
- `batch_size = 32`

Chapter 6

Experiment and results

6.1 Experiment Setup

6.1.1 Resource

Kaggle¹ is a great AI & ML community for machine learners share, stress test, and stay up-to-date on all the latest ML techniques and technologies, offering access to datasets, notebooks, etc. For training machine learning models, GPU acceleration is essential to speed up computation. By utilizing Kaggle’s free GPU resources, we aim to train the models discussed efficiently.

Kaggle provides both GPU P100 and GPU T4x2, and we have opted for the GPU T4x2. The reason for this choice is its advantages in optimized for mixed-precision training and parallel processing capabilities. While the P100 also supports mixed-precision, the T4’s architecture offers improved performance in this area. Additionally, the T4x2 setup provides two GPUs, enabling parallelism that can further enhance training efficiency. This dual-GPU configuration allows for larger batch sizes and more effective utilization of resources during model fine-tuning, particularly beneficial for transformer-based models. Therefore, the T4x2 setup aligns well with our computational needs and the specific requirements of our training tasks.

6.1.2 Training and Testing Phase

For each model, we split the entire dataset into 80% for training and 20% for testing. To ensure the reproducibility, we set the random seed to 42, which is a popular seed number in machine learning.

For training phase, after many trials and errors, we observe that training on the entire set takes longer than the time limit of a running session. Therefore, we have to come up with another idea that splitting the entire training set into several subsets and training on

¹<https://www.kaggle.com/>

them one by one. This means training on the first subset, then using the results to train on the second subset, and so on. We conduct experiments in two conditions: one where the first sentence of each document is retained, with each subset having 200 samples, and one where the first sentences are removed, with each subset having 400 samples. The reason is that with our implementation, the larger the subset, the more the model shuffles the documents. So with larger subsets, the results are more reliable, making them more suitable for the more difficult case without the first sentence, compared to the case with the first sentence.

For testing phase, we have to use the entire language documents dataset because we want to retrieve among all of the documents.

During the training phase, we only train with the processed queries as discussed in the previous chapter. However, in the testing phase, we test with both the prepared queries and the title to see if our proposed model works well in diverse cases.

6.2 Evaluation Metric

In this evaluation setting, each query in our dataset is associated with only one known relevant (true positive) document. This constraint makes it unsuitable to apply standard ranking metrics such as Precision@k, Recall, or Mean Average Precision (MAP) as in [40], which require multiple relevant documents per query to produce meaningful scores. As a result, we adopt Mean Reciprocal Rank (MRR) as the primary evaluation metric, since it only requires the rank of the first correct result, making it appropriate for scenarios where only a single relevant answer is available for each query. Moreover, Accuracy@1 (Acc@1) and Accuracy@5 (Acc@5) are also utilized to evaluate our model, to assess how often the correct result appears at the top or within the top 5 retrieved documents.

The Reciprocal Rank (RR) calculates the reciprocal of the rank at which the first relevant document was retrieved. When averaged across queries, the measure is called the Mean Reciprocal Rank (MRR) [7].

The Mean Reciprocal Rank is the average of the reciprocal ranks of results for a sample of queries Q

$$\text{MRR} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i}$$

where rank_i refers to the rank position of the first relevant document for the i -th query.

Example 6.2.1. Suppose we have the following two queries for a system. In each case, the system makes predictions, sorted by probability.

Query A:

- Proposed Results: B, C, D

- Correct Response: D
- Rank: 3 $\Rightarrow$ Reciprocal Rank: $\frac{1}{3}$

Query E:

- Proposed Results: F, G, H
- Correct Response: F
- Rank: 1 $\Rightarrow$ Reciprocal Rank: 1

The Mean Reciprocal Rank is calculated as $\frac{1}{2} \left(\frac{1}{3} + 1 \right) = \frac{2}{3}$.

6.3 Experiment Results

6.3.1 Training Loss

In this section, we want to discuss about the loss during training phases of each case mentioned, to give some preliminary comments about our model.

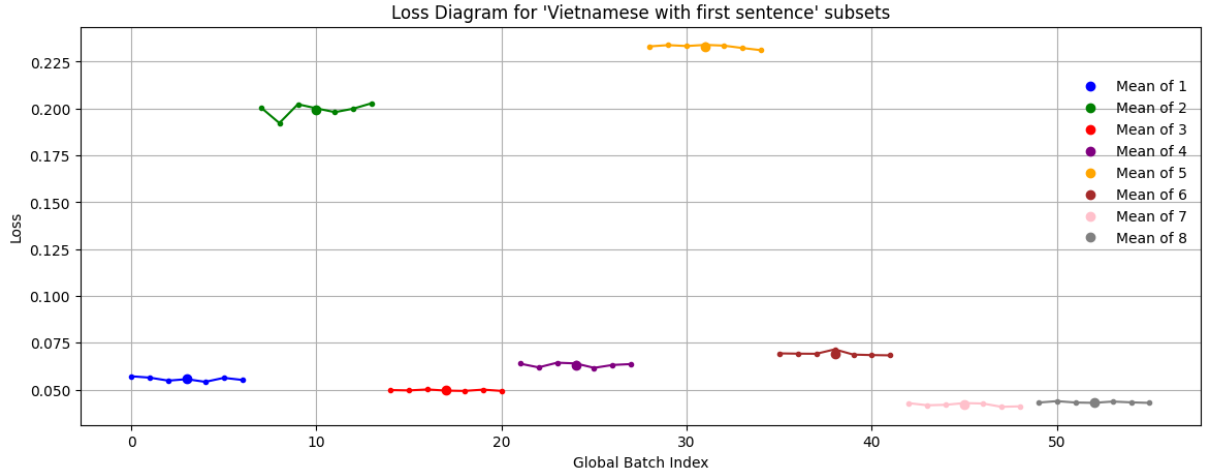


Figure 6.1: Training loss for ‘Vietnamese with first sentence’ subsets

In this case, the loss is generally low and consistent across subsets, especially for the later subsets (7 and 8), with the exception of the second and fifth subsets. Moreover, the results show little variation within each subset.

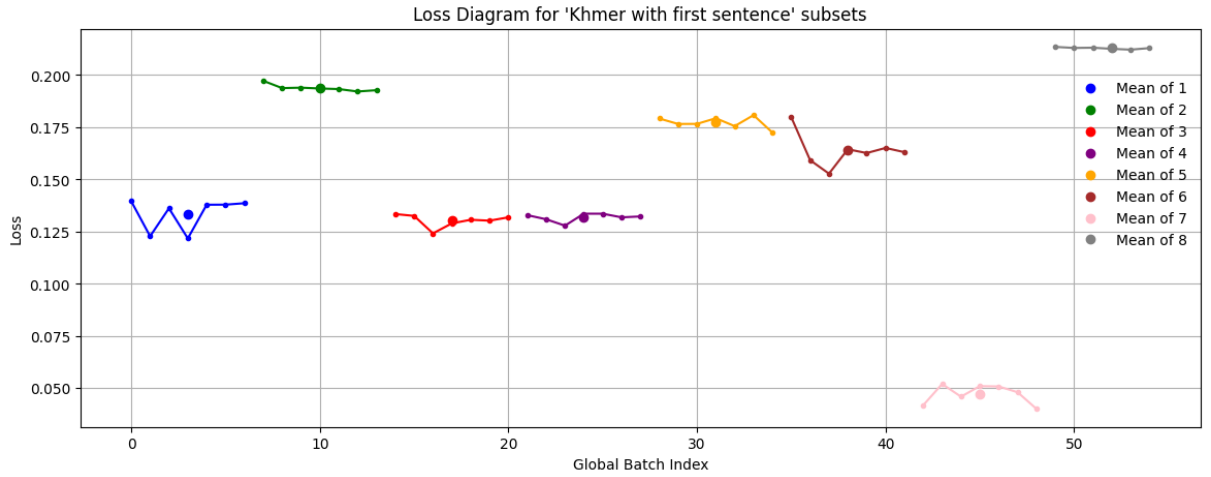


Figure 6.2: Training loss for ‘Khmer with first sentence’ subsets

In this case, the loss is higher and more scattered compared to Vietnamese, and each subset also shows high variation.

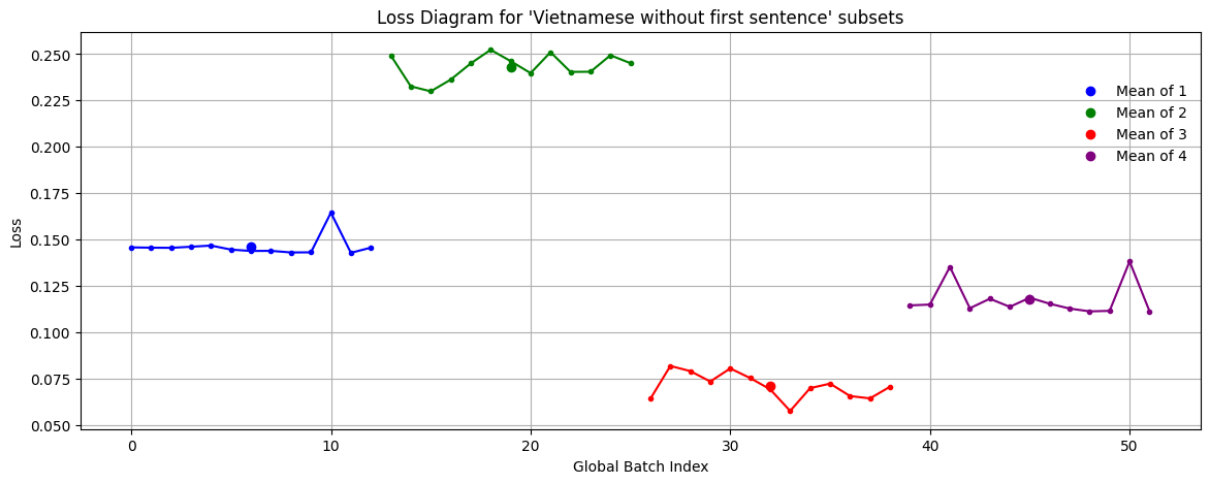


Figure 6.3: Training loss for ‘Vietnamese without first sentence’ subsets

In this case, the loss is noticeably higher and more inconsistent than in the ‘with first sentence’ case, with each subset also showing high variation.

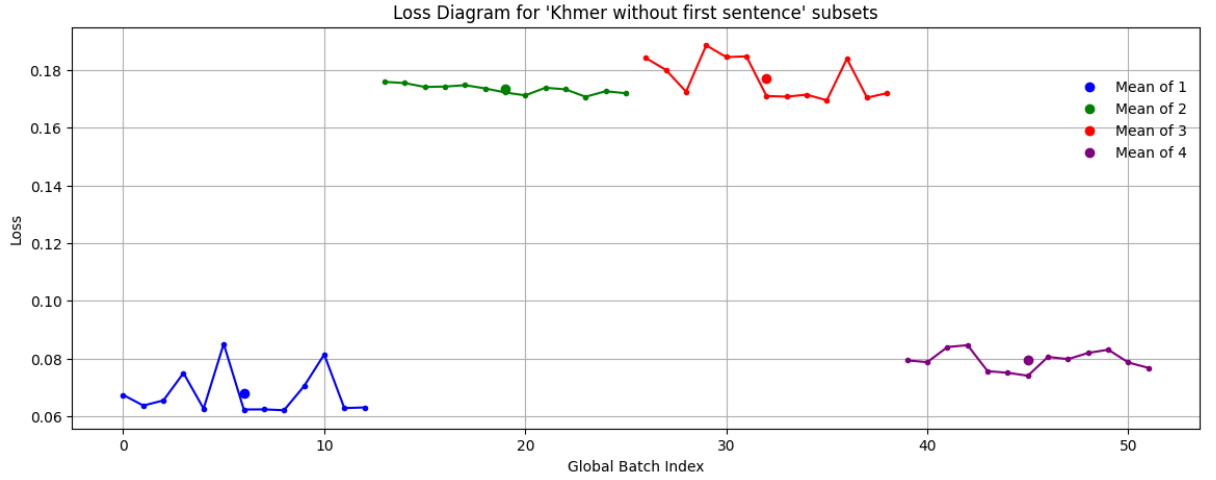


Figure 6.4: Training loss for ‘Khmer without first sentence’ subsets

In this case, the pattern appears similar to the previous Vietnamese case.

In summary, with these loss diagrams, we can learn some remarkable things about our model that the first sentence may plays a critical role, as it reduces loss and variation within a subset. Moreover, the training phase can be considered acceptable because the late subset generally give more stable and lower loss.

6.3.2 Final Results

Table 6.1: Performance results

Retrieved documents	Input query	Vietnamese			Khmer		
		MRR	Acc@1	Acc@5	MRR	Acc@1	Acc@5
Original documents	Query	0.9413	0.9202	0.9626	0.7922	0.7280	0.8741
	Title	0.8893	0.8155	0.9776	0.7878	0.7003	0.9068
Documents without first sentence	Query	0.5106	0.4264	0.6085	0.6072	0.5592	0.6725
	Title	0.8669	0.7830	0.9676	0.7556	0.6650	0.8766

The table shows that the retrieval models generally perform well. For Vietnamese, using the processed query yields the highest performance with MRR of 0.9413, Accuracy@1 of 0.9202, and Accuracy@5 of 0.9626. When using titles, the model also performs well with MRR of 0.8893 and a notable Accuracy@5 of 0.9776. Khmer shows slightly lower but still strong performance, with the query input achieving MRR of 0.7922, Accuracy@1 of 0.7280, and Accuracy@5 of 0.8741. Using titles for Khmer gives a similar result with MRR of 0.7878.

The special thing is in the documents without the first sentence. All metrics decline, the retrieval performance drops noticeably when using queries as input, while using titles give just slightly lower effectiveness. When using prepared queries as inputs for Vietnamese,

MRR decreases 0.5106, while Accuracy@1 and Accuracy@5 drop to 0.4264 and 0.6085, respectively. Khmer shows a similar trend, MRR is 0.6072, Accuracy@1 is 0.5592 and Accuracy@1 is 0.6725. When using titles as input, for Vietnamese, MRR is still high at 0.8669, Accuracy@1 and Accuracy@5 remains strong at 0.7830 and 0.9676, respectively. Khmer also achieves acceptable performance with MRR of 0.7556, Accuracy@1 of 0.6650 and Accuracy@5 at 0.8766.

For the explanation of these results, reviewing the datasets, we observe that there are some the documents are not typically written in deductive format, which means there is an introductory sentence at the beginning, the following supporting sentences explain that first sentence. Instead, the sentences in each paragraph have similar meanings, with no sentence being more important than the others. Therefore, if a document is written in this format, the query and the remaining content may share nothing in common, which leads to poor retrieval performance. About the title input, the high metrics suggest that the titles continue to provide sufficient semantic meaning within the content of the documents.

Chapter 7

Conclusion

7.1 Achieved Results

Throughout this project, we have been exposed to new mathematical knowledge. We have grasped the definitions and properties of Topology, Measure theory and Optimal Transport. We were also able to prove these properties and provide specific examples to illustrate the concepts and properties we have learned. Furthermore, we have reviewed and reinforced our knowledge in the field of computer science, including Linear programming, Transformer, Expectation-Maximization algorithm. And by leveraging these knowledge, we have developed an information retrieval model capable of retrieving relevant documents for queries in both Khmer and Vietnamese, despite the scarcity of annotated training data and linguistic resources. The model uses knowledge distillation, query expansion, and semantic scoring to effectively retrieve relevant content. It is optimized for efficiency and can be deployed on accessible hardware such as T4 GPUs.

To promote transparency, we have published our project’s source code on Github¹. This work lays the foundation for building effective retrieval systems in low-resource languages, providing an open-source baseline for further advancement in multilingual and low-resource IR research.

7.2 Limitations

While the proposed system demonstrates promising results in low-resource language retrieval, several limitations remain that impact its scalability and generalizability.

First, calculating similarity scores between a query and all document chunks, presents a practical bottleneck. This limitation becomes more pronounced as the corpus size grows, restricting the model’s responsiveness in real-time applications or large-scale deployments.

¹Project repository: <https://github.com/sitrongtrang/LRL-IR>

Second, the evaluation dataset is constrained to one relevant document per query. As a result, the evaluation uses Mean Reciprocal Rank, which, while simplifies evaluation and reflects certain retrieval scenarios, limits the ability to assess recall or performance in multi-answer settings. This affects the model’s applicability in broader information retrieval tasks that demand multiple relevant documents.

Third, the model currently supports only two low-resource languages, Vietnamese and Khmer, and within restricted domains. While the architecture is designed to be extensible, adapting it to new LRLs requires a non-trivial amount of linguistic knowledge and preprocessing customization. Languages with different scripts, tokenization behavior, or morphological complexity may demand additional effort to ensure effective retrieval.

These constraints, while addressed partially through design decisions, point to important directions for future improvement and broader applicability of the system.

7.3 Future Research Directions

To address the current limitations, we propose several future directions that can help improve the model’s performance and broader applicability to low-resource language contexts.

To reduce the computational overhead in the semantic searching phase, future work could explore efficient approximate similarity methods such as dense vector quantization, locality-sensitive hashing (LSH), or dual-encoder architectures that allow pre-indexed vector search. This would significantly reduce inference time while maintaining retrieval accuracy.

Expanding the dataset to include multiple relevant documents per query would also enable the use of richer evaluation metrics like Recall@k or Mean Average Precision. This improvement would not only offer deeper insight into model behavior but also more closely reflect real-world information-seeking scenarios.

To make the model more language-agnostic, future research could focus on developing modular pipelines for adaptation to new low-resource languages. This could include tools for automated morphological analysis, unsupervised tokenization, or leveraging multilingual teacher models for language-agnostic knowledge transfer. Reducing the reliance on manual linguistic insight would lower the barrier for deploying retrieval systems across many underrepresented languages.

Finally, the system could benefit from learning with user feedback, where user interactions help refine the relevance model over time. Such feedback loops would not only enhance retrieval quality but also ensure the system evolves alongside language usage and domain shifts, particularly in dynamic cultural contexts.

These directions aim to transform the proposed model from a prototype into a scalable,

inclusive, and adaptive platform for linguistic empowerment and digital equity in low-resource communities.

Bibliography

- [1] T. Babinec, C. Best, M. Bliss, N. Brendler, E. Fu, A. Fung, T. Klein, A. Larson, T. Lee, J. Madonna, J. Mousseau, N. Posavetz, M. Rosenberg, D. Rogers, A. Sardone, J. Shaler, S. Srinivasan, P. Troyan, J. Yim, E. Uible, D. V. Farowe, P. Warmker, Z. Wu, and N. Zhang. Introduction to topology. <https://www.math.colostate.edu/~renzo/teaching/Topology10/Notes.pdf>, 2007.
- [2] C. Buckley, G. Salton, J. Allan, and A. Stinghal. Automatic query expansion using smart. In *Proceedings of the 3rd Text Retrieval Conference*, pages 69–80, 1994.
- [3] M. Charikar, B. Chen, C. Re, and E. Waingarten. Fast algorithms for a new relaxation of optimal transport, 2023.
- [4] M. K. Chinnakotla, K. Raman, and P. Bhattacharyya. Multilingual prf: english lends a helping hand. *Proceedings of the 33rd international ACM SIGIR conference on Research and development in information retrieval*, 2010.
- [5] M. K. Chinnakotla, K. Raman, and P. Bhattacharyya. Multilingual pseudo-relevance feedback: Performance study of assisting languages. In *Proceedings of the 48th Annual Meeting of the Association for Computational Linguistics*, pages 1346–1356. Association for Computational Linguistics, 2010.
- [6] N. Courty, R. Flamary, D. Tuia, and A. Rakotomamonjy. Optimal transport for domain adaptation, 2016.
- [7] N. Craswell. *Mean Reciprocal Rank*, pages 1703–1703. Springer US, 2009.
- [8] M. Cuturi. Sinkhorn distances: Lightspeed computation of optimal transportation distances, 2013.
- [9] A. P. Dempster, N. M. Laird, and D. B. Rubin. Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society: Series B*, 39:1–38, 1977.
- [10] A. Dessein, N. Papadakis, and J.-L. Rouas. Regularized optimal transport and the rot mover’s distance, 2018.

- [11] A. Fan, S. Bhosale, H. Schwenk, Z. Ma, A. El-Kishky, S. Goyal, M. Baines, O. Celebi, G. Wenzek, V. Chaudhary, N. Goyal, T. Birch, V. Liptchinsky, S. Edunov, E. Grave, M. Auli, and A. Joulin. Beyond english-centric multilingual machine translation, 2020.
- [12] R. Hadsell, S. Chopra, and Y. LeCun. Dimensionality reduction by learning an invariant mapping. In *2006 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'06)*, volume 2, pages 1735–1742, 2006.
- [13] G. Hinton, O. Vinyals, and J. Dean. Distilling the knowledge in a neural network, 2015.
- [14] Y. Ida, S. Kanai, K. Adachi, A. Kumagai, and Y. Fujiwara. Fast regularized discrete optimal transport with group-sparse regularizers, 2023.
- [15] Y. Ji, Z. Li, R. Meng, and D. He. Reasoningrank: Teaching student models to rank through reasoning-based knowledge distillation, 2024.
- [16] S. Kim, A. S. Rawat, M. Zaheer, S. Jayasumana, V. Sadhanala, W. Jitkrittum, A. K. Menon, R. Fergus, and S. Kumar. Embeddistill: A geometric knowledge distillation for information retrieval, 2023.
- [17] W. Lu, J. Jiao, and R. Zhang. Twinbert: Distilling knowledge to twin-structured bert models for efficient retrieval, 2020.
- [18] G. Mahey, L. Chapel, G. Gasso, C. Bonet, and N. Courty. Fast optimal transport through sliced wasserstein generalized geodesics, 2023.
- [19] M. Mitra, A. Singhal, and C. Buckley. Improving automatic query expansion. In *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, page 206–214. Association for Computing Machinery, 1998.
- [20] B. Nelson. The lebesgue integral. https://users.math.msu.edu/users/banelson/files/lebesgue_integral.pdf, 2016.
- [21] N. Reimers and I. Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2019.
- [22] L. Pang, Y. Lan, J. Guo, J. Xu, and X. Cheng. A study of matchpyramid models on ad-hoc retrieval, 2016.
- [23] L. Pang, Y. Lan, J. Guo, J. Xu, S. Wan, and X. Cheng. Text matching as image recognition, 2016.

- [24] N.M. Pham, H.T Nguyen, and T.H. Do. Multi-stage information retrieval for vietnamese legal texts, 2022.
- [25] N. Reimers and I. Gurevych. Making monolingual sentence embeddings multilingual using knowledge distillation. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2020.
- [26] S. Robertson, H. Zaragoza, and M. Taylor. Simple bm25 extension to multiple weighted fields. *International Conference on Information and Knowledge Management, Proceedings*, pages 42–49, 2004.
- [27] R. Rosenfeld. Two decades of statistical language modeling: where do we go from here? *Proceedings of the IEEE*, 88(8):1270–1278, 2000.
- [28] G. Salton and C. Buckley. Term-weighting approaches in automatic text retrieval. *Information Processing Management*, 24(5):513–523, 1988.
- [29] S. Sasaki, S. Sun, S. Schamoni, K. Duh, and K. Inui. Cross-lingual learning-to-rank with shared representations. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, pages 458–463. Association for Computational Linguistics, 2018.
- [30] F. Schroff, D. Kalenichenko, and J. Philbin. Facenet: A unified embedding for face recognition and clustering. In *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, page 815–823. IEEE, 2015.
- [31] H. Schwenk, G. Wenzek, S. Edunov, E. Grave, and A. Joulin. Ccmatrix: Mining billions of high-quality parallel sentences on the web, 2020.
- [32] M. Thorpe. Introduction to optimal transport. https://www.damtp.cam.ac.uk/research/cia/files/teaching/Optimal_Transport_Notes.pdf, 2018.
- [33] A. Trotman, A. Puurula, and B. Burgess. Improvements to bm25 and language models examined. In *Proceedings of the 19th Australasian Document Computing Symposium*, page 58–65. Association for Computing Machinery, 2014.
- [34] A. A. V, Y. Kakde, P.Bhattacharyya, and G.Ramakrishnan. Structure cognizant pseudo relevance feedback. In *Proceedings of the Sixth International Joint Conference on Natural Language Processing*, pages 982–986. Asian Federation of Natural Language Processing, 2013.

- [35] A. A. V, A. Kankaria, P. Bhattacharyya, and G. Ramakrishnan. Query expansion in resource-scarce languages: A multilingual framework utilizing document structure. *ACM Trans. Asian Low-Resour. Lang. Inf. Process.*, November 2016.
- [36] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need, 2023.
- [37] Y. Xie, X. Wang, R. Wang, and H. Zha. A fast proximal point method for computing exact wasserstein distance, 2019.
- [38] C. Zhai and J. Lafferty. Model-based feedback in the language modeling approach to information retrieval. In *Proceedings of the Tenth International Conference on Information and Knowledge Management*, page 403–410. Association for Computing Machinery, 2001.
- [39] L. Zhao, R. Zbib, Z. Jiang, D. Karakos, and Z. Huang. Weakly supervised attentional model for low resource ad-hoc cross-lingual information retrieval. In *Proceedings of the 2nd Workshop on Deep Learning Approaches for Low-Resource NLP (DeepLo 2019)*, pages 259–264, Hong Kong, China, 2019. Association for Computational Linguistics.
- [40] Z. Huang, P. Yu, and J. Allan. Improving cross-lingual information retrieval on low-resource languages via optimal transport distillation. In *Proceedings of the Sixteenth ACM International Conference on Web Search and Data Mining, WSDM '23*, page 1048–1056. ACM, 2023.